

بسته‌ی مطالعه‌ی LPIC-1 – روز ۱

مبانی لینوکس و معماری سیستم

Linux Overview • Distributions • Kernel • File System Hierarchy

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۲ • روز ۱ (دوشنبه) – مورنینگ ۹:۰۰–۱۳:۰۰ • ایوپنینگ ۱۷:۰۰–۲۱:۰۰

روز ۱ – مبانی لینوکس و معماری سیستم

شروع هفته ۲ – از «شبکه» به «سرور»

مسیر مطالعه‌ی این روز (طبق پلن ۳۰ روزه): مورنینگ: ۱۰:۴۵ → Distributions & Kernel (10:45) → Linux Overview (09:00–10:30) → File System Hierarchy (12:30–13:00) → ایوپنینگ: Lab: نصب لینوکس و اتصال & SSH (17:00–18:30) → Files (18:45–20:15) → File Operations (20:30–21:00)

هدف روز: بفهمی «لینوکس» دقیقاً چیست (کرنل + فضای کاربر)، تفاوت توزیع‌ها را بدانی، درخت فایل‌سیستم را از حفظ بشناسی، سریع ناوبری کنی و عملیات فایل را بدون خطا انجام دهی. این روز پایه‌ی پنج روز بعد است.

0) چرا هفته ۱ به هفته ۲ وصل است؟ (پل زدن بین دو هفته)

هفته ۱ روی روتر و سویچ کار کردی؛ هفته ۲ می‌روی روی سروری که همان شبکه را سرویس می‌دهد. همان مفاهیم، اسم‌های جدید:

مفهوم در هفته ۱ (شبکه)	معادل در هفته ۲ (لینوکس)	یادداشت
IOS روتر (سیستم‌عامل اختصاصی)	کرنل لینوکس + فضای کاربر	در IOS همه‌چیز داخل یک باینری است؛ در لینوکس سرویس‌ها فرایندهای جدا هستند
show ip route	ip addr / ip route	همان مفاهیم، ابزار جدید (روز ۶ هفته ۲)
ACL روی روتر	nftables / iptables روی لینوکس	فیلتر بسته در لایه‌ی میزبان (روز ۶ هفته ۲)
DHCP سرور روی IOS	isc-dhcp-server / dnsmasq روی لینوکس	همان DORA، پیاده‌سازی متفاوت
SSH به روتر	SSH به سرور (sshd_config)	روز ۶ همین هفته
Flash/NVRAM/startup-config	/boot , /etc , دیسک و پارتیشن	روز ۲ همین هفته
copy run start (ذخیره‌ی کانفیگ)	unit files systemd , systemctl	روز ۵ همین هفته

جمله‌ی کلیدی برای مصاحبه:

«شبکه بسته‌ها را می‌رساند، سیستم‌عامل آن‌ها را به سرویس تبدیل می‌کند؛ هر دو باید قابل مشاهده، قابل کانفیگ و قابل ترابل‌شوت باشند.»

۱) مرور لینوکس (09:00–10:30)

۱.۱ لینوکس دقیقاً چیست؟

«لینوکس» در زبان دقیق یعنی **کرنل لینوکس** – هسته‌ای که توسط **لینوس توروالدز** در ۱۹۹۱ منتشر شد. اما وقتی مردم می‌گویند «لینوکس»، منظورشان **سیستم‌عامل کامل** است:

```
سخت افزار
|
v
Kernel (سیسکال‌ها، شبکه، درایورها، فایل سیستم، حافظه، مدیریت پردازنده)
|
v
User space (برنامه‌های کاربر، GNU ابزارهای C، کتابخانه‌ی)
|
v
Application (سرویس‌ی که تو می‌نویسی یا نصب می‌کنی)
```

بخش	وظیفه	اگر خراب شود چه می‌بینی
Kernel	زمان‌بندی پردازنده، مدیریت حافظه، VFS، شبکه، درایور، سیسکال‌ها	کرنال پنیک، فریز، ندیدن دیسک/کارت شبکه
User space	اجرای سرویس، پوسته‌ی فرمان، کتابخانه‌ها	فرمان‌ها کار نمی‌کنند، سرویس بالا نمی‌آید ولی سیستم روشن است

دو نکته‌ای که در مصاحبه امتیاز می‌گیرد:

۱. کرنل **monolithic** است اما قابل بارگذاری در زمان اجرا (ماژول): **lsmod** , **modprobe** , **modinfo** .
۲. کرنل و فضای کاربر با **syscall** حرف می‌زنند (**strace** این حرف‌ها را نشان می‌دهد – روز ۴).

۱.۲ مدل‌های لایسنس: نرم‌افزار آزاد چیست؟

- **GPLv2**: لینوکس تحت این لایسنس است → اگر توزیع کنی، باید سورس را هم بدهی (copyleft).
- **نرم‌افزار آزاد ≠ رایگان**: «آزاد» یعنی آزادی اجرا/مطالعه/تغییر/توزیع، نه صفر بودن قیمت.
- **Open Source** تعریف عملی‌تری از همان ایده‌ها دارد.
- در مصاحبه اگر بپرسند «چرا لینوکس در سرور غالب است؟» → لایسنس بدون هزینه‌ی per-seat، پایداری، اسکرپت‌پذیری، اکوسیستم ابزار، امکان بازرسی کد و قابلیت سفارشی‌سازی.

۱.۳ تاریخچه‌ی ۳۰ ثانیه‌ای (برای سؤال «از لینوکس چه می‌دانی؟»)

سال	رویداد
۱۹۶۹-۷۰	یونیکس در آزمایشگاه‌های بل (کن تامسون، دنیس ریچی)
۱۹۸۳	پروژه‌ی GNU (ریچارد استالمن) – ابزارهای آزاد، ولی کرنل نداشت
۱۹۹۱	کرنل لینوکس 0.01 توسط لینوس توروالدز
۱۹۹۳	Debian متولد شد (اولین توزیع بزرگ با ساختار بسته‌ای)
۱۹۹۴	Red Hat Linux, لینوکس 1.0
۲۰۰۴	Ubuntu (بر پایه‌ی Debian) – لینوکس به دسکتاپ
۲۰۱۱	اولین سالی که لینوکس بیشترین سهم را در بازار سرور داشت
الان	لینوکس در ~۹۰٪ سرورهای وب، همه‌ی ابرهای بزرگ، اندروید و اکثر دستگاه‌های IoT

۱.۴ کاربردهای امروزی و ارتباط با CEH (هفته ۳)

- سرور وب/دیتابیس/ایمیل، هایپروایزر (KVM)، کانتینر (Docker/K8s)، سوپرکامپیوتر، روترهای نرم‌افزاری (VyOS, pfSense).
- برای امنیت: تقریباً همه‌ی ابزارهای CEH (Nmap, Metasploit, tcpdump, Netcat) روی لینوکس اجرا می‌شوند؛ درک لینوکس = درک سطح حمله.

۲) توزیع‌ها و کرنل (10:45–12:15)

۲.۱ توزیع (Distribution) چیست؟

Distro = کرنل + بوت‌لومتر + ابزارهای GNU + سیستم مدیریت بسته + مخازن + نصب‌کننده + پشتیبانی.

خانواده	مثال‌ها	مدیر بسته	ابزار اصلی	مخزن/پشتیبانی
Debian	Debian, Ubuntu, Mint, Kali, Kali Purple	apt + dpkg	.deb	۵ ساله LTS (Ubuntu LTS)
Red Hat	Fedora, CentOS, AlmaLinux, Rocky, RHEL, Stream	dnf + rpm / yum	.rpm	RHCSA/RHCE؛ چرخه‌ی ۱۰ ساله در RHEL
SUSE	SLES, openSUSE Leap/Tumbleweed	zypper + rpm	.rpm	YaST؛ قوی در SAP
Arch	Arch, Manjaro	pacman	rolling release	مستندات عالی (Arch Wiki)
Gentoo	Gentoo	portage (emerge)	کامپایل از سورس	بهینه‌سازی حداکثری
مستقل/سایر	Alpine (apk), Void (xbps), Slackware	–	–	Alpine برای کانتینر (busybox, musl)

تفاوت‌های عملی که در مصاحبه پرسیده می‌شود:

- Ubuntu/Debian در سرور وب و فضای ابری، RHEL/Rocky در سازمان‌های بزرگ و بانک‌ها، Alpine در کانتینرها.

- فرمان نصب: `apt install nginx` در برابر `dnf install nginx` — مکانیزم مشابه، مسیر متفاوت.
- RHEL پول دارد ولی پشتیبانی و گارانتی دارد؛ Rocky/Alma «باینری سازگار» با RHEL هستند.

⚠ **اشتباه رایج مبتدی:** «Kali را روی سرور نصب می‌کنم چون ابزار امنیتی دارد.» Kali یک توزیع **آزمون نفوذ/دسکتاپ** است؛ برای سرور از Debian/Ubuntu Server یا Rocky استفاده کن. (این جواب در مصاحبه‌ی امنیت خیلی خوب می‌نشیند.)

۲.۲ نسخه‌گذاری کرنل

```
4 . 15 . 0 - 200 . fc42 . x86_64
| | | | |
| | | | +-- architecture
| | | +----- distro patch level (Fedora)
| | +----- kernel patch/ABI build
| +----- bugfix release
| +----- major / feature version
+----- (mainline)
```

- `uname -r` → رشته‌ی کامل منتشرشده؛ `uname -a` → همه‌چیز؛ `cat /proc/version` هم همان اطلاعات.
- هسته‌های LTS (مثل ۶.۱، ۶.۶) برای سرور؛ نسخه‌های اصلی هر ۹~۱۰ هفته منتشر می‌شوند.
- در Debian/Ubuntu: `linux-image-6.8.0-31-generic` , `linux-headers-...` (هدرها برای کامپایل درایور لازم‌اند).
- `/boot/vmlinuz-<ver>` تصویر فشرده‌ی کرنل، `/boot/initrd.img-<ver>` یا `initramfs` تصویر رم‌دیسک اولیه است (روز ۵ توضیح کامل بوت).

۲.۳ مازول‌ها، `dmesg` و `/proc` و `/sys`

```
uname -a # معماری و اطلاعات کرنل
lsmod | head # مازول‌های بارگذاری‌شده
modinfo e1000 # (مسیر، پارامترها، لایسنس)
sudo modprobe -r e1000 # حذف مازول
sudo modprobe e1000 # بارگذاری مازول
sudo insmod ./mydrv.ko # بارگذاری دستی فایل مازول
dmesg | tail -30 # پیام‌های حلقه‌ی کرنل (سخت‌افزار، درایورها)
dmesg -T | tail # با زمان خوانا
journalctl -k -b # پیام‌های کرنل این بوت از systemd
cat /proc/cpuinfo # مشخصات پردازنده
cat /proc/meminfo # حافظه
cat /proc/cmdline # آرگومان‌هایی که کرنل با آن‌ها بوت شد
ls /sys/class/net # لیست اینترفیس‌های شبکه
ls pci ; lsusb # سخت‌افزارهای PCI/USB
```

تفاوت `/proc` و `/sys` (سؤال کلاسیک مصاحبه):

- `/proc` یک فایل‌سیستم مجازی است؛ فایل‌ها واقعی نیستند بلکه پنجره‌ای به داخل کرنل‌اند (`/proc/<pid>` , `/proc/meminfo`).
- `/sys` (sysfs) مدل **دستگاه‌ها/درایورها** را نمایش می‌دهد (ساختار سخت‌افزاری، نه فرایندها).
- هر دو روی RAM هستند و جای دیسک اشغال نمی‌کنند.

- `/boot/config-<ver>` → تنظیمات کرنل (`=y` داخل کرنل، `=m` ماژول، `is not set` خاموش).
- `/boot/grub/grub.cfg` → فایل تولیدشده‌ی GRUB؛ هرگز دستی ویرایشش نکن، در `/etc/default/grub` تغییر بده و `update-grub` (Debian) یا `grub2-mkconfig` (RHEL) بزن. (جزئیات کامل در روز ۵).
- از روز ۵ به بعد: `systemd-analyze blame`، `systemctl` — برای ترابل‌شوت بوت.

۳) سلسله‌مراتب فایل‌سیستم (12:30–13:00)

۳.۱ FHS — استاندارد سلسله‌مراتب

مسیر	محتوا	نکته‌ی مصاحبه
<code>/</code>	ریشه	همه‌چیز زیر آن است
<code>/sbin</code> ، <code>/bin</code>	برنامه‌های پایه	در توزیع‌های مدرن symlink به <code>/usr/bin</code> و <code>/usr/sbin</code>
<code>/lib</code> ، <code>/lib64</code>	کتابخانه‌ها و ماژول‌های کرنل	<code>/lib/modules/<ver></code>
<code>/usr</code>	برنامه‌ها و داده‌های کاربری (<code>/usr/bin</code> ، <code>/usr/lib</code> ، <code>/usr</code>) (<code>share</code>)	بزرگ‌ترین بخش سیستم
<code>/usr/local</code>	نرم‌افزاری که خودت کامپایل/نصب کردی	مدیر بسته به آن دست نمی‌زند
<code>/etc</code>	فایل‌های کانفیگ	متنی؛ باید نسخه‌پشتیبان داشته باشند
<code>/var</code>	داده‌ی متغیر: لاگ (<code>/var/log</code>)، صف‌ها، کش، وب	اولین جایی که پر می‌شود
<code>/tmp</code>	فایل موقت، پاک شدن در بوت (روی tmpfs)	مکان خطرناک (perm 1777 + sticky)
<code>/home</code>	پوشه‌ی کاربران	<code>/root</code> جدا برای root است
<code>/root</code>	خانه‌ی root	perm 700
<code>/dev</code>	فایل‌های دستگاه (<code>/dev/tty</code> ، <code>/dev/null</code> ، <code>/dev/sda</code>)	udev آن‌ها را می‌سازد
<code>/sys</code> ، <code>/proc</code>	فایل‌سیستم مجازی	پنجره به کرنل
<code>/boot</code>	کرنل، GRUB، initramfs	در بوت دوباره mount می‌شود
<code>/opt</code>	نرم‌افزارهای جانبی (third-party)	مثل <code>/opt/google/chrome</code>
<code>/srv</code>	داده‌ی سرویس‌ها (ftp، www محلی)	کمتر استفاده می‌شود
<code>/</code> <code>/media</code> ، <code>/mnt</code>	مانت دستی / مانت خودکار رسانه	<code>/mnt</code> برای ادمین، <code>/media</code> برای USB
<code>/run</code>	داده‌ی زمان اجرا (tmpfs): PID، سوکت‌ها	پس از ری‌بوت خالی است
<code>/lost+found</code>	فایل‌های بازیابی‌شده‌ی fsck	فقط در پارتیشن‌های ext

```

pwd # الان کجا هستیم
ls -lah # همه + حجم خوانا + جزئیات
ls -ltr # مرتب بر اساس زمان (جدیدترین آخر)
ls -d */ # فقط پوشه‌ها
cd /etc ; cd ~ ; cd - # رفتن، خانه، برگشت به محل قبلی
cd .. # یک سطح بالاتر
tree -L 2 /etc # (نصب باشد tree) درخت دو سطحی
realpath /usr/bin/vi # symlink مسیر واقعی پس از باز کردن
readlink -f /usr/bin/vi # همان کار
which python3 # مسیر اجرایی در PATH
type ls # alias / (بایستی؟ (مهم!))
command -v ls # alias مسیر بدون در نظر گرفتن

```

مسیرهای مطلق و نسبی: `/var/log/syslog` مطلق است؛ `../../etc/hosts` نسبی. در اسکریپت‌ها همیشه مطلق بنویس تا به `cd` وابسته نباشی.

PATH چیست؟ ترتیب جست‌وجوی فرمان‌ها:

```

echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
export PATH="$PATH:/opt/tools/bin" # افزودن مسیر جدید (موقت)

```

نکته‌ی امنیتی: اگر `PATH` در ابتدای `PATH` باشد، اجرای ناخواسته‌ی فایل مخرب در پوشه‌ی جاری ممکن است – هرگز این کار را نکن.

۴) آزمایشگاه ۱ – نصب لینوکس، ورود و اتصال (17:00–18:30) SSH

۴.۱ انتخاب محیط آزمایشگاه

گزینه	مناسب برای	چرا
Ubuntu Server LTS + VirtualBox / VMware	مبتدی تا متوسط	snapshot برای تست خطرناک، شبکه‌ی NAT/Bridged
Multipass / WSL2	سریع روی ویندوز/مک	سبک، شبکه‌ی مجازی آماده
(Debian/Alpine) Docker/Podman	تمرین فرمان‌ها و اسکریپت	سریع، ولی systemd فعال نیست
VPS ارزان (۱ GB)	تمرین SSH، فایروال، سرویس واقعی	تجربه‌ی واقعی؛ امنیت مهم است
Kali/Parrot (VM جدا)	ادامه در هفته ۳ (CEH)	ابزارهای امنیتی

توصیه برای این برنامه‌ی ۳۰ روزه: یک VM اوبونتو سرور LTS ۲۴.۰۴ + یک VM Rocky/Alma 9 (برای دیدن تفاوت `dnf` و `firewalld`) + در هفته ۳ Kali.

۴.۲ نصب گام به گام (Ubuntu Server 24.04 LTS در VirtualBox)

```
1) دانلود ISO: ubuntu.com/download/server (ubuntu-24.04.x-live-server-amd64.iso)
2) VirtualBox > New:
   Name: srv1 | Type: Linux | Version: Ubuntu (64-bit)
   RAM: 2048 MB (حدافل) | CPU: 2 | Disk: 20 GB (VDI, dynamically allocated)
3) Settings > Network: Adapter 1 = NAT (و Adapter 2 = Host-only اختیاری از میزبان - برای دسترسی از میزبان)
4) Settings > Storage: ISO را در درایو نوری ضمیمه کن
5) Start:
   Language: English | Keyboard: English (US)
   Network: DHCP (پیش‌فرض را نگه دار (بعداً استاتیک می‌کنیم - روز ۶)
   Storage: Use an entire disk + LVM (LVM ۲ تجربه کن - روز ۲)
   Profile: نام کاربری و رمز عبور قوی، نام، نام سرور
   SSH: "Install OpenSSH server" را حتماً تیک بزن
   Snaps: هیچ‌کدام (برای یادگیری، خالی بگذار)
6) sudo reboot و login پس از ریست
```

۴.۳ هویت خود را تثبیت کن

```
whoami # من کی هستم
id # uid, gid گروه ها
hostnamectl # مجازی سازی، OS نام سیستم، نسخه
cat /etc/os-release # توزیع و نسخه (اسکرپت پذیر)
uname -r # نسخه کرنل
uptime # مدت روشن بودن، تعداد کاربر
who ; w # کاربران وارد شده و کاری که می کنند
last | head # تاریخچه ورود
echo $SHELL # پوسته پیش فرض
ip -br a # آدرسها و اینترفیسها
```

تعریف دقیق چند اصطلاح (بیرسند بگو):

- **Terminal** = برنامه‌ای که ورودی/خروجی متنی را نمایش می‌دهد. **Shell** = مفسری که فرمان را می‌خواند و اجرا می‌کند. **Console** = ترمینال مستقیم روی دستگاه. **TTY** = دستگاه ترمینال مجازی (`/dev/tty1`).
- **tty** می‌گوید الان روی چه ترمینالی هستی؛ اگر SSH باشی چیزی مثل `/dev/pts/0` می‌دهد (**pt** = pseudo-terminal).

۴.۴ اتصال SSH از میزبان

```
# روی میزبان (مشتري)
ssh user@192.168.56.10 # ورود با رمز
ssh-keygen -t ed25519 -C "lab-key" # ساخت کلید (روز ۶ کامل توضیح میدهیم)
ssh-copy-id user@192.168.56.10 # کپی کلید عمومی به سرور
ssh user@192.168.56.10 'uptime; ip -br a' # اجرای فرمان بدون ورود تعاملی
scp ./file.txt user@192.168.56.10:/tmp/ # کپی فایل
```

```
SSH روی سرور: وضعیت سرویس #
systemctl status ssh # است ssh نام سرویس Debian/Ubuntu
systemctl status sshd # است sshd نام RHEL/Rocky
ss -tlnp | grep :22 # گوش می‌دهد #
```

اتصال هفته ۱ به هفته ۲: در هفته ۱ با SSH به روتر وصل می‌شدی (ssh admin@10.1.1.1) و ACL می‌زدی تا فقط subnet مدیریت دسترسی داشته باشد. حالا همان کار را «در سمت سرور» می‌کنی و در روز ۶ با sshd_config و firewall/nft سختش می‌کنی.

۴.۵ تمرین‌های خراب‌کن و درست‌کن (Break & Fix)

#	خرابی	علامت	تشخیص	راه‌حل
1	رمز کاربر را عوض کن و اشتباه بزن	ورود ناموفق در <code>lastb</code> / <code>journalctl</code>	<code>sudo journalctl -u ssh -n 20</code>	ورود کنسول، <code>passwd</code> ، رمز را برگردان
2	سرویس SSH را stop کن	Connection refused	<code>systemctl status ssh</code>	<code>sudo systemctl start ssh</code>
3	سرویس را disable کن و ریست کن	بعد از ریست SSH بالا نیست	<code>systemctl is-enabled ssh</code>	<code>sudo systemctl enable --now ssh</code>
4	فایل مجوز کلید را <code>chmod 644</code> کن	Permissions 0644 ... are too open	<code>ls -l ~/.ssh</code>	<code>chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys</code>

۵) ناوبری و فایل‌ها (18:45–20:15)

۵.۱ خواندن فایل‌ها – ابزار درست برای کار درست

```
cat /etc/hostname # فایل کوچک
head -20 /var/log/syslog # خط اول ۲۰
tail -20 /var/log/syslog # خط آخر ۲۰
tail -f /var/log/syslog # تعقیب زنده (برای خروج Ctrl+C)
less /etc/services # خروجی آخر G بعدی n صفحه به صفحه: /جست‌وجو
wc -l /etc/passwd # تعداد خط
nl file.txt # شماره‌گذاری خطوط
grep -n "error" /var/log/syslog # پیدا کردن خط با شماره
sed -n '10,20p' file.txt # 10 تا 20 فقط خط
```

سه ستون قدرت (**less**) که هر ادمینی باید بداند: **/pattern** جست‌وجو → **n** نتیجه‌ی بعدی → **G** انتهای فایل → **g** ابتدا → **Ctrl+G** اطلاعات فایل → **q** خروج.

۵.۲ جست‌وجوی فایل: **find** و **locate**

```
find /etc -name "*.conf" # بر اساس نام
find /var -type f -size +100M # فایل‌های بزرگ‌تر از ۱۰۰ مگ
find /home -user ali # مالک خاص
find /tmp -mtime +7 -type f # قدیمی‌تر از ۷ روز
find /var/log -type f -exec grep -l "ERROR" {} \; # اجرای فرمان روی نتایج
find / -xdev -perm -4000 -type f 2>/dev/null # SUID فایل‌های (بسیار مهم امنیتی)
find . -maxdepth 2 -type d -name "conf*" # محدود به عمق
locate sshd_config # از دیتابیس (سرچ)
sudo updatedb # به‌روزرسانی دیتابیس
```

ابزار	سرعت	تازگی	چه وقت
find	کند	لحظه‌ای و دقیق	مدیریت دیسک، امنیت، اسکرپت
locate	بسیار سریع	وابسته به updatedb	جست‌وجوی سریع مسیر

۵.۳ معماری جریان و لوله (Pipe) – قلب لینوکس

```
ps aux | grep nginx | grep -v grep
journalctl -u ssh | tail -100 | grep -i "failed"
find /var/log -name "*.log" | xargs -I{} du -h {} | sort -h | tail -10
cut -d: -f1,3 /etc/passwd | sort -t: -k2 -n | head
awk -F: '($3>=1000 {print $1, $3})' /etc/passwd
sed 's/old/new/g' file.txt > new.txt
tr 'a-z' 'A-Z' < file.txt | head -5
sort -u names.txt | wc -l
uniq -c sorted.txt | sort -nr | head
diff -u a.conf b.conf
```

> و >> و 2> و &> :

```
cmd > out.txt # stdout -> فایل (بازنویسی)
cmd >> out.txt # stdout -> انتهای فایل
cmd 2> err.txt # stderr -> فایل
cmd &> all.txt # stdout + stderr
cmd > out.txt 2>&1 # همان (ترتیب مهم است!)
cmd 2>&1 > out.txt # اشتباه رایج: stderr می‌رود: اشتباه رایج
cmd | tee log.txt # هم نمایش بده هم ذخیره کن
```

توصیفگر	شماره	معنای عملی
stdin	0	ورودی پیش‌فرض (کیبورد/فایل)
stdout	1	خروجی معمولی
stderr	2	پیام خطا

۵.۴ فشرده‌سازی و آرشیو

```
tar -czvf backup.tar.gz /etc/nginx # ساخت آرشیو gzip
tar -tzvf backup.tar.gz # فهرست بدون باز کردن
tar -xzvf backup.tar.gz -C /restore # مسیر مشخص
tar -cjf b.tar.bz2 dir/ # bzip2 (کوچک‌تر، کندتر)
tar --exclude='*.log' -czf web.tgz /var/www
gzip -9 big.log ; gunzip big.log.gz
xz -9 big.tar # بهترین نسبت فشرده‌سازی
zip -r site.zip /var/www ; unzip -l site.zip
```

+ مقایسه: tar آرشیو می‌سازد (چند فایل → یک فایل)، gzip/xz فشرده می‌کند. tar.gz = هر دو. برای بکاپ tar -czf always تست باز کردن – بکاپی که تست نشده، بکاپ نیست.

۵.۵ لینک‌ها: سخت (Hard) در برابر نرم (Symbolic)

```
ln target.txt hard_link # hard link: همان inode، شماره‌دهی link +1
ln -s /etc/nginx/nginx.conf /root/nginx.conf.link # symlink
ls -li target.txt hard_link # شماره‌ی inode توجه:
readlink -f /root/nginx.conf.link
```

ویژگی	Hard Link	Symbolic (Soft) Link
اشاره به	inode (داده‌ی واقعی)	مسیر یک نام دیگر
عبور از فایبل سیستم	ناممکن ✗	✓ ممکن
پوشه	✗ (استثنا: <code>.</code> و <code>..</code>)	✓
اگر مقصد حذف شود	داده می‌ماند (فقط یک نام کم می‌شود)	لینک دنگلینگ (شکسته) می‌شود
اندازه‌ی نمایشی	اندازه‌ی داده	اندازه‌ی رشته‌ی مسیر
مثال واقعی	بکاپ سخت‌گیرانه‌ی فایل	<code>/etc/rc2.d</code> , <code>/usr/bin/python3</code>

سؤال طلایی مصاحبه: «تفاوت hard link و soft link در چه چیزی است؟» → جواب: inode در برابر مسیر؛ برگرفته از شماره‌دهی link در `ls -l` (ستون دوم) و `ln -s` برای symlink.

۵.۶ جست‌وجو در محتوا با `grep` و خویشاوندان

```
grep -i "failed" auth.log # بزرگی/کوچکی حروف به
grep -r "Listen 80" /etc/apache2 # بازگشتی
grep -v "^#" nginx.conf # حذف کامنت‌ها (بدون -v می‌دهد)
grep -E "error|fail|warn" app.log # عبارت با قاعده‌ی توسعه یافته
grep -c "404" access.log # شمارش
grep -A2 -B2 "panic" dmesg.txt # خط قبل و بعد ۲
zgrep "attack" /var/log/syslog.*.gz
```

۵.۷ `sed` و `awk` – دو چاقوی همه‌کاره

```
sed -i 's/8080/80/g' site.conf # جایگزینی در جای خود (careful! : -i.bak)
sed -n '/start/,/end/p' file # چاپ بازه‌ی بین دو الگو
sed '/^$/d' file # حذف خطوط خالی
awk '{print $1, $NF}' /var/log/x.log # ستون اول و آخر
awk -F: '$3 >= 1000 {print $1}' /etc/passwd # کاربران واقعی
awk '{sum+=$5} END {print sum}' access.log # جمع ستون ۵
awk '/GET/ {c++} END {print c}' access.log # شمارش درخواست‌های GET
```

۶) عملیات فایل (20:30–21:00)

۶.۱ ساخت، کپی، جابه‌جایی، حذف – به‌همراه سوئیچ‌های ایمن

```
mkdir -p /srv/app/{logs,conf,data} # ساخت درختی در یک فرمان (brace expansion)
cp -av /etc/nginx /backup/nginx-$(date +%F) # -a حفظ مجوزها -v نمایش، با تاریخ
cp --preserve=all file /dest/ # صریح‌ترین حالت حفظ مالکیت/زمان
mv file1 file2 # جابه‌جایی/تغییر نام (همان دستور)
rm -i file.txt # با تأیید
rm -rf /tmp/cache/* # مراقب فاصله‌ها! (اسلش آخر)
rmdir emptydir ; rm -rf dir
install -D -m 750 app.sh /usr/local/bin/app.sh # کپی + ساخت مسیر + مجوز
rsync -a --delete /var/www/ /backup/www/ # همگام‌سازی (بهترین ابزار بکاپ)
rsync -avz --progress /data/ user@10.20.0.5:/data/
```

⚠ ترسناک‌ترین دستور دنیا: `rm -rf / --no-preserve-root`. همیشه جلوی `rm -rf` مسیر را با `ls / pwd` چک کن و اگر مطمئن نیستی `-i` بگذار.

۶.۲ ساخت فایل و محتوا

```
touch -t 202601011200 newfile # ساخت/تغییر زمان
echo "hello" > f.txt # نوشتن
printf "%s\n" a b c # چاپ قالب‌دار (از echo در اسکریپت بهتر از)
cat > notes.txt <<'EOF' # heredoc: نوشتن چندخطی بدون مفسر
خط اول
خط دوم
EOF
truncate -s 0 app.log # خالی‌کردن لاگ بدون حذف فایل (خیلی مفید)
```

۶.۳ فایل‌های ویژه و دستگاه‌ها

```
ls -l /dev/null /dev/zero /dev/random /dev/urandom
dd if=/dev/zero of=/tmp/test.img bs=1M count=100 # ساخت فایل ۱۰۰ مگی
head -c 32 /dev/urandom | od -An -tx1 # رندوم امن
stat file.txt # inode، (سه‌گانه!) مجوز، زمان‌ها
```

سه زمان فایل (سؤال کلاسیک): `atime` آخرین دسترسی، `mtime` آخرین تغییر محتوا، `ctime` آخرین تغییر متادیتا. `ls -l` تنها `mtime` را نشان می‌دهد؛ برای همه `stat` بزن.

۶.۴ مقایسه و بررسی فایل

```
diff -u old.conf new.conf # تفاوت‌ها (patch خروجی قابل)
vimdiff a.txt b.txt
cmp -l file1 file2 | head # تفاوت بایتی
md5sum file ; sha256sum file # هش (تأیید دانلود، یکپارچگی)
file mystery.bin # نوع واقعی فایل (نه پسوند!)
```

Self-Test روز ۱ (۱۵ سؤال – اول جواب بده، بعد پاسخنامه)

1. تفاوت «کرنل لینوکس» و «توزیع لینوکس» در یک جمله؟
2. کرنل مونولیتیک یعنی چه و ماژول چه نقشی دارد؟
3. تفاوت `/proc` و `/sys` چیست؟
4. چهار زیرشاخه `/var` را نام ببر و بگو چه چیزی داخلشان است.
5. `/usr/local` با `/opt` چه تفاوتی دارد؟
6. تفاوت `which` و `type` چه زمانی مهم می‌شود؟
7. دستور `find /var/log -type f -size +50M -exec ls -lh {} \;` چه می‌کند؟
8. تفاوت `hard link` و `symlink` + دستور ساخت هرکدام؟
9. در `cmd > out.txt 2>&1` ترتیب اجرای ریدایرکت‌ها چه اهمیتی دارد؟
10. چرا `less` بهتر از `cat` برای فایل بزرگ است؟ حداقل سه کلید میان‌بر بگو.
11. تفاوت `tar -czf b.tgz dir` و `gzip -r dir` چیست؟
12. `rsync -a --delete src/ dst/` چه خطری دارد و چرا در بکاپ مفید است؟
13. تفاوت `ctime` , `mtime` , `atime` و دستور نمایش همه؟
14. چرا `/tmp` مجوز `1777` دارد؟ آن ۱ چیست؟
15. فرق `apt purge nginx` و `apt remove nginx` و `apt autoremove` ؟

۸) بانک سؤال مصاحبه‌ی روز ۱ (۱۲ سؤال با پاسخ مدل)

- ۱) لینوکس چیست؟ کرنل لینوکس یک هسته‌ی مونولیتیک با ماژول‌های قابل بارگذاری است که مدیریت پردازنده، حافظه، فایل‌سیستم، شبکه و درایورها را برعهده دارد. آنچه «سیستم‌عامل لینوکس» می‌نامیم مجموعه‌ای است از کرنل + ابزارهای GNU + مدیر بسته + سرویس‌ها + پیکربندی‌ها که یک توزیع را می‌سازند.
- ۲) چرا لینوکس در سرور استاندارد است؟ پایداری و آپ‌تایم بالا، منابع کم برای footprint کوچک، اسکریپت‌پذیری و خودکارسازی، امنیت و مدل مجوزها، هزینه‌ی لایسنس، اکوسیستم ابزار، و پشتیبانی از کانتینر/مجازی‌سازی.
- ۳) تفاوت `Ubuntu` و `CentOS/Rocky` را برای پروژه‌ی سازمانی چگونه توضیح می‌دهی؟ `Ubuntu` روی `apt` / `.deb` است، انتشار هر ۶ ماه و LTS ۵ ساله؛ `Rocky/Alma` باینری‌سازگار با `RHEL` روی `dnf` / `.rpm` با چرخه‌ی ۱۰ ساله، `SELinux` فعال و پشتیبانی تجاری. انتخاب بر اساس سیاست سازمان، نیاز پشتیبانی و اکوسیستم نرم‌افزاری است.
- ۴) کاربر فضای کاربر و کرنل چه تفاوتی دارند و چطور با هم حرف می‌زنند؟ کرنل در حالت ممتاز اجرا می‌شود و دسترسی مستقیم به سخت‌افزار دارد؛ فرایندهای کاربر در حالت محدود اجرا می‌شوند و از طریق `system call` (و `/proc` , `/sys`) درخواست می‌دهند. `strace` این گفت‌گو را قابل مشاهده می‌کند.
- ۵) `FHS` چیست و چرا اهمیت دارد؟ استاندارد سلسله‌مراتب فایل‌سیستم که تعیین می‌کند فایل‌ها کجا باشند. اهمیتش در قابلیت انتقال اسکریپت‌ها، انتظار ابزارها از مسیرها، و عیب‌یابی/بکاپ هدفمند است.
- ۶) اگر بگویم / پر شده، اولین ۵ فرمانی که می‌زنی؟

```
df -hT ; df -i # و inode فضا
du -xh --max-depth=1 / | sort -h # پرمصرفترین شاخه‌ها
find /var -type f -size +500M -exec ls -lh {} \;
lsof +L1 | head # فایلهای حذفشده‌ای که در حالتی باز نگه داشته شده
journalctl --disk-usage # حجم ژورنال
```

سپس: پاک‌سازی لاگ‌های قدیمی، `journalctl --vacuum-size=200M`، چرخش لاگ با `logrotate`، بررسی `/tmp` و کش‌ها.

(۷) تفاوت `cat`، `less`، `tail -f` و کدام در چه حالتی؟ `cat` برای فایل کوچک، `less` برای مرور تعاملی و جست‌وجو، `tail -f` برای تعقیب زنده‌ی لاگ. در سرور همیشه اول `less` (تا حافظه ترمینال پر نشود) و برای مشاهده‌ی زمان وقوع رخداد `tail -f` یا `journalctl -f`.

(۸) چگونه یک فایل را در همه‌ی فایل‌های پیکربندی `/etc` پیدا می‌کنی که «Listen» دارد؟

```
grep -RIn --exclude-dir=.git "Listen" /etc
find /etc -type f -name "*.conf" -exec grep -Hn "Listen" {} +
```

(۹) تفاوت `>` و `>>` و خطر اصلی‌شان؟ `>` بازنویسی می‌کند، `>>` اضافه می‌کند. خطر `>` : با `>` روی خود فایل ورودی، فایل قبل از خوانده‌شدن خالی می‌شود (`sort file > file` فایل را نابود می‌کند).

(۱۰) چطور یک درخت پوشه با مجوزهای مشخص در یک فرمان می‌سازی؟

```
mkdir -p /srv/app/{conf,logs,data} && chmod -R 750 /srv/app && chown -R app:app /srv/app
```

(۱۱) `find` و `locate` را کجا انتخاب می‌کنی؟ `locate` سریع است ولی از دیتابیس و شاید قدیمی؛ `find` لحظه‌ای و دقیق با شرط‌های پیچیده (اندازه، زمان، مالک، مجوز). در اسکریپت‌های حساس و بکاپ همیشه `find`.

(۱۲) چطور مطمئن می‌شوی بکاپ سالم است؟ هش می‌گیریم (`sha256sum`)، آرشیو را در مسیر موقت باز می‌کنم و تعداد/اندازه‌ی فایل‌ها را با مبدأ مقایسه می‌کنم (`tar -tzf | wc -l`)، و در صورت امکان بازیابی روی سیستم آزمایشی (restore drill) انجام می‌دهم. بکاپ تست‌نشده بکاپ نیست.

۹) چک‌لیست تکمیل روز ۱

- [] یک VM لینوکس نصب کرده‌ام و از میزبان با SSH وصل می‌شوم
- [] `cat /etc/os-release`، `hostnamectl`، `uname -r` را اجرا کرده و معنای خروجی را می‌دانم
- [] `/proc/meminfo`، `/proc/cpuinfo`، `dmesg`، `modinfo`، `lsmod` را دیده‌ام
- [] درخت FHS را از حفظ می‌گویم و برای هر شاخه یک مثال فایل دارم
- [] `find` با شرط اندازه/زمان/مالک اجرا کرده‌ام و `locate` را به‌روز کرده‌ام
- [] `pipe` و ریدایرکت (`|`، `>`، `>>`، `2>`، `&>`، `&`) را تمرین کرده‌ام
- [] `tar` ساخته، فهرست کرده و باز کرده‌ام و هش گرفته‌ام
- [] `hard link` و `symlink` ساخته و تفاوت `inode` را با `ls -li` دیده‌ام
- [] چهار تمرین Break & Fix آزمایشگاه را انجام داده‌ام
- [] به هر ۱۲ سؤال مصاحبه بلند جواب داده‌ام

سؤال‌های این روز، پاسخ‌های کامل‌شان و آزمون‌های جمع‌بندی در انتهای همین بسته (فصل پاسخنانه و آزمون نهایی هفته ۲) آمده است.

بسته‌ی مطالعه‌ی LPIC-1 – روز ۲

فرمان‌های GNU/Linux، مجوزها و فایل سیستم‌ها

Text Processing • Permissions • ACL • Links • Filesystems • LVM

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۲ • روز ۲ (سه‌شنبه) – مورنینگ ۹:۰۰–۱۳:۰۰ • ایوبینینگ ۱۷:۰۰–۲۱:۰۰

روز ۲ – فرمان‌های GNU/Linux، مجوزها و فایل سیستم‌ها

مسیر مطالعه‌ی این روز (طبق پلن ۳۰ روزه): مورنینگ: (10:45–12:15) → File Permissions → Text Processing (09:00–10:30) → (12:30–13:00) Links: Hard & Soft ایوبینینگ: Lab: مجوزها و کاربران (18:30–17:00) → (18:45–20:15) → مبانی فایل سیستم، پارتیشن و LVM (20:30–21:00) هدف روز: پردازش متن حرفه‌ای (grep/sed/awk/cut/sort/uniq/tr)، درک عمیق مدل مجوزها (rwx, octal, special bits), و شناخت فایل سیستم‌ها، پارتیشن بندی و LVM. (umask, ACL).

0) مرور ۳ دقیقه‌ای روز ۱

موضوع دیروز	امروز کجا به کار می‌آید
FHS (/home , /var , /etc)	محل کاربران (/etc/passwd), لاگ‌ها (/var/log) و مجوزها
pipe/ grep / find	ابزارهای پردازش متن امروز؛ هر دو با هم ترکیب می‌شوند
hard/symlink	عمیق‌تر: inode , شمارنده‌ی link, stat
عملیات فایل	cp -a و rsync مجوزها را حفظ می‌کنند – امروز می‌فهمی چرا مهم است
ls -l	ستون اول = مجوزها؛ امروز رمزگشایی‌اش می‌کنیم

جمله‌ی وصل‌کننده: دیروز فایل‌ها را پیدا و جابه‌جا کردی؛ امروز محتوا را پردازش می‌کنی و کنترل می‌کنی چه کسی چه کاری مجاز است – دو مهارتی که ۸۰٪ کار روزمره‌ی ادمین را می‌سازند.

۱) پردازش متن (09:00–10:30)

۱.۱ خانوادگی ابزارها

ابزار	کار اصلی	مثال کلیدی
grep	جست‌وجو با الگو	<code>grep -RIn "error" /var/log</code>
cut	بریدن ستون/کاراکتر	<code>cut -d: -f1 /etc/passwd</code>
sort	مرتب‌سازی	<code>sort -t: -k3 -n /etc/passwd</code>
uniq	حذف/شمارش تکراری‌های مجاور	<code>uniq -c</code>
tr	تبدیل/حذف کاراکتر	<code>tr '[:lower:]' '[:upper:]'</code>
sed	ویرایش جریانی متن	<code>sed -Ei 's/x/y/g' f</code>
awk	پردازش ستونی و محاسبات	<code>awk -F: 'NR<=5{print \$1}'</code>
head/tail	ابتدا/انتهای فایل	<code>tail -n +2</code>
wc	شمارش خط/کلمه/بایت	<code>wc -l</code>
paste/join/split	ترکیب، اتصال، تقسیم	<code>split -l 1000 big.txt part_</code>
tee	انشعاب خروجی	<code>cmd tee out.log</code>
column/tabulate	جدول‌سازی خوانا	<code>mount column -t</code>

۱.۲ grep در عمق

```
grep -RIn --include="*.conf" "Listen" /etc # فقط فایل‌های *.conf بازگشتی + شماره‌ی خط +
grep -v "^#" nginx.conf | grep -v "^$" # حذف کامنت و خط خالی
grep -E "(GET|POST) /admin" access.log # ERE
grep -P "\b\d{1,3}(\.\d{1,3}){3}\b" file # PCRE: الگوی IP
grep -oE "[0-9]+\.[0-9]+" log | sort -u # بخش منطبق را چاپ کن
grep -L "permit" *.conf # فایل‌هایی که الگو را ندارند
grep -c "" file # تعداد خط (wc -l معادل)
```

نکته‌ی عملکردی: برای فایل‌های بزرگ و الگوهای پیچیده، `grep -F` (رشته‌ی ثابت، بدون regex) بسیار سریع‌تر است؛ `rg` (ripgrep) جایگزین مدرن و سریع‌تری برای پوشه‌های بزرگ است.

۱.۳ sed در عمق – سه مورد پرکاربرد

```
sed -i.bak -E 's/^#?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config # کانفیگ را عوض کن
+ بکاپ
sed -n '100,120p' app.log # بازه‌ی خط
sed -n '/ERROR/,+3p' app.log # به همراه ۳ خط بعد ERROR هر
sed -e 's/\t/ /g' -e 's/*//g' messy.txt # چند ویرایش پشت‌سرهم
sed '/^#/d; /^$/d' file # حذف کامنت و خط خالی
sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3\/\2\/\1/' dates.txt # جا به جایی گروه‌ها
```

⚠️ **قاعده‌ی طلایی:** `-i` بدون بکاپ (`-i.bak`) روی فایل کانفیگ = دردسر. و همیشه بعدش سرویس را با `ssh -t` ، `nginx -t` یا `named-checkconf` تست کن.

۱.۴ awk در عمق – زبان کوچک اما کامل

```
awk -F: '{print $1}' /etc/passwd # ستون اول
awk -F: '$3>=1000 && $1!="nobody" {print $1,$3}' /etc/passwd
awk '{s+=$10} END {printf "Total bytes: %d\n", s}' access.log
awk '$9==404 {c++} END {print "404s:", c}' access.log
awk 'NR==1{print} /CRITICAL/{print NR: "$0}" app.log # هدر + خطوط حساس
awk -F'[|]{' '{print $2}' logs | sort | uniq -c | sort -nr | head
awk '{a[$1]++} END {for (k in a) print a[k], k}' ips.txt
```

تفکیک نقش‌ها (برای مصاحبه): `grep` انتخاب خط، `sed` ویرایش خط، `awk` انتخاب و سازمان‌دهی فیلد و محاسبه. ترکیب این سه + `pipe` = پردازش داده بدون هیچ زبان برنامه‌نویسی.

۱.۵ نمونه‌ی واقعی: گزارش از لاگ وب (تمرین جدی این بلوک)

```
# ۱۰ IP پرمصرف
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head -10

# ۱۰ مسیر پر بازدید
awk '{print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head -10

# در هر ساعت xx تعداد خطاهای 5
awk '$9 ~ /^5/ {print $4}' access.log | cut -d: -f1 | sort | uniq -c

# (RFC1918) کاربرانی که فقط از داخل شبکه هستند
grep -E "(10\.|192\.168\.|172\.(1[6-9]|2[0-9]|3[01])\.)" access.log | wc -l
```

پل به هفته ۱: همان تحلیلی که روی یک روتر با ACL counter می‌کردی، اینجا روی لاگ سرور می‌کنی؛ در هفته ۳ (CEH) همین الگو، تحلیل رخداد حمله می‌شود.

۱.۶ فایل‌های متنی مهم سیستم

```
cat /etc/passwd # نام:UID:GID:GECOS:home:shell
sudo cat /etc/shadow # نام:هش:تاریخ‌های سیاست رمز
cat /etc/group # نام:GID:اعضا
sudo cat /etc/gshadow
cat /etc/fstab # چه چیزی در بوت مانده شود
cat /etc/hosts /etc/hostname /etc/resolv.conf
cat /etc/os-release /etc/shells /etc/services
ls -l /etc/passwd /etc/shadow /etc/group # مجوزها را مقایسه کن
```

ساختار `/etc/passwd` را از حفظ بدان:

```
ali : x : 1000 : 1000 : Ali A , Room 12 , 555 , 555 : /home/ali : /bin/bash
^ ^ ^ ^ ^ ^ ^
| | | | +-- GECOS ( توضیح ) | +-- shell ورود
| | | +----- GID خانه اصلی
| | +----- UID
| +----- (x یعنی در /etc/shadow) جای رمز
+----- نام ورود
```

۲) مجوزهای فایل (10:45–12:15)

۲.۱ رمزگشایی ls -l

```
-rwxr-xr-- 1 ali devs 4096 Sep 26 10:00 script.sh
||| ||| |||
||| ||| +++---- مجوز سایرین (others): r--
||| +++----- مجوز گروه (group): r-x
+++----- مجوز مالک (user): rwx
+----- پایپ p | سوکت s | b/dev | c/dev | l symlink | دایرکتوری d | نوع: - فایل معمولی
```

کاراکتر	روی فایل	روی پوشه
r (4)	خواندن محتوا	فهرست کردن نامها (ls)
w (2)	تغییر محتوا	ساخت/حذف/تغییر نام ورودیها
x (1)	اجرای فایل	ورود (cd) و دسترسی به محتوا

مهمترین نکته‌ی این بخش: حذف یک فایل به مجوز پوشه‌ی والد بستگی دارد نه به مجوز خود فایل. اگر روی پوشه w و x داشته باشی، می‌توانی فایل فقط خواندنی را هم پاک کنی.

۲.۲ حالت octal و جمع‌زدن

octal	rwX	octal	rwX
5	r-x	0	---
6	-rw	1	x--
7	rwX	2	-w-
		4	--r

```
chmod 644 file # rw-r--r--
chmod 600 id_rsa # rw----- (!کلید خصوصی)
chmod 755 /usr/local/bin/app
chmod 640 /etc/shadow-like-file
chmod u+x,g-w,o-rwx script.sh # حالت نمادین
chmod -R 750 /srv/app # بازگشتی (با احتیاط)
chmod --reference=ref.txt target.txt
```

مقدار	رایج برای
644	فایل معمولی، سند، کانفیگ خواندنی
600	داده‌ی خصوصی: کلید SSH خصوصی، کانفیگ با رمز
755	برنامه‌ی اجرایی، پوشه‌ی عمومی
700	پوشه‌ی خانه‌ی خصوصی، اسکرپت حساس
664/775	همکاری گروهی (group-writable)
1777	/tmp

۲.۳ مالکیت: `chown` / `chgrp`

```
sudo chown ali file
sudo chown ali:devs file
sudo chown -R www-data:www-data /var/www/html
sudo chgrp devs /srv/project
sudo chown --from=olduser:oldgroup newuser file
```

UID/GID عددی: از `chown 1000:1000 file` می‌توانی عدد هم بدهی (در سناریوهای کانتینر لازم می‌شود).

۲.۴ `umask` – مجوز پیش‌فرض از کجا می‌آید؟

```
umask # 0022 مثلا
umask -S # u=rwx,g=rx,o=rx
umask 027 # فقط برای همین شل
```

- فایل جدید: `umask - 666` . با `022` → `644` . پوشه‌ی جدید: `umask - 777` → `755` .
- `umask 077` برای داده‌ی خصوصی (پوشه‌ها `700` ، فایل‌ها `600`) .
- مکان دائمی: `/etc/profile` ، `/etc/bash.bashrc` یا `~/.bashrc` (روز ۳) .

جدول تمرینی (خودت پرش کن):

umask	پوشه	فایل
022	755	644
027	750	640
077	700	600
002	775	664

۲.۵ بیت‌های ویژه: SUID، SGID، Sticky (بسیار مهم)

```
-rwsr-xr-x root root /usr/bin/passwd <- SUID
drwxrwsr-x root devs /srv/shared <- SGID روی پوشه
drwxrwxrwt root root /tmp <- Sticky
```

بیت	عدد	روی فایل	روی پوشه
SUID	4000	اجرا با UID مالک (مثل <code>passwd</code> که با دسترسی root فایل shadow را می‌نویسد)	-
SGID	2000	اجرا با GID گروه مالک	فایل‌های جدید در پوشه، گروه پوشه را به ارث می‌برند
Sticky	1000	-	فقط مالک/ root می‌تواند فایل‌های دیگران را حذف کند (<code>/tmp</code>)

```

chmod u+s app # افزودن SUID
chmod 4755 app # SUID + 755
chmod g+s /srv/shared # SGID روی پوشه‌ی اشتراکی
chmod 1777 /tmp ; chmod +t /tmp # Sticky
find / -xdev -perm -4000 -type f 2>/dev/null # های سیستم SUID شکار

```

نگاه امنیتی (پل به هفته ۳): لیست SUID را منظم بگیر و با لیست پایه مقایسه کن. اگر SUID باینری غیرمنتظره دیدی (مثل `nmap`، `cp`، `find`، `vim`)، احتمال **privilege escalation** جدی است. `sudo -l` هم برای دیدن مجوزهای `sudo` لازم است. این دو سؤال، سؤال استاندارد مصاحبه‌ی SOC/Pentest است: `bash find / -perm -4000 -type f 2>/dev/null | sort > /root/` `suid-baseline.txt` # `diff baseline.txt <(find / -perm -4000 -type f 2>/dev/null | sort) #` مقایسه‌ی هفتگی

۲.۶ ACL – وقتی ۳ کلاس کافی نیست

```

mount | grep acl # پشتیبانی fs
getfacl /srv/project # دیدن ACL
setfacl -m u:mary:rw /srv/project/report.doc # کاربر خاص + rw
setfacl -m g:audit:r-x /srv/project # گروه خاص
setfacl -m d:u:mary:rw /srv/shared # ACL پیش‌فرض (default)
setfacl -x u:mary /srv/project/report.doc # حذف یک ورودی
setfacl -b /srv/project/report.doc # حذف کل ACL
getfacl --all-effective /srv/project | head

```

خروجی `getfacl` کلیدواژه‌ها: `user::rw`، `group::r-x`، `other::r-x`، `mask::rw` (سقف مؤثر مجوزهای extended)، `default:...`

نکته‌ی مصاحبه: **mask** در ACL سقف مجوزهای ACL entries است؛ اگر تغییری اعمال نشد، اول `mask` را چک کن.

۲.۷ مجوز sudo و سیاست دسترسی

```

sudo visudo # ویرایش امن /etc/sudoers (قبل از خروج تست)
sudo -l # من چه چیزی می‌توانم
sudo -u www-data whoami # اجرا به نام کاربر دیگر
sudo -k # باطل کردن کش رمز

```

```

# نمونه ورودی /etc/sudoers.d/ops
%devs ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
deploy ALL=(root) NOPASSWD: /usr/bin/rsync --server *
Defaults env_reset, timestamp_timeout=5, logfile="/var/log/sudo.log"

```

مراحل RU (از پلن ۳۰ روزه): **su** (سوئیچ با رمز کاربر مقصد) در برابر **sudo** (اجرا با سیاست دقیق و لاگ). در سرور مدرن: root را از SSH غیرفعال کن، **sudo** با لاگ و گرانولار بده، و **sudo** را جایگزین «همه‌ی کارها با root» کن.

۳) لینک‌ها: درک عمیق (12:30–13:00) inode

۳.۱ inode چیست؟

هر فایل روی ext4 یک **inode** دارد: متادیتا (مالک، مجوز، زمان‌ها، اندازه، شمارش لینک، اشاره‌گر به بلوک‌های داده) – ولی نام فایل در inode نیست؛ نام در «دایرکتوری» نگاشت می‌شود به inode.

```
stat file.txt | head -8
ls -li file.txt hard_link # inode یکسان -> hard link
ls -li file.txt symlink # inode متفاوت، نوع l -> symlink
ln file.txt h2.txt ; ls -l file.txt h2.txt | awk '{print $2, $9}' # شمارش لینک
rm file.txt ; cat h2.txt # داده باقی است تا آخرین نام حذف شود
```

عملیات	Hard	Soft
<code>ls -l</code>	شمارنده‌ی link را زیاد می‌کند	فایل جدید نوع <code>l</code>
حذف مبدأ	داده سالم، فقط نام کم می‌شود	لینک می‌شکند (<code>readlink</code> مسیر را نشان می‌دهد)
روی پوشه	ممنوع (حلقه‌ی بی‌پایان)	مجاز
بین فایل سیستم‌ها	ممنوع (inode محلی است)	مجاز
کاربرد واقعی	بکاپ سخت‌گیرانه، فایل‌های یکسان بدون کپی	<code>/etc/rc2.d/S01nginx</code> , <code>python3 -> python3.12</code>

۳.۲ لینک در عمل: سوئیچ نسخه‌ها

```
ls -l /usr/bin/python3 # python3 -> python3.12
sudo ln -sf /opt/app-2.0 /opt/app-current # تغییر مسیر سرویس
readlink -f /opt/app-current
find / -xtype l 2>/dev/null # های شکسته در سیستم symlink پیدا کردن
```

۴) آزمایشگاه ۱ – مجوزها، کاربران و گروه‌ها (18:30–17:00)

۴.۱ توپولوژی سناریو

- شرکت «پارس‌داده» سه تیم دارد: **audit** , **ops** , **devs**. یک پوشه‌ی پروژه‌ی مشترک `/srv/project` با این قواعد:
- **devs** خواندن/نوشتن؛ **ops** خواندن/نوشتن؛ **audit** فقط خواندن.
 - فایل‌های جدید گروه **devs** بگیرند (SGID).
 - کاربر **ali** (دولوپر) اجازه‌ی `sudo` فقط برای `restart` کردن `nginx`.

```
# گروه‌ها و کاربران ۱)
sudo groupadd devs ; sudo groupadd ops ; sudo groupadd audit
sudo useradd -m -G devs -s /bin/bash ali
sudo useradd -m -G ops -s /bin/bash sara
sudo useradd -m -G audit -s /bin/bash reza
sudo passwd ali ; sudo passwd sara ; sudo passwd reza

# دسترسی‌ها + SGID + پوشه‌ی پروژه ۲)
sudo mkdir -p /srv/project
sudo chown root:devs /srv/project
sudo chmod 2775 /srv/project # rwxrwsr-x (SGID)
setfacl -m g:audit:rx /srv/project # audit (روی گروه) فقط خواندن

# تست: با هر کاربر وارد شو و بین چه می‌توانی ۳)
su - ali -c 'touch /srv/project/a.txt && ls -l /srv/project'
su - sara -c 'touch /srv/project/s.txt'
su - reza -c 'touch /srv/project/r.txt' # باشد Permission denied باید
ls -l /srv/project ; getfacl /srv/project

# ۴) file: SUID فایل خصوصی و
echo "secret" | sudo tee /srv/project/report.txt >/dev/null
sudo chmod 640 /srv/project/report.txt
sudo chown root:devs /srv/project/report.txt

# ۵) sudo محدود برای ali
echo 'ali ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx' | sudo tee /etc/sudoers.d/ali
sudo chmod 440 /etc/sudoers.d/ali
sudo -l -U ali
su - ali -c 'sudo systemctl restart nginx' # اگر nginx نه خطای سرویس، sudo نصب نیست، باید رد شود
su - ali -c 'sudo cat /etc/shadow' #
```

۴.۳ آزمون درک (جواب بده، بعد چک کن)

1. با `umask 027` یک فایل و یک پوشه بساز. مجوزها چیست؟ (`640` , `750`)
2. `chmod 1777 /tmp` روی `/tmp` یعنی چه؟ اگر `sticky` نبود چه اتفاقی می‌افتاد؟ (هر کسی هر فایلی را می‌توانست حذف کند)
3. یک فایل `600` داخل پوشه‌ی `755` – آیا کاربر دیگر می‌تواند آن را حذف کند؟ چرا؟ (خیر، چون `w` روی پوشه ندارد)
4. تفاوت `chmod u+s f` و `chmod g+s f` ؟ (SUID در برابر SGID – اولی روی باینری، دومی روی فایل/پوشه)
5. `find / -xdev -perm -4000 -type f 2>/dev/null` را اجرا کن – چند فایل داری و آیا همه انتظارشان را داری؟

#	خرابی ساخته شده	علامت	تشخیص/رفع
1	<code>chmod 000 ~/.ssh/id_rsa</code>	SSH خطای کلید می‌دهد	<code>chmod 600 ~/.ssh/id_rsa</code>
2	<code>chmod -R 777 ~/.ssh</code>	ورود کلید رد می‌شود (too open)	<code>chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys</code>
3	حذف کاربر بدون حذف خانه	house قفل، <code>userdel</code> شکایت	<code>find / -nouser</code> یا <code>sudo userdel -r ali</code>
4	<code>umask 077</code> روی سرور وب	وب سرور فایل‌ها را نمی‌خواند	<code>umask 022</code> یا <code>chmod</code> هدفمند
5	<code>chown -R user:user /</code> (فاجعه!)	سیستم دیگر بالا نمی‌آید	بازیابی از اسنپ‌شات/بکاپ – پس هرگز این را آزمایش نکن

۵ (find / locate / which) – مدیریت فایل در مقیاس (18:45–20:15)

۵.۱ find در عمق

```
find / -xdev -type f -size +1G # بزرگ‌تر از ۱ گیگ (بدون رفتن روی NFS)
find /var -type f -mtime -1 # تغییر یافته در ۲۴ ساعت گذشته
find /var/log -type f \( -name "*.log" -o -name "*.gz" \) -size +10M
find /home -type d -perm -002 # world-writable پوشه‌های (خطر امنیتی)
find . -type f -exec chmod 640 {} + # اجرای کارا (xargs فقط یک بار)
find /data -name "*.tmp" -delete
find /etc -newer /etc/passwd # (تراپل‌شوت تغییرات)
find / -type f -user unknown 2>/dev/null # یتم UID محلی برای
find /tmp -type f -mtime +3 -exec rm -f {} \; # پاک‌سازی (با -print کن)
```

الگوی امن: همیشه اول `-print` بگذار و خروجی را ببین، بعد `-delete` / `rm -exec` بزن.

`\; -exec {} +` در برابر `-exec {}`: اولی برای هر فایل یک فرایند اجرا می‌کند (کند)، دومی همه را در یک فرمان جمع می‌کند (سریع). در اسکریپت‌ها همیشه `+` یا `xargs`.

۵.۲ لاگ‌چرخانی سریع (کاربرد واقعی find)

```
find /var/log -name "*.log" -mtime +30 -exec gzip {} \;
find /var/log -name "*.gz" -mtime +365 -delete
du -sh /var/log/* | sort -h | tail -5
```

(`/etc/logrotate.d/` در `logrotate` است – کانفیگ‌ها در `/etc/logrotate.d/`)

```
which nginx # اولین مورد در PATH
whereis nginx # مان پیچ، سورس، مان پیچ
type -a ls # همهی معناها: alias/builtin/functions/binary
command -v ls # امن برای اسکریپت
hash -r # پاک کردن کش مسیر فرمان های شل
```

۶) فایل سیستم، پارتیشن و LVM – مبانی (20:30–21:00)

۶.۱ انواع فایل سیستم

فایل سیستم	ویژگی	کاربرد
ext4	پیش فرض اکثر توزیع ها، ژورنال دار، پایدار	سرور عمومی
XFS	ژورنال دار، عملکرد عالی روی فایل های بزرگ، کوچک شدنی نیست	RHEL/Rocky، دیتابیس /LV بزرگ
Btrfs	snapshot، CoW، فشرده سازی، زیرحجم	openSUSE، NAS، rollback
ZFS	استخر، snapshot، RAID-Z، checksum	سرور ذخیره سازی/مجازی سازی
vfat/exFAT	سازگار با همه	USB، پارتیشن EFI
swap	ناحیه ی جانشین حافظه	swap پارتیشن/فایل
tmpfs	در RAM و سریع	/run ، /tmp (محتوای آن پایدار نیست)
NFS/CIFS	شبکه ای	اشتراک بین سرورها/ویندوز

تفاوت ژورنال (ext4) و CoW (Btrfs): ژورنال اتمی بودن تغییرات متادیتا را تضمین می کند (بازیابی سریع بعد از قطع برق)؛ CoW داده را نمی نویسد تا تراکنش کامل شود و امکان snapshot ارزان می دهد.

۶.۲ ابزارهای مشاهده و ساخت

```
lsblk -f # فایل سیستم + درخت دیسک + UUID
blkid # LABEL و UUID تمام دستگاه ها
df -hT ; df -i # تعداد و inode فضا
du -sh /var/* | sort -h | tail
mount | column -t
findmnt -T /var/www # کدام دستگاه این مسیر را نگه داشته
mount /dev/sdb1 /mnt/data
umount /mnt/data # busy اگر: lsof +D /mnt/data
```

ساخت فایل سیستم و پارتیشن (روی یک دیسک آزمایشی حتماً!):

```

sudo fdisk -l /dev/sdb
sudo fdisk /dev/sdb # n ساخت، t نوع، w ذخیره (MBR: پارتیشن اصلی؛ GPT: ۱۲۸ حداکثر ۴ پارتیشن اصلی)
sudo partprobe /dev/sdb
sudo mkfs.ext4 -L data /dev/sdb1
sudo mkfs.xfs /dev/sdb2
sudo tune2fs -l /dev/sdb1 | head -20 # ویژگی‌های ext4
sudo fsck -y /dev/sdb1 # chk فایل سیستم (نمایش mount روی)
sudo e2label /dev/sdb1 data ; sudo xfs_admin -L data /dev/sdb2

```

۹.۳ /etc/fstab – مانت پایدار در بوت

```

# <device> <mount point> <fstype> <options> <dump> <pass>
UUID=8f4e...c2 / ext4 defaults 0 1
UUID=1a2b...9d /data xfs defaults,noatime 0 2
UUID=77aa...11 none swap sw 0 0
//srv/share /mnt/share cifs credentials=/root/.smb,uid=1000 0 0

```

گزینه	معنا
defaults	rw,suid,dev,exec,auto,nouser,async
noatime	atime به روز نشود (کارایی)
noexec/nosuid/noddev	سخت‌گیری امنیتی برای /home , /tmp
ro	فقط خواندنی
nofail	اگر دستگاه نبود، بوت شکست نخورد (برای USB)
_netdev	وابسته به شبکه (مانت پس از شبکه)

```

sudo findmnt --verify # اعتبارسنجی fstab (قبل از ری‌بوت)
sudo mount -a # تست بدون ری‌بوت
sudo systemctl daemon-reload

```

⚠ رایج‌ترین فاجعه‌ی مبتدی: UUID اشتباه در fstab → سیستم در حالت emergency بوت می‌شود. همیشه `findmnt --verify` و `mount -a` بزن.

۹.۴ LVM – سه لایه‌ی اصلی

```

PV (Physical Volume: /dev/sdb1, /dev/sdc1, ...)
|
v
VG (Volume Group: pool ذخیره‌سازی، مثلا vg_data)
|
v
LV (Logical Volume: vg_data/lv_www => فایل سیستم ext4/xfs => /var/www)

```

```

sudo pvcreate /dev/sdb1 /dev/sdc1
sudo pvdisplay ; sudo pvs
sudo vgcreate vg_data /dev/sdb1 /dev/sdc1
sudo vgs ; sudo vgdisplay
sudo lvcreate -L 20G -n lv_www vg_data
sudo mkfs.ext4 /dev/vg_data/lv_www
sudo mount /dev/vg_data/lv_www /var/www

# کشسان بودن دلیل اصلی استفاده از) بزرگ کردن (LVM
sudo lvextend -L +10G /dev/vg_data/lv_www
sudo resize2fs /dev/vg_data/lv_www # ext4 (برای xfs: xfs_growfs /var/www)
# بزرگ کردن با پر کردن کل فضای آزاد
sudo lvextend -l +100%FREE -r /dev/vg_data/lv_www # -r = resize همزمان
# خطرناک؛ با بکاپ) -L -5G lvreduce کوچه کن بعد resize2fs اول ext4: کوچه کردن

```

مزیت LVM	چرا مهم است
توسعه‌ی آنلاین	بدون ریboot و بدون downtime ظرفیت اضافه می‌کنی
اسنپ‌شات	بکاپ همدوس قبل از ارتقا/پیچ
pool مشترک	چند دیسک یک فضای منطقی
نام‌گذاری معنایی	<code>vg_data/lv_www</code> به جای <code>/dev/sdb3</code>

۶.۵ swap

```

free -h ; swapon --show
sudo mkswap /dev/sdb2 ; sudo swapon /dev/sdb2
sudo fallocate -l 2G /swapfile ; sudo chmod 600 /swapfile
sudo mkswap /swapfile ; sudo swapon /swapfile
cat /proc/sys/vm/swappiness # 10 پیش‌فرض 60؛ برای سرور دیتا بیس معمولاً

```

قاعده‌ی سرانگشتی: ۱ برابر RAM تا ۴ گیگ، ۲ گیگ برای بالاتر، یا swap کوچه اگر hibernation نمی‌خواهی.

۶.۶ سهمیه (Quota) – مقدماتی

```

sudo mount -o remount,usrquota,grpquota /
sudo quotacheck -cugm / ; sudo quotaon -vug /
sudo edquota -u ali
sudo repquota -a

```

Self-Test روز ۲ (۱۵ سؤال)

1. تفاوت `grep` , `sed` , `awk` را در یک عبارت کوتاه بگو.
2. `awk -F: '$3>=1000 {print $1}' /etc/passwd` چه می‌کند؟
3. تفاوت `\;` `-exec {}` و `+` `-exec {}` در `find` ؟
4. چرا حذف فایل به مجوز پوشه وابسته است، نه خود فایل؟
5. با `umask 027` مجوز فایل و پوشه‌ی جدید چیست؟

6. SUID چیست و چرا `passwd` به آن نیاز دارد؟
7. تفاوت `chmod g+s` روی فایل و روی پوشه؟
8. `sticky bit` چه مشکلی را در `/tmp` حل می‌کند؟
9. تفاوت `setfacl -m d:u:x:rw` و `setfacl -m u:x:rw`؟
10. `mask` در ACL چه نقشی دارد؟
11. تفاوت `userdel ali` و `userdel -r ali`؟
12. XFS را نسبت به ext4 چه وقت انتخاب می‌کنی؟ یک محدودیت مهم XFS چیست؟
13. در `fstab`، `nofail` و `_netdev` چه وقت لازم می‌شوند؟
14. سه دلیل استفاده از LVM به جای پارتیشن خام؟
15. `lvextend -l +100%FREE -r /dev/vg0/lv0` چه می‌کند؟

۸) بانک سؤال مصاحبه‌ی روز ۲ (۱۲ سؤال با پاسخ مدل)

- ۱) مجوز **764** یعنی چه؟ مالک `rw` (۷)، گروه `rw-` (۶)، سایرین `---` (۴). برای اسکریپت اجرایی و فایل مشترک گروهی رایج است، هرچند **755** یا **750** امن‌ترند — `other` خواندن دارد که برای بکاپ/لاگ بی‌خطر ولی برای داده‌ی حساس اشتباه است.
- ۲) چطور یک فایل را فقط برای یک کاربر خاص قابل نوشتن می‌کنی بدون تغییر مالکیت؟ با `ACL`: `setfacl -m u:mary:rw file` (و در صورت نیاز روی پوشه `chown :grp` و `chmod 660`).
- ۳) SUID را چطور پیدا و مدیریت می‌کنی؟

```
find / -xdev -perm -4000 -type f 2>/dev/null | sort
```

سپس با خط پایه مقایسه، مورد غیرمنتظره را با `chmod u-s` خنثی می‌کنم و بسته‌ی مربوطه را بررسی می‌کنم. SUIDهای رایج: `su`، `passwd`، `mount`، `sudo`، `pkexec`. هر SUID اضافی = ریسک ارتقای دسترسی.

۴) `sudo` و `su` را مقایسه کن و در شرکت کدام را توصیه می‌کنی؟ `su` به یک شل کامل با هویت دیگر می‌دهد و رمز حساب مقصد را می‌خواهد؛ `sudo` فرمان‌های مشخص را با لاگ و سیاست دقیق اجرا می‌کند. توصیه: ورود `root` مستقیم غیرفعال (`PermitRootLogin no`)، کاربران با کلید SSH، و `sudo` گرانولار در `/etc/sudoers.d` با لاگ مرکزی.

۵) کاربران می‌گویند «نمی‌توانم وارد پوشه‌ی مشترک شوم». مراحل عیب‌یابی؟ `namei -l /srv/shared/sub` (زنجیره‌ی مجوز همه‌ی والد‌ها)، `getfacl -p -R`، بررسی عضویت گروه با `id user` (و نیاز به `login` مجدد پس از `usermod -aG`)، بررسی `/SELinux` (`ausearch -m avc -ts recent`، `dmesg | grep -i denied`) و مجوز پوشه‌ی والد. در ۹۰٪ موارد یک `x` کم روی یک پوشه‌ی میانی است.

۶) تفاوت `chmod 2775` و `chmod 775` روی پوشه‌ی پروژه چیست؟ `2775` بیت SGID را روشن می‌کند؛ فایل‌های ساخته‌شده در آن پوشه گروه پوشه را به ارث می‌برند به‌جای گروه اصلی کاربر. برای پوشه‌های اشتراکی تیمی حیاتی است؛ بدون آن فایل‌ها گروه شخصی می‌گیرند و هم‌تیمی‌ها دسترسی ندارند.

۷) در دیسک `df -h` فضا هست ولی «No space left on device» می‌گیرم. چرا؟ احتمالاً `inode` تمام شده (`df -i`) — میلیون‌ها فایل کوچک یا `mail queue`. راه‌حل‌ها: پیدا کردن پوشه‌ی پرمصرف در `ode` (`find / -xdev -type f | wc -l`)، پاک‌سازی `/var/`، `spool`، `tmpwatch`، `systemd-tmpfiles`. علت دیگر: فایل حذف‌شده‌ای که فرایند باز نگه داشته (`lsof +L1`) یا سهمیه/ژورنال.

۸) `find / -size +100M` چه کار می‌کند؟ پارامترهای مهم دیگر `find`؟ فایل‌های بزرگ‌تر از ۱۰۰ مگابایت (بلوک ۵۱۲ بایتی برای `+` / `-`، `-maxdepth`، `-newer`، `-perm`، `-user/-group`، `-mtime/-mmin`، `-type f/d/l`، `-iname`، `-delete`، `-exec`، `-xdev`).

۹) تفاوت **rm** و **unlink**، و آیا راه بازیابی هست؟ **rm** رابط کاربرپسند است (سوئیچهای **-i/-r/-f**) و **unlink** فقط یک نام را حذف می‌کند (یک فایل، بدون بازگشت). داده فقط وقتی واقعاً آزاد می‌شود که آخرین فرایند بسته شود و شمارنده‌ی لینک صفر شود. بازیابی: اگر فرایند باز است از **/proc/<pid>/fd/N** کپی کن؛ وگرنه ابزارهایی مثل **extundelete/testdisk/photorec** روی **fs** دیسک **unmount** شده. راه بهتر: بکاپ و اسنپ‌شات.

۱۰) **inode** را در یک جمله توضیح بده و بگو چرا برای ذخیره‌سازی مهم است. ساختار متادیتای هر فایل که محل داده، مالک، مجوز و زمان‌ها را نگه می‌دارد و نام فایل بخشی از آن نیست؛ چون تعداد **inode** در زمان **mkfs** ثابت است، ذخیره‌سازی‌های «فایل کوچک و پرشمار» می‌توانند با **inode** اشباع شوند.

۱۱) **/etc/fstab** را با چه چیز می‌نویسی، **UUID** یا **/dev/sdb1**؟ چرا؟ **UUID** (یا **/dev/mapper/.../LABEL** برای **LVM**). نام دستگاه‌های **/dev/sdX** می‌تواند بین بوت‌ها جابه‌جا شود (اضافه‌شدن دیسک/USB) و **fstab** را بشکند؛ **UUID** پایدار است. برای شبکه **//server/share** + **_netdev** و برای USB‌های موقت **noatime**.

۱۲) **LVM** را در سه لایه توضیح بده و یک سناریوی کاری بگو. **PV** روی دیسک/پارتیشن، **VG** به‌عنوان **pool** مشترک، **LV** به‌عنوان حجم منطقی که فایل‌سیستم روی آن ساخته می‌شود. سناریو: دیسک ریشه بدون فضای آزاد است؛ دیسک جدید اضافه می‌کنم (**pvcreate**، **vgextend**)، سپس **lvextend -r** و فایل‌سیستم آنلاین رشد می‌کند – بدون **downtime**. مزیت دیگر: **snapshot** قبل از ارتقای بسته‌ها.

۹) چک‌لیست تکمیل روز ۲

- با **grep/sed/awk/cut/sort/uniq** یک گزارش از فایل لاگ ساختم
- مجوز **octal** و نمادین را بدون نگاه‌کردن به جدول می‌نویسم
- **umask** را تغییر دادم و تفاوت فایل/پوشه را دیدم
- **SUID/SGID/Sticky** را ساخته، تست و مستند کرده‌ام
- **ACL** برای یک کاربر و یک گروه ست کرده و **getfacl** را خوانده‌ام
- **sudo** محدود برای یک کاربر ساخته و با **sudo -l** تست کرده‌ام
- آزمایشگاه تیم‌ها (**devs/ops/audit**) را کامل اجرا کرده‌ام
- **find** با **-perm**، **-mtime**، **-size**، **-exec** تمرین کرده‌ام
- دیسک آزمایشی را پارتیشن‌بندی، **mkfs** و در **fstab** با **UUID** ثبت کرده‌ام
- یک **VG** با دو دیسک ساختم، **LV** ساختم و با **lvextend -r** بزرگش کردم
- به هر ۱۲ سؤال مصاحبه‌ی این روز بلند جواب داده‌ام

بسته‌ی مطالعه‌ی LPIC-1 – روز ۳

پوسته‌ها و اسکریپت‌نویسی **Bash**

Shell Types • Variables • Conditionals • Loops • Debugging

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۰۲ • روز ۳ (چهارشنبه) – مورنینگ ۹:۰۰–۱۳:۰۰ • ایوینینگ ۱۷:۰۰–۲۱:۰۰

روز ۳ – پوسته‌ها و اسکریپت‌نویسی Bash

مسیر مطالعه‌ی این روز (طبق پلن ۳۰ روزه): مورنینگ: (10:45–12:15) Variables → Shell Types (09:00–10:30) → Conditionals: if / test (12:30–13:00) ایوبینینگ: Script → Loops (18:45–20:15) → Lab: Bash Scripting (17:00–18:30) → Debugging (20:30–21:00)

هدف روز: شناخت انواع پوسته، متغیرها و محیط، شرط‌ها، حلقه‌ها، ورودی/خروجی، خطاگیری (set -euo pipefail) و تبدیل «کارهای تکراری» به اسکریپت قابل اتکا. این روز، ابزار بقیه‌ی هفته است.

0) مرور ۳ دقیقه‌ای روزهای ۱ و ۲

موضوع	امروز کجا به کار می‌آید
pipe و ریدایرکت (روز ۱)	اسکریپت‌ها ورودی/خروجی را همین‌طور مدیریت می‌کنند
grep/sed/awk (روز ۲)	بلوک‌های ساختمانی اسکریپت‌های واقعی
مجوز 755 و شَبَینگ	اسکریپت باید اجرایی باشد و مفسر داشته باشد
PATH (روز ۱)	اسکریپت‌های شخصی در ~/bin یا /usr/local/bin
/etc/fstab , LVM (روز ۲)	اسکریپت بکاپ و مانیتور فضا (تمرین امروز)

جمله‌ی وصل‌کننده: دو روز گذشته فرمان‌ها را تایپ می‌کردی؛ امروز همان فرمان‌ها را قابل تکرار، مستند و مطمئن می‌کنی – یعنی از «کاربر لینوکس» به «ادمین/مهندس خودکارسازی» تبدیل می‌شوی. در مصاحبه، تفاوت اصلی در همین است.

۱) انواع پوسته و محیط (09:00–10:30)

۱.۱ پوسته چیست و چه انواعی دارد؟

پوسته (Shell) یک مفسر فرمان است: خط را می‌خواند، بسط می‌دهد (متغیر، glob، آرگومان)، سپس اجرا می‌کند و نتیجه‌ی خروج را برمی‌گرداند.

پوسته	مسیر	ویژگی
sh / dash	/bin/sh	POSIX، سبک، اسکریپت‌های نصب و initrd (dash در Debian)
bash	/bin/bash	پیش‌فرض اکثر توزیع‌ها: تاریخچه، job control، آرایه، [[]]
zsh	/bin/zsh	تکمیل هوشمند، پلاگین (macOS پیش‌فرض)
ksh	/bin/ksh	یونیکس تجاری، اسکریپت‌های قدیمی
tcsh/csh	/bin/csh	C-like، تقریباً بازنشسته
fish	/usr/bin/fish	کاربرپسند، ولی نحو متفاوت (اسکریپت‌نویسی محدود)

```
cat /etc/shells # پوسترهای مجاز سیستم
echo $SHELL # پوستر ورود
chsh -s /bin/bash ali # تغییر پوستر (برای دیگران sudo)
exec bash # جایگزینی پوستر جاری بدون خروج از نشست
bash --posix script.sh # اجرای اسکریپت در حالت POSIX
```

شل ورود (login) در برابر شل غیرورود: شل ورود فایل‌های `/etc/profile` و یکی از `~/bash_login` / `~/bash_profile` را می‌خواند؛ شل تعاملی غیرورود (مثل باز کردن ترمینال جدید یا `bash`) `~/bashrc` را. شل غیرتعاملی (اسکریپت) هیچ‌کدام را مگر با `BASH_ENV` درست‌کردن «چرا در cron متغیرهای من نیست؟» از همین‌جا می‌آید.

```
/etc/profile -> ~/.bash_profile -> ~/.bashrc -> /etc/bash.bashrc
~/.bashrc -> /etc/bash.bashrc -> ترمینال
اسکریپت -> فقط محیط ارث‌بری‌شده
```

۱.۲ متغیرها: تعریف، بسط و محیط

```
name="Ali" # بدون فاصله دور = (خطای = فاصله «command not found»)
echo "$name" # نقل‌قول دوگانه: بسط انجام می‌شود
echo '$name' # نقل‌قول تک: خام
echo "${name}jan" # آکولاد برای جدا کردن نام از متن مجاور
readonly PI=3.14 # ثابت
unset name # حذف
export PATH="$PATH:$HOME/bin" # متغیر محیطی برای فرزندان
env | sort | head -20 # محیط فعلی
declare -p name # نوع و مقدار
```

سه سطح یعنی «متغیر شل» در برابر «متغیر محیطی»: متغیر شل فقط در همان پوستر است؛ با `export` به فرایندهای فرزند ارث می‌رسد. سرویس‌ها و cron، پوسترهای تو را ندارند → در اسکریپت‌ها محیط را صریح تنظیم کن.

```
# متغیرهای مهمی که باید بشناسی
echo "$USER $HOME $PWD $OLDPWD $SHELL $HOSTNAME"
echo "$PATH"
echo "$RANDOM" # عدد تصادفی
echo "$((5*3))" # حساب صحیح
date +"%F %T" # قالب زمان در اسکریپت
echo "$$" # PID (کاربرد: قفل فایل)
echo "$?" # کد خروج آخرین فرمان
echo "$# $@ $1" # تعداد/همه/اولین آرگومان
echo "${1:-default}" # مقدار پیش‌فرض اگر خالی بود
echo "${file%.txt}" # حذف کوتاه‌ترین پسوند از انتها
echo "${path##*/}" # فقط نام فایل از مسیر
echo "${path%/*}" # فقط پوشه
echo "${str//a/b}" # جایگزینی همه
echo "${#str}" # طول رشته
```

جدول پارامترهای موقعیتی و ویژه – از حفظ:

پارامتر	معنا
\$0	نام اسکریپت/پوسته
\$1..\$9 , \${10}	آرگومان‌ها
\$#	تعداد آرگومان
\$@	همه‌ی آرگومان‌ها (هر کدام جدا – در حلقه‌ها همین را بزن)
\$*	همه به صورت یک رشته
\$?	کد خروج آخرین فرمان (0 = موفق)
\$\$	PID پوسته
\$!	PID آخرین فرایند پس‌زمینه
\$-	فلگ‌های فعال پوسته

بسط‌های قدرتمند دیگر:

```
echo {1..5} # 1 2 3 4 5
echo {a..c}{1..2} # a1 a2 b1 b2 c1 c2
mkdir -p /tmp/{in,out,done} # ساخت چند پوشه
for f in *.log; do echo "$f"; done
$((RANDOM%100)) # حساب
$(command) یا `command` # جایگزینی فرمان (شکل دوم را استفاده نکن)
```

۱.۳ نقل قول‌ها، glob و escape – منبع ۹۰٪ اشکالات اسکریپت

شکل	رفتار	مثال
'...'	هیچ بسطی نمی‌شود	\$HOME → '\$HOME'
"..."	متغیر و () بسط می‌شود، glob نه	/root → "\$HOME"
بدون نقل قول	word splitting + glob	\$HOME → ممکن است شکسته شود
\x	خنثی‌سازی معنی خاص	\$HOME → \xHOME
\$(...)	جایگزینی فرمان	\$(date +%F)

```
# تفاوت مرگ‌آور در اسکریپت:
rm -rf $dir # خالی باشد dir اگر -> rm -rf / !!!
rm -rf "$dir" # بررسی خالی بودن
[ -n "$dir" ] || { echo "خالی است"; exit 1; }
```

✓ قانون شماره ۱ اسکریپت‌نویسی: همیشه **"\$var"** بنویس. هم اسپیس‌های درون نام فایل را حفظ می‌کند و هم از انفجار glob جلوگیری می‌کند.

```
ping -c1 10.20.0.1 >/dev/null 2>&1
if [ $? -eq 0 ]; then echo "UP"; else echo "DOWN"; fi

cmd1 && cmd2 # cmd1 فقط اگر cmd2 موفق (کد خروج 0)
cmd1 || cmd2 # cmd2 فقط اگر cmd1 ناموفق
cmd1 ; cmd2 # هر دو، بی‌توجه به نتیجه
false || echo "اجرا شد"
mkdir -p /a && cd /a && touch ok
```

کدهای خروج معنادار (برای مصاحبه): 0 = موفق، 1 = خطای عمومی، 2 = استفاده‌ی نادرست از فرمان، 126 = غیرقابل اجرا (مجوز)، 127 = فرمان پیدا نشد، n+128 = کشته‌شده با سیگنال (137 = SIGKILL/OOM، 143 = SIGTERM، 130 = Ctrl+C).n

۲) متغیرها و ورودی کاربر در اسکریپت (10:45–12:15)

۲.۱ اسکلت استاندارد یک اسکریپت حرفه‌ای

```
#!/usr/bin/env bash
# هدف: بکاپ دایرکتوری وب | نویسنده/تاریخ : backup.sh | نام
set -euo pipefail # -e توقف روی خطا، -u تعریف‌نشده، -o pipefail خطای pipe
IFS=$'\n\t' # جداکننده‌ی امن
shopt -s nullglob # حلقه روی الگوی بی‌نتیجه اجرا نشود

readonly SCRIPT_NAME="$(basename "$0")"
readonly BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

log() { printf '%s [%s] %s\n' "$(date +%F %T)" "${SCRIPT_NAME}" "$*"; }
die() { log "ERROR: $*" >&2; exit 1; }
trap 'log "خطا در خط $LINENO"' ERR
trap 'log "پایان با کد $?"' EXIT
```

گزینه	معنا و چرا مهم
<code>set -e</code>	اسکریپت روی اولین خطا متوقف می‌شود (بدون آن، ادامه می‌دهد و فاجعه می‌سازد)
<code>set -u</code>	استفاده از متغیر تعریف‌نشده خطا می‌دهد (جلوگیری از <code>rm -rf /</code>)
<code>set -o pipefail</code>	خطای وسط pipe هم باعث شکست می‌شود
<code>trap</code>	تمیزکاری در خروج (<code>trap cleanup EXIT</code>)، گرفتن خطا و سیگنال

```

if [ "$#" -lt 2 ]; then
    echo "usage: $0 <source-dir> <dest-dir>" >&2
    exit 2
fi
src="$1" ; dst="$2"
[ -d "$src" ] || die "مسیر مبدأ وجود ندارد" "$src"
mkdir -p "$dst" || die "ساخت مقصد ناموفق بود"

while [ "$#" -gt 0 ]; do # پردازش کلیدها به صورت دستی
    case "$1" in
        -v|--verbose) VERBOSE=1 ;;
        -n|--dry-run) DRY=1 ;;
        -*) die "کلید ناشناخته: $1" ;;
        *) files+=("$1") ;;
    esac
    shift
done

getopts "vh" opt # روش استاندارد POSIX
usage() { echo "usage: $0 [-v] [-h]"; }

```

در مصاحبه بپرسند «ورودی کاربر را چطور می‌گیری؟»: `read` برای تعاملی، `$1/$@` برای پارامتری، `getopts` برای کلیدها، و هرگز بدون اعتبارسنجی از ورودی استفاده نکن (تزییق فرمان: `$(...)` داخل ورودی اجرا می‌شود اگر در `eval` بگذاری).

۲.۳ `read` و ورودی/خروجی

```

read -r -p "نام سرویس: " svc # -r برای پشتیبانی از backslash
read -r -s -p "رمز: " pass # -s بصدای رمز
while IFS= read -r line; do echo ">> $line"; done < file.txt # الگوی طلایی خواندن خط به خط
ls -l /var/log | head -3 | while read -r f; do echo "$f"; done
printf '%s\n' "$var" # % و -n در برابر چاپ امن
echo "متن" 2>& stderr # خطا به stderr
exec > "/var/log/${SCRIPT_NAME}.log" 2>&1 # لاگ به لاگ

```

۳) شرط‌ها: test, [,], (13:00–12:30)

۳.۱ آزمون‌های پرکاربرد

معنا	آزمون رشته	معنا	آزمون فایل
رشته خالی است	-z "\$s"	وجود دارد	-e f
رشته غیرخالی	-n "\$s"	فایل معمولی	-f f
برابر (رشته‌ای)	"\$a" = "\$b"	دایرکتوری	-d f
نابرابر	"\$a" != "\$b"	خواندنی/نوشتنی/اجرایی	-r/-w/-x f
مقایسه‌ی الفبایی	"\$a" \< "\$b"	حجم < ۰	-s f
regex (فقط با [])	[[\$a =~ ^[0-9]+\$]]	جدیدتر از	f1 -nt f2

معنا	مقایسه‌ی عددی
برابر، نابرابر	-ne , -eq
کوچک‌تر، کوچک‌تر/مساوی	-le , -lt
بزرگ‌تر، بزرگ‌تر/مساوی	-ge , -gt

```
if [[ -f /etc/nginx/nginx.conf && -x /usr/sbin/nginx ]]; then ... fi
if [[ "$(id -u)" -ne 0 ]]; then die "root این اسکریپت با"; fi
if [[ "$os" == "ubuntu" || "$os" == "debian" ]]; then ... fi
if (( $(echo "$load > 5.0" | bc -l) )); then echo "load بالا"; fi # مقایسه‌ی اعشاری
```

تفاوت [] و [] : [] یک ساختار خود پویسته است: word splitting ندارد، && / || و regex (=~) و glob درونش کار می‌کند و خطای «unary operator expected» نمی‌دهد. داخل [] همه‌چیز باید نقل‌قول شود و && ندارد. در **bash** همیشه [] ، در POSIX/ sh باید [] .

۳.۲ if/elif/else , case , و مقایسه‌ی رشته‌ای دقیق

```
case "$1" in
  start) systemctl start nginx ;;
  stop) systemctl stop nginx ;;
  restart|reload) systemctl restart nginx ;;
  status) systemctl status nginx ;;
  *) echo "usage: $0 {start|stop|restart|status}" >&2; exit 2 ;;
esac

case "$(file -b --mime-type "$f")" in
  text/*) echo "text" ;;
  image/*) echo "image" ;;
  *) echo "other" ;;
esac
```

```
# پاک کردن آرگومان از کاراکترهای خطرناک
sanitize() { printf '%s' "$1" | tr -cd 'A-Za-z0-9._-/'; }
# فایل موقت امن
tmp="$(mktemp)" ; trap 'rm -f "$tmp"' EXIT
# قفل اجرای همزمان (جلوگیری از اجرای دو نسخه همزمان)
exec 9>/var/lock/myscript.lock ; flock -n 9 || die "نسخه دیگری در حال اجراست"
```

۴) آزمایشگاه ۱ – ساخت اسکریپت واقعی (17:00–18:30)

۴.۱ سناریو: اسکریپت بکاپ با چرخش (rotation) و سلامت‌سنجی

الزامات: مسیر مبدأ و مقصد از آرگومان؛ اگر مقصد نبود بسازد؛ فایل با **tar.gz** و مهر زمان؛ نگه‌داشتن فقط ۷ نسخه‌ی آخر؛ لاگ در **/var/**
log/backup.log؛ کد خروج معنادار؛ قابل اجرا با **cron**.

```
#!/usr/bin/env bash
# /usr/local/bin/backup.sh – نسخه ۷ نگهداری با چرخشی
set -euo pipefail
IFS=$'\n\t'

readonly KEEP=7
readonly LOG="/var/log/backup.log"
readonly NAME="$(basename "$0")"

log() { printf '%s [%s] %s\n' "$(date +%F %T)" "$NAME" "$*" | tee -a "$LOG"; }
die() { log "ERROR: $*" >&2; exit 1; }

[[ $# -eq 2 ]] || die "usage: $NAME <source-dir> <dest-dir>"
src="$1"; dst="$2"
[[ -d "$src" ]] || die "مبدأ نیست: $src"
mkdir -p "$dst" || die "مقصد ساخته نشد: $dst"

stamp="$(date +%Y%m%d-%H%M%S)"
base="$(basename "$src")"
archive="${dst}/${base}-${stamp}.tar.gz"

log "شروع بکاپ: $src -> $archive"
tar -czf "$archive" -C "$(dirname "$src")" "$base" || die "شکست خورد"
[[ -s "$archive" ]] || die "آرشیو خالی است!"

# سلامت‌سنجی: فهرست آرشیو و مقایسه‌ی تعداد فایل
n_src=$(find "$src" -type f | wc -l)
n_arc=$(tar -tzf "$archive" | grep -cv '/' || true)
log "تعداد فایل مبدأ=$n_src در آرشیو=$n_arc"
[[ "$n_arc" -ge "$n_src" ]] || die "ناسازگاری تعداد فایل!"

# چرخش: حذف نسخه‌های قدیمی
mapfile -t olds <<(ls -lt "${dst}/${base}/*.tar.gz" >/dev/null | tail -n +$(KEEP+1))
for f in "${olds[@]:-}"; do [[ -n "$f" ]] && rm -f -- "$f" && log "حذف قدیمی: $f"; done

log "حجم: $(du -h "$archive" | cut -f1)"
exit 0
```

```
sudo install -m 750 backup.sh /usr/local/bin/backup.sh
sudo /usr/local/bin/backup.sh /etc /var/backups
sudo crontab -e
# 0 2 * * * /usr/local/bin/backup.sh /etc /var/backups
```

۴.۲ تست‌هایی که حرفه‌ای‌ها انجام می‌دهند

```
bash -n backup.sh # فقط بررسی نحو
shellcheck backup.sh # همیشه بزن – (نصب: apt install shellcheck) تحلیل ایستا
sudo bash -x backup.sh /etc /var/backups | head -40 # ردیابی اجرا (debug)
/usr/local/bin/backup.sh /no/such/dir /tmp ; echo "exit=$?" # باید 1 بدهد
for i in {1..10}; do sudo /usr/local/bin/backup.sh /etc /tmp/bk >/dev/null; done # تست چرخش
ls -lt /tmp/bk | wc -l # باید ۷ باشد
```

#	خرابی	علامت	تشخیص/رفع
1	حذف شِبنگ (<code>#!/usr/bin/env bash</code>)	اجرا با <code>sh</code> و خطای <code>[[: not found</code>	شِبنگ را برگردان، <code>chmod +x</code>
2	<code>chmod 644 backup.sh</code>	<code>Permission denied</code> هنگام اجرا	<code>chmod 750</code>
3	نقل‌قول نداشتن <code>\$src</code> و پوشه با فاصله	مسیر نصفه، <code>tar: ... not found</code>	<code>"\$src"</code>
4	<code>set -e</code> را کامنت کن و <code>tar</code> را خراب کن	چرخش روی آرشیو خالی انجام می‌شود	بازگرداندن <code>set -euo pipefail</code>
5	<code>KEEP=0</code>	همه‌ی بکاپ‌ها پاک می‌شوند	اعتبارسنجی ورودی: <code>[[\$KEEP -ge 1]]</code>
6	خط CRLF (فایل روی ویندوز ویرایش‌شده)	<code>bad interpreter: ^M</code>	<code>sed -i 's/\r\$//'</code> یا <code>dos2unix</code>

۵) حلقه‌ها (18:45–20:15)

۵.۱ for – چهار شکل

```
for i in 1 2 3; do echo "$i"; done
for i in {1..5}; do echo "$i"; done
for i in $(seq 10 -2 2); do echo "$i"; done
for f in /var/log/*.log; do echo "== $f =="; done
for ((i=0; i<5; i++)); do echo "شمارنده: $i"; done # سیک C
for host in 10.20.0.{1..5}; do ping -c1 -W1 "$host" >/dev/null && echo "$host UP" || echo "$host DOWN"; done
while read -r user uid; do echo "$user:$uid"; done <<(awk -F: '{print $1, $3}' /etc/passwd)
```

۵.۲ while و until

```
i=0 ; while [[ $i -lt 3 ]]; do echo "$i"; ((i++)); done
until systemctl is-active --quiet nginx; do echo "در انتظار nginx..."; sleep 2; done
while :; do echo "(برای خروج Ctrl+C)"; sleep 1; done
while true; do
    if ping -c1 -W1 10.20.0.1 >/dev/null; then break; fi
    sleep 2
done
```

شمارنده‌ها در bash: `((i++))`، `((i+=5))`، `let i++`، `i=$((i+1))`. توجه: `((i++))` وقتی `i=0` باشد **کد خروج 1** می‌دهد (چون مقدار قبلی صفر است) و با `set -e` اسکریپت را می‌کشد – این تله‌ی معروف را در مصاحبه دوست دارند: راه‌حل `i=$((i+1))` یا `((i+=1))` `|| true`.

```
for u in alice bob carol; do
  [[ "$u" == "bob" ]] && continue # از این عبور کن
  [[ "$u" == "carol" ]] && break # حلقه را ببند
  echo "بررسی $u"
done
```

۵.۴ کاربرد واقعی: گزارش روزانه‌ی سلامت سرور

```
#!/usr/bin/env bash
# health.sh - (07:00 کرون روزانه) گزارش سریع سلامت سرور
set -uo pipefail
printf '%-28s %s\n' "=== گزارش سلامت $(hostname) $(date +%F %T) ==="
printf '%-28s %s\n' "Uptime" "$(uptime -p)"
printf '%-28s %s\n' "Load(1/5/15)" "$(cut -d ' ' -f1-3 /proc/loadavg)"
printf '%-28s %s\n' "RAM" "$((free -h | awk '/Mem:/{print $3}/"$2" used, "$7" avail"}')")
printf '%-28s %s\n' "Swap" "$((free -h | awk '/Swap:/{print $3}/"$2"')")
echo "--- % سیستم‌های بالای 80 ---"
df -hP | awk 'NR>1 && int($5) >= 80 {print $6, $5}'
echo "--- % های بالای inode80 ---"
df -iP | awk 'NR>1 && int($5) >= 80 {print $6, $5}'
echo "--- سرویس‌های failed ---"
systemctl --failed --no-legend | awk '{print $1}' || true
echo "--- خطای sshd آخر ۵ ---"
journalctl -u ssh -n 200 --no-pager 2>/dev/null | grep -i "failed" | tail -5
echo "--- گوش‌دهنده‌های شبکه ---"
ss -tulnp | grep -v "127.0.0.1\|:::1" | head -10
```

۶) خطاگیری و اشکال‌زدایی (20:30–21:00)

۶.۱ ابزارهای دیباگ

```
bash -n script.sh # بررسی نحو بدون اجرا
bash -x script.sh # ردیابی خط‌به‌خط (PS4 را سفارشی کن: export PS4='+$LINENO: ')
set -x ; ... ; set +x # اسکرپت از اسکرپت
shellcheck -S style script.sh # توصیه‌های سبک هم می‌دهد
time script.sh # اندازه‌گیری زمان
strace -f -e trace=openat,execve script.sh 2>&1 | tail -30 # فراخوانی‌های سیستمی
bash -c 'echo "$BASH_VERSION"' # نسخه‌ی bash
```

```
trap 'echo "خطا در خط $LINENO, 2< "$?" ERR' ERR
trap 'echo "پاک‌سازی..."; rm -f "$tmp"' EXIT
```

```
exec ls -l /tmp # جایگزینی پوسته با فرمان (بدون فرزند جدید)
( cd /tmp && ls ) # زیرپوسته: تغییر پوشه بیرون اثر ندارد
{ cd /tmp && ls; } # بلوک جاری: تغییر پوشه بیرون اثر دارد
( umask 077; touch secret.txt ) # تنظیم موقت بدون آلودگی محیط
```

تفاوت () و { } سؤال محبوب مصاحبه است: زیرپوسته‌ی () محیط جدا می‌سازد، بلوک { } در همان پوسته اجرا می‌شود (و نیاز به ; انتهای آخرین فرمان و فاصله‌ی اجباری قبل از { دارد).

۶.۳ خروجی/ورودی پیشرفته

```
command > out.txt 2> err.txt # جدا
command &> all.txt # ادغام
command 2>/dev/null # خفه کردن خطا (فقط وقتی مطمئنی!)
exec 3< file.txt # توصیفگر دلخواه برای خواندن
while read -r l <&3; do echo "$l"; done ; exec 3<&-
cmd | tee -a log.txt | grep -i error
printf '%s\n' "${array[@]}" # چاپ آرایه
declare -A map=( [a]=1 [b]=2 ) ; echo "${map[a]}" # آرایه‌ی انجمنی (bash 4+)
readarray -t lines < file.txt ; echo "${#lines[@]}"
```

Self-Test ۳ (۱۵ سؤال)

1. تفاوت 'single' و "double" و بدون نقل‌قول؟
2. چرا `rm -rf $dir` خطرناک است و راه امن چیست؟
3. تفاوت `@` و `*` و کدام در حلقه؟
4. کد خروجی 127 و 126 و 137 چه معنایی دارند؟
5. `set -euo pipefail` هر بخش چه می‌کند؟
6. تفاوت `[]` و `[[ ]]` و کدام در `sh`؟
7. تفاوت `while read -r line; do ... done < f` و `cat f | while read ...`؟
8. تفاوت `( cd /tmp; ls )` و `{ cd /tmp; ls; }`؟
9. تفاوت `$(...)` و `${...}`؟
10. `case` را برای `start|stop|restart` بنویس.
11. `mktemp` چه مشکلی را حل می‌کند؟
12. `flock` در اسکریپت کرون چرا لازم است؟
13. `((i++))` با `set -e` چه تله‌ای دارد؟
14. `bash -x` و `shellcheck` هر کدام چه می‌دهند؟
15. در `cron` چرا اسکریپت کار نمی‌کند ولی دستی کار می‌کند؟ (سه علت با راه‌حل)

۸) بانک سؤال مصاحبه‌ی روز ۳ (۱۴ سؤال با پاسخ مدل)

۱) **شل چیست و چرا bash؟** لایه‌ی تفسیر فرمان بین کاربر و کرنل؛ خواندن، بسط و اجرا و مدیریت job. bash پیش‌فرض اکثر توزیع‌ها، سازگار با POSIX، دارای آرایه / [] و مستندات فراوان؛ برای اسکریپت قابل انتقال `#!/bin/sh` + نحو POSIX.

۲) **تفاوت متغیر محیطی و متغیر شل؟** متغیر شل فقط در همان نمونه‌ی پوسته وجود دارد؛ با `export` وارد محیط فرایند می‌شود و به فرزندان ارث می‌رسد. سرویس‌ها/cron محیط شل تعاملی تو را ندارند؛ در unit/systemd یا کرون، متغیر را صریح تعریف کن.

۳) **چگونه ورودی را امن اعتبارسنجی می‌کنی؟** چک تعداد آرگومان، `getopts` برای کلیدها، بررسی وجود مسیرها، `case` روی مقادیر مجاز، reject کاراکترهای خطرناک (`'_-.0-9A-Za-ztr -cd`)، نقل‌قول کامل، `--` قبل از مسیرها برای پایان کلیدها، و هرگز `eval` روی ورودی کاربر. در PowerShell/Python هم همین اصل: whitelist نه blacklist.

۴) **یک اسکریپت بنویس که اگر دیسک بالای ۸۵٪ شد ایمیل هشدار بدهد (خطوط کلیدی).**

```
df -hP | awk 'NR>1 && int($5) >= 85 {print $6, $5}' | while read -r mnt use; do
  printf 'WARN %s usage %s\n' "$mnt" "$use" | mail -s "Disk ${use} on $(hostname)" ops@example.com
done
```

نکته‌ها: از `df -P` (خروجی POSIX) استفاده کن، روی مقصدهایی که `tmpfs` هستند فیلتر بگذار، در cron با `MAILTO=` لاگ را کنترل کن، و بهترین حالت: ارسال به سیستم مانیتورینگ (Zabbix/Prometheus node_exporter) نه ایمیل خام.

۵) **trap را توضیح بده و یک کاربرد واقعی.** `trap` ثبت هندلر برای سیگنال‌ها یا pseudosignal های `EXIT`، `ERR`، `DEBUG` است. کاربرد: پاک‌کردن فایل موقت و باز کردن قفل در خروج — `tmp=$(mktemp); trap 'rm -f "$tmp"' EXIT` — که در اسکریپت‌های طولانی و کرون حیاتی است تا روی خطا زباله باقی نماند.

۶) **چطور اسکریپت را در تمام سرورها اجرا می‌کنی؟** گزینه‌ها: حلقه‌ی `ssh` روی لیست میزبان‌ها `for h in $(cat hosts); do ssh` یا `Ansible` (`ansible all -m script -a script.sh`)، `bash -s' < script.sh; done` ("h" \$) یا ابزارهای پیکربندی مثل Puppet/Chef. برای ۲ سرور اسکریپت، برای ۲۰۰ سرور Ansible.

۷) **تفاوت > و 2>&1 را در cmd > f 2>&1 و cmd 2>&1 > f؟** در اولی stderr به همان جایی که stdout اشاره می‌کند (فایل) وصل می‌شود؛ در دومی stderr به ترمینال فعلی وصل و بعد stdout به فایل منتقل می‌شود. ترتیب ریدایرکت‌ها معنا دارد چون چپ‌به‌راست ارزیابی می‌شوند.

۸) **تفاوت bash script.sh و ./script.sh و source script.sh؟** `bash script.sh` : مفسر جدید؛ به مجوز اجرا نیاز ندارد. `./` . `script.sh` : شَبَنگ تعیین می‌کند چه مفسری، به `+x` نیاز دارد. `source` : در پوسته‌ی جاری اجرا می‌شود، متغیرها و `cd` باقی می‌مانند (برای `source ~/.bashrc`).

۹) **یک اسکریپت با اجرای هم‌زمان (parallel) می‌خواهی که منتظر همه بماند.**

```
for h in 10.20.0.{1..20}; do ping -c1 -W1 "$h" >/dev/null && echo "$h UP" & done
wait
echo "همه پوشه‌ها تمام شد"
```

با محدودیت هم‌زمانی: `parallel` یا `xargs -P 10 -I{} sh -c 'ping -c1 -W1 {} >/dev/null && echo {} UP' < hosts` با کنترل تعداد.

۱۰) **روی فایل‌های کانفیگ را در اسکریپت چطور ایمن می‌کنی؟** الگو را دقیق و anchored بنویس (نه عمومی)، `-i.bak` بگذار، قبل از نوشتن نسخه‌ی پشتیبان با مهر زمان بساز، تست صحت بعد از تغییر (`nginx -t`، `sshd -t`)، و در صورت خطا rollback خودکار با `trap`. هرگز روی همه‌ی فایل‌های یک پوشه به‌صورت بازگشتی `sed -i` نزن.

۱۱) **«کد خروج» چه اهمیتی در خودکارسازی دارد؟** ابزارهای خودکارسازی (Ansible, Jenkins, cron, systemd) موفقیت/شکست را از کد خروج می‌فهمند. اسکریپتی که همیشه ۰ برمی‌گرداند، خطاها را پنهان می‌کند و alerts نمی‌سازد؛ اسکریپت حرفه‌ای کدهای متفاوت برای «خطای ورودی»، «خطای اجرا» و «خطای وابستگی» برمی‌گرداند.

۱۲) تفاوت `$()` و `backtick` و چرا اولی را ترجیح می‌دهیم؟ هر دو جایگزینی فرمان انجام می‌دهند؛ `$()` تودرتو خواناتر و درست کار می‌کند (`$(echo $(date +%F))`) و شل مدرن از آن پشتیبانی می‌کند؛ `backtick` در باطل‌سازی نقل‌قول و تودرتویی مشکل‌ساز است.

۱۳) اسکریپت باید «idempotent» باشد – یعنی چه و چطور؟ اجرای چندباره همان نتیجه را بدهد و خرابی نسازد. روش‌ها: بررسی پیش‌شرط قبل از عمل (`touch f` || `[[ -f f ]]`)، `mkdir -p` به جای `mkdir`، استفاده از `systemctl enable --now` (تری‌گرهایی که خطا نمی‌دهند)، پشتیبان‌گیری با مهر زمان، و بازنویسی کامل فایل کانفیگ به جای `append` بی‌قید.

۱۴) کد خروج 137 در یک سرویس چه معنایی دارد و چطور عیب‌یابی می‌کنی؟ $9 + 128 = \text{SIGKILL}$ ؛ رایج‌ترین علت **OOM Killer** است. بررسی `memory limit` در کانتنر/cgroup، `journalctl -k | grep -i oom`، `dmesg | grep -i -E "oom|killed process"`، اندازه‌گیری مصرف واقعی با `smem / ps_mem`، سپس رفع با محدودسازی، افزایش RAM یا اصلاح نشنث حافظه.

۹) چک‌لیست تکمیل روز ۳

- تفاوت شل ورود/غیرورود را با `~/.bash_profile` و `~/.bashrc` تست کرده‌ام
- `declare`، `readonly`، `unset`، `export` و بسط‌های `${var...}` را تمرین کرده‌ام
- اسکریپت `backup.sh` را ساختم، اجرا کردم و چرخش ۷ نسخه را تست کردم
- `set -euo pipefail` و `trap` را در اسکریپت به کار برده‌ام
- با `getopts` و `case` کلید پردازش کرده‌ام
- حلقه‌های `for` (سبک C، لیست، `glob`) و `while read -r` را نوشته‌ام
- اسکریپت `health.sh` را زدم و در `crontab` روزانه ثبتش کردم
- با `bash -n`، `bash -x` و `shellcheck` یک اسکریپت را دیباگ کرده‌ام
- `flock` را برای جلوگیری از اجرای هم‌زمان آزمایش کرده‌ام
- ۶ تمرین خراب‌کن و درست‌کن این روز را انجام داده‌ام

بسته‌ی مطالعه‌ی LPIC-1 – روز ۴

مدیریت فرایند و کنترل کار

Processes • ps/top • Signals • nice/renice • Job Control • cron/at/timers

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۲ • روز ۴ (پنجشنبه) – مورنینگ ۹:۰۰–۱۳:۰۰ • ایوبینینگ ۱۷:۰۰–۲۱:۰۰

روز ۴ – مدیریت فرایند و کنترل کار (Job Control)

مسیر مطالعه‌ی این روز (طبق پلن ۳۰ روزه): مورنینگ: (۰۹:۰۰–۱۰:۳۰) Processes Overview → `kill`، `ps`، `top`، `pstree` (۱۰:۴۵–۱۲:۱۵) → `nice`، `renice`، `nohup` (۱۲:۳۰–۱۳:۰۰) ایوبینینگ: (۱۳:۰۰–۱۷:۳۰) Lab: Job Control → `nice`، `renice` (۱۸:۴۵–۲۰:۱۵) → `cron` و زمان‌بندی (۲۱:۰۰–۲۲:۳۰)

هدف روز: بفهمی فرایند چگونه ساخته و زمان بندی می شود، فرایندها را ببینی و اولویت بندی کنی، سیگنال ها و تفاوت SIGTERM / SIGKILL را بدانی، job control را در ترمینال بچرخانی، و کارها را با cron / at / systemd timers زمان بندی کنی.

0) مرور ۳ دقیقه ای روزهای ۱ تا ۳

موضوع	امروز کجا به کار می آید
<code>/proc/<pid></code> (روز ۱)	هر فرایند یک پوشه در <code>/proc</code> دارد؛ منبع اطلاعات <code>top</code> , <code>ps</code>
مجوز و مالکیت (روز ۲)	تنها مالک <code>root</code> می تواند سیگنال بفرستد <code>kill</code> با <code>EPERM</code>
<code>!</code> و <code>&</code> و <code>wait</code> (روز ۳)	همان کنترل جاب، اینجا عمیق تر: <code>bg</code> , <code>fg</code> , <code>jobs</code>
<code>cron</code> در مثال بکاپ (روز ۳)	امروز کامل: <code>systemd timer</code> , <code>at</code> , <code>anacron</code> , <code>crontab</code>
<code>?</code> و کد خروج (روز ۳)	کد خروج فرایند = زبان ارتباطی ابزارهای مانیتورینگ

جمله ی وصل کننده: تا دیروز «چه چیزی اجرا شود» را می نوشتی؛ امروز می فهمی چه چیزی در حال اجراست، چقدر منابع می خورد، و چه وقت باید بمیرد – دقیقاً کار روزمره ی یک ادمین در ترابل شوت («سرور کند شده»).

۱) فرایند چیست؟ (09:00–10:30)

۱.۱ فرایند، برنامه، نخ (Thread)

مفهوم	تعریف
برنامه (Program)	فایل روی دیسک؛ ثابت و غیرفعال
فرایند (Process)	برنامه ی در حال اجرا با PID، فضای آدرس، مالک، حالت
نخ (Thread)	جریان اجرای موازی داخل یک فرایند؛ حافظه را مشترک دارند
دیمن (Daemon)	فرایند پس زمینه بدون ترمینال کنترل (<code>systemd</code> , <code>nginx</code> , <code>sshd</code>)

fork و exec – دو سیسکال بنیادی:

- پوسته با `fork` یک کپی از خودش می سازد (فرایند فرزند).
- فرزند با `exec` حافظه ی خود را با برنامه ی جدید جایگزین می کند.
- والد با `wait` منتظر کد خروج می ماند.

```
strace -f -e trace=clone,execve,wait4 bash -c 'ls /tmp' 2>&1 | head -20 # دیدن این جریان در عمل
```

حالت	معنا	راه حل
Running (R)	روی CPU یا در صف آزاد CPU	-
Sleeping (S/D)	منتظر رویداد؛ D خواب غیرقابل قطع (I/O)	بررسی دیسک/ماونت
Stopped (T)	متوقف (SIGSTOP یا Ctrl+Z)	<code>kill -CONT / bg / fg</code>
Zombie (Z)	تمام شده ولی والدش <code>wait</code> نگرفته	حل با والد؛ اگر والد خراب است، والد را restart کن
Orphan	والدش مرده؛ <code>init/systemd</code> (PID 1) سرپرستی می گیرد	نیازی به اقدام نیست

```
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/ {print}' # شکار زامبی‌ها
ps -eo pid,ppid,user,stat,cmd --sort=-pcpu | head -10 # CPU پرمصرف‌ترین
pstree -p 1234 # درخت فرزندان
```

نکته: زامبی هیچ منبعی جز یک ورودی در جدول فرایند نمی‌خورد؛ اگر زیاد شوند، **والد** مشکل دارد نه فرزند. (سؤال کلاسیک مصاحبه).

۱.۳ PID و namespace

```
echo $$ # PID پوسته‌ی جاری
pgrep -a nginx # PID + خط فرمان
pidof nginx # فقط PID
ls -ld /proc/[0-9]* | wc -l # تعداد فرایندهای فعلی
cat /proc/$$/status | head -20 # جزئیات فرایند جاری
cat /proc/1/cmdline | tr '\0' ' '; echo # PID 1 چیست (systemd/init/نتینر)
systemctl status <svc> | head -5
```

PID 1 در سرورهای سنتی `systemd` (یا `init`) و در کانتینر معمولاً برنامه‌ی اصلی است؛ اگر PID 1 در کانتینر سیگنال‌ها را مدیریت نکند، `docker stop` با `SIGTERM` کار نکرده و بعد از timeout با `SIGKILL` کشته می‌شود (پل مستقیم به روز ۵ و کار کانتینری).

۲ دیدن فرایندها: `ps`, `top`, `htop`, `uptime` (10:45–12:15)

۲.۱ ps – ترکیب‌های ضروری

```
ps aux # زمان شروع، فرمان، CPU، MEM، کاربر: BSD سبک
ps -ef # ستون‌های مختلف PPID System V: سبک
ps aux --sort=-%mem | head # پرمصرف‌ترین حافظه
ps -ef | grep -v grep | grep nginx
ps -eo pid,ppid,user,stat,%cpu,%mem,etime,cmd
ps -eLf # نمایش نخ‌ها (LWPs)
ps -C nginx -o pid,etime,%cpu
ps -p 1234 -o pid,cmd,lstart # این فرایند چه وقت شروع شده
watch -n2 'ps aux --sort=-%cpu | head -8'
```

رمزگذاری ستون **STAT**: R اجرا، S خواب، D خواب IO، T متوقف، Z زامبی، < اولویت بالا، N nice مثبت (اولویت پایین)، s سرپرست نشست، l چندنخی، + در گروه پیش‌زمینه.

حالت دسته‌ای (برای لاگ/اسکرپت) `top -d 2 -o %CPU -n 1 -b > topdump.txt #`

کلیدهای **top** که باید بلد باشی:

کلید	کار
1	نمایش هر CPU جدا
M / P	مرتب‌سازی بر اساس CPU / حافظه
k	کشتن فرایند (PID می‌پرسد)
r	renice کردن
c	نمایش خط فرمان کامل
u	فیلتر بر اساس کاربر
H	نمایش نخ‌ها
f	انتخاب ستون‌ها
W	ذخیره‌ی تنظیمات در <code>~/.toprc</code>

خواندن هدر **top** (سؤال طلایی):

```
top - 10:04:12 up 3 days, 2:14, 2 users, load average: 1.20, 0.85, 0.42
Tasks: 178 total, 2 running, 176 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.9 us, 2.0 sy, 0.0 ni, 91.1 id, 0.5 wa, 0.5 hi, 0.0 si, 0.0 st
MiB Mem : 3934.0 total, 412.3 free, 1190.5 used, 2331.2 buff/cache
MiB Swap: 2048.0 total, 1900.0 free, 148.0 used. 2210.9 avail Mem
```

بخش	معنا	نشانه‌ی مشکل
load average	میانگین فرایندهای آماده/منتظر در ۱/۵/۱۵ دقیقه	بالا تر از تعداد هسته‌ها = صف
us/sy	زمان کاربر/کرنل	sy بالا = سیسکال/دراپور یا حمله
wa	انتظار I/O	wa بالا = دیسک کند یا مشکل ذخیره‌سازی
st	زمان دزدیده‌شده (مجازی‌سازی)	st بالا = هایپروایزر شلوغ
buff/cache	کش کرنل (قابل آزادسازی)	فضای آزاد کم ولی avail خوب = نرمال
si/so	swap in/out	غیرصفر شدن = فشار حافظه

قاعده‌ی load average: مقدار را تقسیم بر تعداد CPU کن؛ **1 <** سالم، **1-2** شلوغ، **2 >** مشکل جدی. یا از `nproc` استفاده کن: `echo "load: $(cut -d' ' -f1-3 /proc/loadavg) cores: $(nproc)"`

```
htop -u nginx # تعاملی با نوار و درخت (apt install htop)
atop 5 3 # فرایندی با تاریخچه سیستمی +
iostat -xz 2 5 # latency فشار دیسک و #
iotop -oPa # فرایند I/O کدام
vmstat 2 5 # در زمان CPU/swap/خلاصه حافظه #
sar -u 1 5 ; sar -r 1 5 # تاریخچه سیستم (sysstat)
pidstat -d 2 5 # هر فرایند I/O
free -h ; cat /proc/meminfo | head
```

نقشه‌ی عیب‌یابی «سرور کند شده است»:

علامت	ابزار	علت محتمل
load بالا + CPU بالا	<code>ps --sort=-%cpu</code> , <code>top</code>	فرایند سرکش، کد بد، ماینر
load بالا + CPU کم + <code>wa</code> بالا	<code>iotop</code> , <code>iostat</code>	دیسک کند/خراب، NFS، بکاپ سنگین
مصرف حافظه بالا + swap	<code>smem</code> , <code>vmstat</code> , <code>free</code>	نشت حافظه، کمبود RAM
تعداد فرایند زیاد + زامبی	<code>pstree</code> , <code>ps</code>	باگ اپلیکیشن در مدیریت فرزند
شبکه اشباع	<code>dstat -n</code> , <code>nload</code> , <code>iftop</code> , <code>ss</code>	ترافیک/حمله (پل به روز ۶)

۳) کنترل فرایند: سیگنال‌ها، `kill` , `killall` , `nice` (12:30–13:00 و ادامه‌ی عصر)

۳.۱ سیگنال‌ها – زبان صحبت با فرایند

شماره	نام	پیش‌فرض	کاربرد
1	SIGHUP	پایان	بارگذاری مجدد کانفیگ (دیمن‌ها)
2	SIGINT	پایان	همان <code>Ctrl+C</code>
3	SIGQUIT	Core dump	خروج با تولید کور
9	SIGKILL	پایان	غیرقابل گرفتن/نادیده‌گرفتن – آخرین راه
15	SIGTERM	پایان	درخواست مؤدبانه‌ی پایان (پیش‌فرض <code>kill</code>)
18/19	SIGCONT/SIGSTOP	ادامه/توقف	توقف سخت و ادامه
20	SIGTSTP	توقف	همان <code>Ctrl+Z</code>
10/12	SIGUSR1/2	آزاد	تعریف اپلیکیشن (مثل reopen لاگ)
28	SIGWINCH	نادیده	تغییر اندازه‌ی ترمینال

```
kill -l # لیست تمامی سیگنالها
kill 1234 # پیشفرض = SIGTERM (15)
kill -TERM 1234 ; kill -15 1234 # همان
kill -9 1234 # کشتن قهری (SIGKILL)
kill -HUP $(cat /run/nginx.pid) # نرم‌افزاری reload
pkill -u ali -f "python worker" # بر اساس کاربر و الگوی خط فرمان
pkill -TERM -x nginx # مطابق دقیق نام
killall nginx # تمامی نمونه‌های یک نام
pgrep -a -f "celery" # اول پیدا کن، بعد بکش (روش حرفه‌ای)
ss -tlnp | grep 8080 # چه کسی پورت را گرفته
fuser -v 8080/tcp ; fuser -k 8080/tcp # (با احتیاط) چه کسی و کشتن آن
lsof -i :8080
```

⚠ **قانون طلایی:** همیشه اول **SIGTERM** (15) بفرست تا فرایند فرصت پاک‌سازی/بستن فایل داشته باشد؛ **SIGKILL** (9) را برای فرایند قفل‌کرده بگذار. **kill -9** روی دیتابیس = ریسک خرابی فایل‌های داده.

kill -0 PID هیچ سیگنالی نمی‌فرستد و فقط بررسی می‌کند که فرایند وجود دارد و اجازه‌ی سیگنال‌دادن داری — ابزار استاندارد بررسی در اسکریپت:

```
if kill -0 "$pid" 2>/dev/null; then echo "در حال اجراست"; else echo "مرده است"; fi
```

۳.۲ اولویت‌بندی: **nice** و **renice**

محدوده‌ی **nice** از -20 (بالاترین اولویت) تا 19 (پایین‌ترین)؛ پیش‌فرض 0. فقط **root** می‌تواند **nice** منفی بدهد.

```
nice -n 10 tar -czf big.tgz /data # اجرا با اولویت پایین
sudo nice -n -5 /usr/bin/important-task # (نیاز به root) اولویت بالا
renice -n 5 -p 1234 # تغییر اولویت فرایند جاری
sudo renice -n -10 -u www-data # تمامی فرایندهای یک کاربر
ionice -c3 -p 1234 # I/O اولویت (idle کلاس)
chrt -f 10 ./realtime-task # (با احتیاط شدید) real-time زمان‌بندی
ps -eo pid,ni,cmd | head
```

نکته‌ی مصاحبه: **nice** روی CPU اثر می‌گذارد، ولی I/O را کنترل نمی‌کند؛ برای بکاپ‌های سنگین که دیسک را می‌خواهاند **ionice -c3** یا **ionice -c2 -n7** بگذار. (روی سرورهای دیتابیس، بکاپ شبانه + **ionice** + **nice** = الگوی حرفه‌ای).

۳.۳ محیط فرایند و **disown / setsid / nohup**

```
nohup long-task > /var/log/long.log 2>&1 & # SSH مقاوم به قطع
disown -h %1 # job جدا کردن
setsid long-task </dev/null >/tmp/t.log 2>&1 & # نشست جدید مستقل
sleep 1000 & jobs ; fg %1 ; bg %1 # کنترل جاب
Ctrl+Z -> bg # توقف و ادامه در پس‌زمینه
```

برای کارهای طولانی در سرور، **screen** یا **tmux** استاندارد عملی است: **tmux attach** ; **tmux ls** ; **bash tmux new -s work** ; **tmux kill-session -t work** (جدا شدن) , **D** , **Ctrl+B** ; **-t work**

۴) آزمایشگاه ۱ – کنترل جاب و رفتار سیگنال‌ها (17:00–18:30)

۴.۱ کنترل جاب گام به گام

```
sleep 300 & # شروع در پس‌زمینه
sleep 400 &
jobs -l # وضعیت PID جاب‌ها با
fg %1 # آوردن جاب ۱ به پیش‌زمینه
# Ctrl+Z: داخل اجرا
jobs # است Stopped اکنون
bg %2 # ادامه در پس‌زمینه
kill %1 # (نه PID) کشتن جاب با شماره
jobs ; wait # منتظر پایان همه
```

۴.۲ آزمایش رفتار سیگنال‌ها با یک اسکریپت نمونه

```
#!/usr/bin/env bash
# مدیریت سیگنال SIGTERM و SIGKILL نمایش تفاوت
trap 'echo "[$(date +%T)] SIGTERM خروج"; exit 0' TERM
trap 'echo "[$(date +%T)] SIGINT (Ctrl+C)"' INT
trap 'echo "[$(date +%T)] SIGHUP reload می‌کنم"' HUP
trap 'echo "[$(date +%T)] EXIT دارم خارج می‌شوم"' EXIT
echo "PID=$$ سیگنال بفرست: kill -TERM $$"
while true; do sleep 2; done
```

```
chmod +x /tmp/sig-demo.sh ; /tmp/sig-demo.sh & # اجرا
demo=$!
kill -HUP $demo # reload باید پیام
kill -TERM $demo # باید پیام پاک‌سازی و خروج بدهد
# تفاوت 9 و 15 trap، آزمایش دوم: بدون
python3 -c "import time; time.sleep(300)" & p=$!
kill -9 $p ; sleep 0.5 ; ps -p $p -o pid,stat,cmd # فرایند سریعاً مرده است
```

۴.۳ شبیه‌سازی سناریوی «سرور کند شده است» (تمرین طلایی این روز)

```
# ۱) ساخت بار CPU
sudo nice -n 5 bash -c 'while :; do :; done' & # یک هسته را پر کن
uptime ; top -b -n1 | head -12
ps -eo pid,ppid,user,%cpu,cmd --sort=-%cpu | head -5

# ۲) ساخت فشار حافظه و swap
python3 -c '<<'PY' &
a=[]
while True: a.append(' ' * 50_000_000) # تا OOM با زود ادامه می‌دهد؛ Ctrl+C ببند
PY
free -h ; vmstat 2 3

# ۳) خاتمه‌ی تمیز
pkill -f 'while : ' ; pkill -f '50_000_000'
```

پس از هر بخش، این پرسش‌ها را بلند جواب بده:

1. load average چقدر شد و نسبت به `nproc` چه می‌گوید؟
2. در `top` کدام ستون‌ها جهش کردند (`us` , `wa` , `si/so`)؟
3. چه ابزاری سریع‌ترین تشخیص را می‌داد؟
4. اگر این سناریو روی سرور واقعی رخ دهد، **مرحله‌ی مهار** چیست و بعد از آن چه گزارش/اقدامی می‌نویسی؟

۴.۴ تمرین‌های خراب‌کن و درست‌کن

#	خرابی	علامت	تشخیص/رفع
1	<code>kill -9</code> روی یک دیمن	سرویس بالا نمی‌آید/فایل lock می‌ماند	<code>systemctl start</code> و اگر لازم بود حذف lock، سپس بررسی چرایی
2	فرایند <code>sleep</code> را با <code>kill -STOP</code>	در <code>ps</code> حالت <code>T</code> ، CPU صفر	<code>kill -CONT <pid></code>
3	پر کردن پورت ۸۰ با <code>python3 -m http.server 80</code>	nginx استارت نمی‌شود: <code>Address already in use</code>	<code>ss -tlnp grep :80</code> → کشتن دارنده
4	یک زامبی بساز (فرزند بدون <code>wait</code>)	وضعیت <code>Z</code> در <code>ps</code> ، والد خواب	کشتن/ریستارت والد
5	<code>nohup</code> را فراموش کن و SSH را قطع کن	کار نیمه‌کاره می‌ماند، <code>SIGHUP</code> می‌گیرد	<code>systemd-run / tmux / nohup</code>

۵) زمان‌بندی کارها: `cron`، `at`، `systemd timer` (18:45–20:15)

۵.۱ `cron` – ستون پنج‌گانه‌ی زمان

```
* * * * * command
| | | | |
| | | | +---- یکشنبه، نام‌ها (0 و 7 = یکشنبه، روز هفته)
| | | +----- ماه (1-12، نام‌ها)
| | +----- روز ماه (1-31)
| +----- ساعت (0-23)
+----- دقیقه (0-59)
```

نمونه	معنا
<code>0 2 * * *</code>	هر روز ۰۲:۰۰
<code>*/10 * * * *</code>	هر ۱۰ دقیقه
<code>0 */4 * * *</code>	هر ۴ ساعت
<code>30 6 * * 1-5</code>	دوشنبه تا جمعه ۰۶:۳۰
<code>0 0 1 * *</code>	اول هر ماه نیمه‌شب
<code>@reboot</code>	در بوت
<code>@hourly</code> , <code>@monthly</code> , <code>@weekly</code> , <code>@daily</code>	نام‌های جایگزین

```
crontab -l # لیست جاب‌های من
crontab -e # ویرایش
sudo crontab -u www-data -l
ls -l /etc/cron.{d,daily,hourly,weekly,monthly}/ # جاب‌های سیستمی
cat /etc/crontab ; ls /etc/cron.d/
sudo systemctl status cron # Debian/Ubuntu | crond در RHEL
journalctl -u cron --since "1 hour ago" ; grep CRON /var/log/syslog | tail
```

شش علت کلاسیک «کرون کار نمی‌کند» (جواب نمره‌آور در مصاحبه):

1. PATH محدود کرون (`/usr/bin:/bin`) → مسیر مطلق بگذار یا `PATH=` در `crontab` تعریف کن.
2. محیط متفاوت → متغیرها (`LANG` , `SHELL` , `HOME`) را صریح تنظیم کن.
3. خروجی/خطا را جایی نمی‌فرستد → `>> /var/log/job.log 2>&1` یا `MAILTO=`.
4. مجوز اجرای اسکریپت یا شبنگ اشتباه (`+x` , خط `CRLF`).
5. % در `crontab` معنای خاص دارد (مثل `date +%F`) → آن را `\%` کن یا در اسکریپت بگذار.
6. سرویس کرون خاموش/غیرفعال است → `systemctl enable --now cron`.

✓ **الگوی حرفه‌ای:** منطق را در اسکریپت بگذار، در `crontab` فقط یک خط باشد: `/usr/local/bin/check.sh >> /var/log/check.log 2>&1`

۵.۲ at – اجرای یک‌باره

```
sudo systemctl enable --now atd # سرویس باید فعال باشد
echo "/usr/local/bin/backup.sh /etc /var/backups" | at 02:00
at -f /tmp/script.sh 10:30 2026-10-01
at -l ; atq # لیست جاب‌ها
at -c 3 | tail -20 # محتوای جاب شماره ۳
atrm 3 # حذف
at now + 3 hours <<< 'wall "جلسه تا ۳ ساعت دیگر"'
batch < /tmp/script.sh # پایین آمد load وقتی
```

تفاوت cron و at : کرون تکراری است، `at` یک‌باره؛ کرون کمترین دقتش یک دقیقه است، `at` می‌تواند `now + 30 seconds` باشد (تقریبی).

۵.۳ systemd timers – مدرن، لاگ‌دار و دقیق

```
# /etc/systemd/system/backup.service
[Unit]
Description=Backup dirotory /etc
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh /etc /var/backups
Nice=10
IOSchedulingClass=idle
```

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Timer for daily backup
[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true # اگر سرور خاموش بود، بعد از بوت اجرا کن
RandomizedDelaySec=300 # پخش کردن بار بین سرورها
Unit=backup.service
[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers --all ; systemctl status backup.timer
systemd-analyze calendar 'Mon..Fri *-*-* 09:00' # تست عبارت زمان
journalctl -u backup.service -n 30
```

مزیت timer بر cron	توضیح
لاگ کامل	<code>journalctl -u <unit></code>
وابستگی و ترتیب	<code>Requires=</code> , <code>After=</code>
اجرای از قلم افتاده	<code>Persistent=true</code>
کنترل منابع	<code>MemoryMax</code> , <code>IOSchedulingClass</code> , <code>Nice</code>
پخش بار	<code>RandomizedDelaySec</code>

۵.۴ تمرین عملی این بلوک (اجرا کن)

```
# ۱) یک جاب کرون که هر دقیقه ساعت را در فایل بنویسد (تست سریع)
( crontab -l 2>/dev/null; echo "* * * * * date +%F\ \%T >> /tmp/cron-test.log" ) | crontab -
sleep 120 ; tail -3 /tmp/cron-test.log
crontab -l | grep -v cron-test | crontab - # پاکسازی
```

```
# ۲) لاگش را ببین journalctl بساز و با OnCalendar=minutely یک timer
# ۳) یا لاگ ببین /var/spool/at یک پیام بنویس و در `at now + 1 minute` با
```

۶ Self-Test روز ۴ (۱۵ سؤال)

- تفاوت برنامه، فرایند و نخ در یک جمله؟
- `fork` و `exec` هر کدام چه می‌کنند؟
- زامبی چیست و آیا مصرف CPU دارد؟ چطور رفع می‌شود؟
- `load average: 8.0` روی سیستمی با ۴ هسته چه معنایی دارد؟
- در `top` ، `wa` بالا چه می‌گوید و با چه ابزاری عمیق‌تر می‌روی؟
- `buff/cache` چرا همیشه «مصرف بد» نیست؟
- تفاوت `SIGTERM` و `SIGKILL` و ترتیب درست استفاده؟
- `kill -HUP` روی یک دیمن چه می‌کند؟

9. `kill -0 PID` چه کاربرد عملی دارد؟
10. محدوده‌ی `nice` چیست و چه کسی می‌تواند مقدار منفی بدهد؟
11. چرا `ionice` لازم است وقتی `nice` گذاشته‌ای؟
12. تفاوت `bg` و `disown` ؟
13. پنج ستون `cron` را بنویس و `1-5 * * 15 3` را تفسیر کن.
14. چرا اسکریپت در `cron` کار نمی‌کند ولی دستی کار می‌کند؟ سه علت.
15. سه مزیت `systemd timer` بر `cron` ؟

۷) بانک سؤال مصاحبه‌ی روز ۴ (۱۴ سؤال با پاسخ مدل)

- ۱) فرایند در لینوکس چطور ساخته می‌شود؟ با `fork` از فرایند والد (یا `vfork` / `clone`) و سپس `execve` برای بارگذاری برنامه‌ی جدید؛ والد با `wait` کد خروج را جمع می‌کند. تنها راه ساخت فرایند در لینوکس `fork` است، این سؤال را خیلی می‌پرسند.
- ۲) «سرور کند شده است» – پنجره‌ی اول چیست؟ `uptime` (load نسبی به `nproc`)، `top` / `htop` برای CPU و حافظه و `wa`، `vmstat` 2 5 برای فشار حافظه و `swap`، `iostat -xz 2 5` برای دیسک، `ss -s` برای شبکه. سپس فرایند مقصر را با `ps --sort=-` پیدا و با `perf top` / `strace -p` عمیق‌تر می‌کنم.
- ۳) تفاوت `ps aux` و `ps -ef` ؟ هر دو همه‌ی فرایندها را نشان می‌دهند، ولی سبک BSD (`aux`) ستون‌های `%CPU/%MEM/VSZ/RSS` و `COMMAND` دارد و سبک System V (`-ef`) `PPID` و `STIME`؛ انتخاب بر اساس خروجی مورد نیاز و عادت توزیع است.
- ۴) یک فرایند را چطور به‌طور کامل و امن متوقف می‌کنی؟ اول `pgrep/pstree` برای تشخیص درست، سپس `SIGTERM` برای خاتمه‌ی تمیز (بستن فایل/کانکشن)، چند ثانیه صبر و بررسی `kill -0`، بعد در صورت نیاز `SIGKILL`. اگر سرویس `systemd` است از `systemctl stop` استفاده می‌کنم که همین چرخه را با `timeout` مدیریت می‌کند. هرگز اول `-9` نمی‌زنم.
- ۵) زامبی بساز و توضیح بده چطور نابودش می‌کنی. زامبی نتیجه‌ی تمام‌شدن فرزند بدون `wait` والد است؛ منبع مصرفی ندارد ولی ورودی جدول فرایند را می‌گیرد. راه‌حل واقعی: اصلاح والد (`wait/waitpid`، `SIGCHLD` signal handler) یا ری‌استارت والد؛ کشتن زامبی مستقیماً ممکن نیست چون مرده است. `systemd` / `init` زامبی‌های یتیم را جمع می‌کند.
- ۶) `nohup` چه می‌کند و چرا نجات‌دهنده است؟ `SIGHUP` را نادیده می‌گیرد و ورودی/خروجی را از ترمینال جدا می‌کند، تا قطع SSH کار را نکشد: `& nohup cmd > log 2>&1`. برای کارهای طولانی در سرور، `screen` / `tmux` را ترجیح می‌دهم چون نشست تعاملی برمی‌گردد؛ و برای کارهای جدی‌تر، `systemd-run --unit` چون زمان‌بندی/ری‌استارت/لاگ دارد.
- ۷) یک بکاپ سنگین سرور دیتابیس را کند کرده است، چه می‌کنی؟ اجرای بکاپ با `nice -n 19` + `ionice -c3` (یا `-c2 -n7`)، محدودسازی نرخ I/O با `cgclassify` / `ionice` یا `IOWeight` `systemd`، انتقال بکاپ به پنجره‌ی زمانی کم‌بار، استفاده از اسنپ‌شات LVM/دیسک به‌جای خواندن مستقیم، فشرده‌سازی با `--rsyncable` یا `pigz` با تعداد رشته محدود، و پایش `iostat` در طول اجرا.
- ۸) تفاوت `kill`، `killall`، `pkill` ؟ `kill` با PID یا job spec، `pkill` بر اساس الگوی نام/کاربر/خط فرمان (خطر تطابق بیش‌ازحد)، `killall` روی همه‌ی نمونه‌های یک نام دقیق. حرفه‌ای‌ترین راه: `pgrep -a` برای تأیید، سپس `kill` مشخص.
- ۹) چطور می‌فهمی چه چیزی روی پورت ۸۰ است؟

```
ss -tlnp 'sport = :80' # یا ss -tlnp | grep :80
و فرایند PID :80 # lsof -i :80
fuser -v 80/tcp
systemctl status $(ss -tlnp | awk ... ) # در نهایت مالک سرویس را بین
```

اگر `Permission denied` در نمایش PID گرفتی، با `sudo` اجرا کن (اطلاعات فرایندها محدود است).

۱۰ تفاوت **Ctrl+C** و **Ctrl+Z** و **Ctrl+D** ؟ **Ctrl+C** سیگنال **SIGINT** (پایان)، **Ctrl+Z** سیگنال **SIGTSTP** (توقف و بازگشت به پوسته)، **Ctrl+D** پایان ورودی (EOF) که در پوسته‌ی خالی یعنی خروج، تفاوت مهم در مصاحبه: **Z** فرایند را نمی‌کشد، فقط متوقف می‌کند.

۱۱ **cron** یا **systemd timer** کدام را در تولید توصیه می‌کنی؟ ترکیب: **timer** مزیت لاگ متمرکز در **journal**، **Persistent=true**، کنترل منابع و وابستگی‌ها را دارد و ترجیح من در سرورهای **systemd** است؛ **cron** برای جاب‌های بسیار ساده، سازگاری با اسکریپت‌های قدیمی و سیستم‌های بدون **systemd** باقی می‌ماند. مهم این است که جاب **idempotent** و لاگ‌دار باشد و در پایش ثبت شود (مثلاً **exit code** به **Zabbix/Prometheus pushgateway**).

۱۲ **at** کجا به کار می‌آید؟ کارهای یک‌باره‌ی زمان‌دار: نگهداری (maintenance) امشب، ری‌استارت زمان‌دار سرویس، یادآوری/اطلاعیه. **at** **batch**، **atrm**، **-l** (اجرا در کم‌باری). برای تکراری‌ها از **cron/timer** استفاده می‌کنم.

۱۳ **strace** را چه وقت می‌زنی و چطور سریع نتیجه می‌گیری؟ وقتی برنامه‌ای «بی‌دلیل گیر کرده/خطا می‌دهد ولی لاگ ندارد». الگوهای مفید:

```
strace -f -p <pid> -e trace=network -tt # مشکل شبکه/کا نکشن
strace -f -e trace=openat,stat ./app 2>&1 | grep -i "ENOENT|EACCES" # فایل گمشده/مجوز #
strace -c -f ./app # خلاصه‌ی زمان‌نی سیکال‌ها
```

در تولید با احتیاط (overhead) و معمولاً کوتاه‌مدت.

۱۴ **lsof** سه کاربرد مهم؟ دیدن فایل‌های باز یک فرایند (**lsof -p <pid>**)، فایل‌های حذف‌شده‌ای که فضا را نگه داشته‌اند (**lsof +L1**)، و پیدا کردن دارنده‌ی پورت/اتصال شبکه (**lsof -i :443**، **lsof -i @10.20.0.5**). همچنین **lsof +D /mnt** برای دیدن چه کسی مانع **umount** است.

۸ چک‌لیست تکمیل روز ۴

- [] **ps aux**، **ps -ef**، **ps -eo ... --sort** را با تفسیر کامل اجرا کرده‌ام
- [] **top** را با کلیدهای **1 P M c u k r H f** تمرین کرده‌ام
- [] هدر **top** را خط‌به‌خط تفسیر می‌کنم (load, us/sy/wa/st, cache, swap)
- [] با **vmstat**، **iostat**، **iotop**، **free** فشار سیستم را دیدم
- [] همه‌ی سیگنال‌های مهم را با مثال اجرا کردم و تفاوت ۹/۱۵ را دیدم
- [] **ionice** / **renice** / **nice** را روی فرایندهای آزمایشی اجرا کردم
- [] **tmux**، **disown**، **nohup** را برای کار طولانی تست کرده‌ام
- [] آزمایشگاه «سرور کند» را ساختم، تشخیص دادم و تمیز بستم
- [] **crontab** با **PATH** لاگ درست ست کردم و اجرا شد
- [] یک **systemd timer** با **Persistent=true** ساختم و با **journalctl** دیدم
- [] به هر ۱۴ سؤال مصاحبه‌ی این روز بلند جواب دادم

بسته‌ی مطالعه‌ی LPIC-1 – روز ۵

سرویس‌های سیستم و فرایند بوت

systemd • Services & Units • Boot Process • GRUB • journald • logrotate

روز ۵ – سرویس‌های سیستم و فرایند بوت

مسیر مطالعه‌ی این روز (طبق پلن ۳۰ روزه): مورنینگ: (09:00–10:30) Systemd Overview → Manage Services (10:45–12:15) → Boot Process & GRUB (12:30–13:00) → GRUB Configuration (18:45 → (18:30–17:00) `systemctl` Lab: ایونینگ: (20:15 → (21:00–20:30) `journalctl` Logs →

هدف روز: زنجیره‌ی بوت را از BIOS/UEFI تا `systemd` بشناسی، `unit` بنویسی و سرویس را مدیریت کنی، GRUB و پارامترهای کرنل را کنترل کنی، و لاگ‌ها را در `journal` حرفه‌ای جست‌وجو کنی.

0) مرور ۳ دقیقه‌ای روزهای ۱ تا ۴

موضوع	امروز کجا به کار می‌آید
<code>/boot/vmlinuz</code> , <code>initramfs</code> (روز ۱)	امروز جای دقیق هر قطعه در زنجیره‌ی بوت
فایل‌سیستم و <code>fstab</code> (روز ۲)	اگر <code>fstab</code> خراب باشد، بوت در <code>emergency mode</code> می‌ایستد
<code>trap</code> / کد خروج (روز ۳)	<code>unit</code> های <code>Type=oneshot</code> موفقیت را از کد خروج می‌فهمند
فرایند، سیگنال، PID 1 (روز ۴)	<code>systemd</code> همان PID 1 است و با سیگنال سرویس‌ها را می‌چرخاند
<code>kill -HUP</code> برای <code>reload</code> (روز ۴)	<code>systemctl reload</code> و <code>daemon-reload</code> همین مفهوم
<code>cron</code> (روز ۴)	مقایسه با <code>systemd timers</code> و بازیابی بوت

جمله‌ی وصل‌کننده: چهار روز گذشته «چه چیزی در حال اجراست» را مدیریت کردی؛ امروز می‌فهمی آن فرایندها چگونه اینجا رسیدند (زنجیره‌ی بوت) و چگونه باید همیشه درست بالا بیایند (سرویس‌ها و وابستگی‌ها).

پل مستقیم به هفته ۱: زنجیره‌ی بوت لینوکس دقیقاً مثل زنجیره‌ی راه‌اندازی روتر است: POST ↔ Power-on Self Test روتر، بوت‌لودر ↔ BootROM/ROMMON، کرنل ↔ `IOS image`، سرویس‌ها ↔ `startup-config`. اگر مصاحبه‌گر شبکه باشی یا ادمین سیستم، این تطبیق را بلد باش؛ در پاسخ تفاوت‌ها را هم بگو (لینوکس چندسرویسی و ماژولار است، `IOS` یکپارچه).

1) مرور (09:00–10:30) systemd

۱.۱ systemd چیست و چرا جای SysV init را گرفت؟

سیستم‌عامل هر سیستم عادی تا SysV/systemd یکی است.

ویژگی	SysV init (/etc/init.d)	systemd
مدل راه اندازی	اسکرپت های شل شماره دار (S01nginx)	unit های اعلانی (declarative)
موازی سازی	ترتیبی (کند)	موازی با حل وابستگی
وابستگی ها	سطحی (شماره ی runlevel)	صریح (Requires= , Wants= , After=)
فعال سازی درخواستی	ندارد	socket/DBus activation
لاگ	متن ساده	journal ساختاریافته با مهر زمان و اولویت
محدودسازی/cgroup	ندارد	کنترل منابع در هر سرویس
ابزار	update-rc.d , chkconfig , service	systemd-analyze , journalctl , systemctl

مفاهیم پایه: unit (واحد) کوچک ترین جزء مدیریتی؛ انواع مهم: .service , .socket , .timer , .target , .mount , .path , .device , target معادل runlevel است (multi-) . (user.target ≈ runlevel 3 , graphical.target ≈ 5).

```
systemctl list-units --type=target --all
systemctl get-default ; sudo systemctl set-default multi-user.target
```

۱.۲ سه سطح «وضعیت» یک سرویس – سؤال طلایی مصاحبه

پرسش	فرمان	معنا
الان در حافظه اجراست؟	<code>systemctl is-active nginx</code>	active / inactive / failed
در بوت بعدی اجرا می شود؟	<code>systemctl is-enabled nginx</code>	enabled / disabled / static / masked
کل وضعیت	<code>systemctl status nginx</code>	هر دو + PID ها + آخرین لاگ ها
چرا شکست خورد؟	<code>journalctl -u nginx , systemctl --failed</code>	خطای واقعی

```
# ترکیب استاندارد و امن برای فعال سازی + شروع
sudo systemctl enable --now nginx
# مثال ها
systemctl list-units --type=service --state=running
systemctl list-unit-files --state=enabled
systemctl cat nginx # نمایش اصلی unit
systemctl show nginx -p MainPID -p ExecStart -p Restart
systemctl edit nginx # ساخت drop-in در /etc/systemd/system/nginx.service.d/override.conf
systemctl daemon-reload # پس از هر تغییر فایل
systemctl restart|reload|try-restart nginx
systemctl mask nginx # جلوگیری کامل از استارت (حتی به عنوان وابستگی)
```

⚠ **disable در برابر mask** : `disable` پیوندهای بوت را حذف می کند (ولی می توان دستی استارت کرد)، `mask` به `/dev/null` پیوند می زند و جلوی هر راه استارت را می گیرد. برای «اصلاً اجرا نشود» از `mask` استفاده کن.

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application Service
Documentation=man:myapp(8) https://example.com/docs
After=network-online.target
Wants=network-online.target
Requires=mysql.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple # simple | forking | oneshot | notify | dbus | idle
User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
EnvironmentFile=-/etc/myapp/env # (تحميل نبودن فایل)
Environment=APP_MODE=production PORT=8080
ExecStartPre=/usr/bin/test -f /opt/myapp/config.yml
ExecStart=/opt/myapp/bin/myapp --config /opt/myapp/config.yml
ExecReload=/bin/kill -HUP $MAINPID # پشتیبانی از reload
ExecStop=/bin/kill -TERM $MAINPID
Restart=on-failure # no | on-failure | on-abnormal | always
RestartSec=5
TimeoutStopSec=20
KillMode=mixed
# سخت‌گیری امنیتی (پل به هفته ۳)
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/var/lib/myapp /var/log/myapp
# کنترل منابع
LimitNOFILE=65535
MemoryMax=1G
CPUQuota=150%
Nice=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

انواع Type – جدول امتیازآور:

مثال	چه وقت	Type
nginx, myapp	فرایند اصلی با ExecStart شروع و تا آخر زنده است	simple (پیش‌فرض)
سرویس‌های قدیمی	فرایند خودش را fork می‌کند و والد می‌رود	forking
مونت، اسکریپت پیکربندی	یک کار انجام می‌دهد و تمام؛ RemainAfterExit=yes اگر باید «فعال» بماند	oneshot
سرویس‌های مدرن	اپ با sd_notify آماده‌بودن را اعلام می‌کند	notify
کارهای تأخیری	بعد از همه اجرا شود	idle

```
systemctl status sshd|ssh # دسترسی راه دور
systemctl status cron|crond # زمان‌بندی
systemctl status NetworkManager # شبکه (دسکتاپ/سرور مدرن)
systemctl status systemd-networkd, systemd-resolved, systemd-timesyncd
systemctl status firewalld|ufw # فایروال (روز ۶)
systemctl status rsyslog|systemd-journald
systemctl list-dependencies multi-user.target # درخت وابستگی
```

(۲) مدیریت سرویس‌ها در عمل (10:45–12:15)

۲.۱ آناتومی یک عیب‌یابی حرفه‌ای (الگو را حفظ کن)

```
sudo systemctl status myapp -l --no-pager # ۱) وضعیت و آخرین لاگ
journalctl -u myapp -n 50 --no-pager # ۲) لاگ سرویس
journalctl -u myapp --since "10 min ago" -p err
systemctl cat myapp # ۳) محتوای unit
sudo myapp --config /opt/myapp/config.yml # ۴) اجرای دستی: خطا را ببین
sudo -u myapp /opt/myapp/bin/myapp # ۵) تست با همان کاربر سرویس (مجوزها/خانه)
systemctl show myapp -p ExecStart -p User -p MainPID
ss -tlnp | grep 8080 # ۶) پورت/وابستگی
systemctl list-dependencies myapp # ۷) وابستگی‌ها
```

پنج علت رایج شکست سرویس در تولید:

1. مجوز فایل/پوشه (کاربر سرویس به لاگ/داده دسترسی ندارد) → `sudo -u user test -r ... , ls -l`
2. پورت اشغال (Address already in use) → `ss -tlnp`
3. متغیر محیطی غایب → `EnvironmentFile`, تست با `sudo -u user env`
4. وابستگی آماده نشده → `Wants=`, `After=` و `network-online.target`
5. محدودیت SELinux/AppArmor → `SELinux/AppArmor` , `ausearch -m avc -ts recent` , `journalctl -k | grep -i denied`

۲.۲ فرمان‌های فشرده‌ی مدیریت

```
systemctl start|stop|restart|reload|status|restart <svc>
systemctl is-active|is-enabled|is-failed <svc>
systemctl enable|disable|mask|unmask <svc>
systemctl list-units --failed ; sudo systemctl reset-failed
sudo systemctl daemon-reexec # systemd (پس از ارتقای)
systemctl list-unit-files --type=service | grep enabled | wc -l
systemd-analyze # زمان کل بوت
systemd-analyze blame | head -15 # کندترین سرویس‌ها
systemd-analyze critical-chain # زنجیره‌ی بحرانی بوت
systemd-analyze verify /etc/systemd/system/myapp.service # اعتبارسنجی unit
```

۳) فرایند بوت و (12:30–13:00) GRUB

۳.۱ زنجیره‌ی کامل بوت (از حفظ بگو)

```
[1] Firmware: BIOS یا UEFI
BIOS: POST، خواندن MBR (446 دیسک)، زنجیره‌ی بوت
UEFI: POST، خواندن پارتیشن ESP (FAT32) و بوت‌لودر EFI
|
v
[2] Bootloader: GRUB2 (یا systemd-boot)
grub.cfg -> پارامترهای خط فرمان کرنل + initramfs + انتخاب کرنل
|
v
[3] Kernel: vmlinuz بارگذاری، در حافظه initramfs
initramfs = رم‌دیسک کوچک یا درایورهای لازم برای رسیدن به ریشه‌ی واقعی
|
v
[4] Kernel: mount به‌عنوان PID 1 (یا systemd) و اجرای LVM/LUKS (یا ریشه /sbin/init)
|
v
[5] systemd: با حل وابستگی‌ها و موازی‌سازی default.target رسیدن به
|
v
[6] login: getty/display-manager -> ورود کاربر
```

سه جدول همراه این زنجیره:

مرحله	فایل/جزء	اگر خراب شود
Firmware	MBR / ESP	دیسک بوت نمی‌شود
بوت‌لودر	<code>/boot/grub/grub.cfg</code>	منوی GRUB نمی‌آید یا کرنل اشتباه بوت می‌شود
کرنل+initramfs	<code>/boot/vmlinuz-</code> <code>/boot/initrd.img-*,*</code>	<code>Panic: VFS: Unable to mount root fs</code>
ریشه	LVM/LUKS, <code>/etc/fstab</code>	<code>Give root password for Emergency mode (maintenance)</code>
systemd	target و unit	سرویس بالا نمی‌آید ولی سیستم بوت می‌شود
ورود	<code>sshd</code> , <code>getty</code>	ورود کنسول/راه‌دور ممکن نیست

۳.۲ initramfs: چه چیزی داخلش است؟

```
ls -l /boot/initrd.img-$(uname -r)
lsinitramfs /boot/initrd.img-$(uname -r) | head -30 # فهرست محتوا
lsinitramfs /boot/initrd.img-$(uname -r) | grep -E "crypt|raid|lvm|ext4|dm-" # ماژول‌های حیاتی
sudo update-initramfs -u -k $(uname -r) # بازسازی پس از تغییر فایل‌های حیاتی
sudo dracut -f # RHEL/Fedora
```

اگر درایور/ماژول LVM یا LUKS را اضافه یا تغییر داده‌ای (`/etc/crypttab` , `/etc/fstab`)، بازسازی `initramfs` الزامی است وگرنه بوت بعدی به `panic` می‌رسد. این یکی از پرتکرارترین اشتباهات ادمین تازه‌کار است.

۳.۳ GRUB – کنترل واقعی بوت

```
cat /etc/default/grub # (اینجا ویرایش کن)
# نمونه :
# GRUB_TIMEOUT=5
# GRUB_DEFAULT=0 # برای منوی تودرتو / نام
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
# GRUB_CMDLINE_LINUX=""
# GRUB_DISABLE_RECOVERY="false"

sudo update-grub # Debian/Ubuntu: تولید /boot/grub/grub.cfg
sudo grub2-mkconfig -o /boot/grub2/grub.cfg # RHEL: BIOS
sudo grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg # RHEL: UEFI
ls /etc/grub.d/ # 00_header, 10_linux, 40_custom, ...
```

```
# e یا GRUB در منوی) افزودن پارامتر کرنل موقت
# - root حالت تک‌کاربره برای بازیابی رمز -
# - graphical برای رد شدن از GRUB_CMDLINE_LINUX="systemd.unit=multi-user.target" یا -
# - remount rw یا احتیاط و سپس) برای ورود مستقیم init=/bin/bash یا -
```

پارامترهای پرکاربرد کرنل: | پارامتر | اثر | |---|---| | `quiet splash` | کم‌کردن نمایش پیام‌ها (برای دیباگ حذفش کن!) | | `rw` / `ro` | |
`mount`	ریشه فقط خواندنی/خواندنی-نوشتنی		`init=/bin/bash`	اجرای شل به جای `init` (نجات)				
`systemd.unit=rescue.target`	بوت به حالت `rescue`		`single` / `1`	تک‌کاربره		`nomodeset`	حل مشکلات گرافیکی/درایور	
`net.ifnames=0`	نام‌گذاری قدیمی اینترفیس‌ها (`eth0`)		`acpi=off` , `noapic`	کار با سخت‌افزار معیوب				

۳.۴ ترتیب بوت سرویس‌ها، `generator` و `systemd-analyze`

```
systemd-analyze ; systemd-analyze blame ; systemd-analyze critical-chain
systemd-analyze plot > /tmp/boot.svg # (خیلی مفید)
systemctl list-dependencies --reverse sshd
systemctl list-jobs
```

۴.۱ سناریو: یک اپلیکیشن پایتونی ساده را به سرویس تبدیل کن

```

# ۱) برنامه‌ی نمونه
sudo mkdir -p /opt/hello/bin /var/log/hello /var/lib/hello
sudo tee /opt/hello/bin/hello.py >/dev/null <<'PY'
#!/usr/bin/env python3
import http.server, socketserver, os
PORT = int(os.environ.get("PORT", "8080"))
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"hello from systemd\n"
        self.send_response(200); self.send_header("Content-Type","text/plain")
        self.send_header("Content-Length", str(len(body))); self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass
socketserver.TCPServer.allow_reuse_address = True
with socketserver.TCPServer(("0.0.0.0", PORT), H) as httpd:
    httpd.serve_forever()
PY
sudo chmod 755 /opt/hello/bin/hello.py

# ۲) کاربر سرویس بدون شل ورود
sudo useradd --system --home /opt/hello --shell /usr/sbin/nologin helloapp
sudo chown -R helloapp:helloapp /opt/hello /var/log/hello /var/lib/hello

# ۳) فایل unit
sudo tee /etc/systemd/system/hello.service >/dev/null <<'UNIT'
[Unit]
Description=Hello HTTP demo service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=helloapp
Group=helloapp
WorkingDirectory=/opt/hello
Environment=PORT=8080
ExecStart=/usr/bin/python3 /opt/hello/bin/hello.py
Restart=on-failure
RestartSec=3
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=full
ProtectHome=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
UNIT

# ۴) اعتبارسنجی، فعال‌سازی و تست
sudo systemd-analyze verify /etc/systemd/system/hello.service
sudo systemctl daemon-reload
sudo systemctl enable --now hello.service
systemctl status hello.service --no-pager
ss -tlnp | grep :8080
curl -sS http://127.0.0.1:8080 ; echo
journalctl -u hello -n 20 --no-pager

```

```
# بالا می‌آورد systemd فرایند را بکش، بین resist: (الف)
sudo pkill -f hello.py ; sleep 5 ; systemctl is-active hello # active (Restart)
journalctl -u hello -n 10 | tail -5

# reload و drop-in ب تغییر پورت با
sudo systemctl edit hello # در فایل override: [Service] Environment=PORT=9090
sudo systemctl daemon-reload ; sudo systemctl restart hello
curl -sS http://127.0.0.1:9090 ; sudo ss -tlnp | grep 909

# سخت‌گیری منابع و مشاهده (ج)
sudo systemctl set-property hello.service MemoryMax=64M CPUQuota=20%
systemctl show hello -p MemoryMax -p CPUQuota
systemctl status hello | head -12 # در سطح مصرف cgroup

# reload (HUP) پشتیبانی
# اضافه کن unit: ExecReload=/bin/kill -HUP $MAINPID
sudo systemctl reload hello ; systemctl show hello -p ExecReload

# و اجبار به مستندسازی mask (ه)
sudo systemctl mask hello ; sudo systemctl start hello # خطا: masked
sudo systemctl unmask hello ; sudo systemctl start hello

# بوت آزمایشی: سرویس را در بوت فعال کن و بعد ری‌بوت کن
sudo systemctl enable hello ; sudo reboot
# systemctl is-active hello ; systemd-analyze blame | head
```

۴.۳ تمرین‌های خراب‌کن و درست‌کن

#	خرابی	علامت	تشخیص/رفع
1	ExecStart مسیر اشتباه	status: 203/EXEC	journalctl -u : مسیر مطلق درست
2	User=wronguser (وجود ندارد)	Failed to determine user credentials	ساخت کاربر یا اصلاح unit
3	WorkingDirectory ناموجود	200/CHDIR	ساخت پوشه یا حذف کلید
4	پورت اشغال	Address already in use	ss -tlnp ، کشتن دارنده
5	فراموشی daemon-reload	تغییرات اثر ندارد؛ خطای «unit modified»	systemctl daemon-reload
6	سرویس بی‌پایان (crash loop) restart	start-limit-hit	Burst / StartLimitIntervalSec ، رفع باگ
7	ProtectHome=yes + خواندن از /	Permission denied بی‌دلیل	ProtectHome=read-only یا انتقال داده
8	سرویس در بوت اجرا نمی‌شود	بعد از ری‌بوت inactive	systemctl enable → is-enabled

۵) GRUB و بازیابی بوت (18:45–20:15)

۵.۱ تغییر تنظیمات GRUB – گام به گام

```
# ۱) پشتیبان
sudo cp /etc/default/grub /etc/default/grub.bak.$(date +%F)
sudo cp /boot/grub/grub.cfg /boot/grub/grub.cfg.bak.$(date +%F)

# ۲) برای دیدن پیام‌های بوت quiet مثال: نمایش منو ۱۰ ثانیه + حذف تنظیمات
sudo sed -i 's/^GRUB_TIMEOUT=.*GRUB_TIMEOUT=10/' /etc/default/grub
sudo sed -i 's/^GRUB_CMDLINE_LINUX_DEFAULT=.*GRUB_CMDLINE_LINUX_DEFAULT=""/' /etc/default/grub

# ۳) اعمال
sudo update-grub # Debian/Ubuntu
sudo grub-mkconfig -o /boot/grub/grub.cfg
grep -n "menuentry\|linux" /boot/grub/grub.cfg | head
```

افزودن ورودی بوت سفارشی (الگوی **40_custom**):

```
sudo tee /etc/grub.d/40_custom >/dev/null <<'GRUB'
#!/bin/sh
exec tail -n +3 $0
menuentry "Debian (rescue mode)" {
    insmod ext2
    set root=(hd0,gpt2)
    linux /vmlinuz root=/dev/vg0/lv_root ro systemd.unit=rescue.target
    initrd /initrd.img
}
GRUB
sudo chmod +x /etc/grub.d/40_custom ; sudo update-grub
```

۵.۲ نجات سیستم از دست رفتن رمز root (سؤال محبوب مصاحبه)

روش A – از GRUB (**init=/bin/bash**):

```
۱) بزن e بوت entry روی GRUB در منوی
۲) در پایان اضافه کن، linux در خط: rw init=/bin/bash
۳) برای بوت Ctrl+X
۴) در شل بدون محدودیت: passwd root
۵) remount و restart:
mount -o remount,ro /
exec /sbin/reboot -f # یا reboot -f
```

روش B – حالت **rescue/systemd**:

```
۱) در GRUB: systemd.unit=rescue.target (یا single)
۲) passwd موجود، سپس root ورود با رمز
```

روش C – دیسک زنده: بوت از ISO، مانت ریشه و **chroot**:

```
sudo mount /dev/vg0/lv_root /mnt && sudo mount /dev/sda1 /mnt/boot
sudo mount --bind /dev /proc /sys /run /mnt/{dev,proc,sys,run}
sudo chroot /mnt passwd root
```

۵.۳ نجات سیستم از fstab خراب (تمرین واقعی)

```
# (موجود است root رمز) لاگین کن emergency mode در ۱)
mount -o remount,rw / # ریشه معمولا فقط خواندنی است
journalctl -xb | grep -i -A3 "failed to mount"
nano /etc/fstab # UUID/کن اصلاح را اشتباه
findmnt --verify ; mount -a # تست بلافاصله
systemctl daemon-reload ; systemctl reboot
# ۲) یا systemd.unit=emergency.target پارامتر GRUB روش میانبر اضطراری:
# یا nofail اضافه کردن x-systemd.automount در فایل badfstab:
```

۵.۴ بازسازی GRUB و رفع «دستگاه بوت نمی‌شود»

```
# BIOS (MBR)
sudo grub-install /dev/sda ; sudo update-grub
# UEFI
sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=debian
efibootmgr -v # ترتیب ورودی‌های بوت
sudo efibootmgr -o 0001,0002 # تغییر ترتیب
# (chroot) نجات CD از
sudo mount /dev/sda2 /mnt && sudo mount --bind /dev /proc /sys /run /mnt/{dev,proc,sys,run}
sudo chroot /mnt grub-install /dev/sda && sudo chroot /mnt update-grub
```

۶) لاگ‌ها با journalctl و rsyslog (21:00–20:30)

۶.۱ journald – لاگ ساختاریافته

```
journalctl # (pager) همه چیز از ابتدا
journalctl -b # (بوت قبلی: حیاتی برای دیباگ -b -1) فقط این بوت
journalctl -u nginx -f # تعقیب زنده یک سرویس
journalctl -u nginx --since "today" --until "1 hour ago"
journalctl -p err -b # فقط خطا و بالاتر
journalctl -p warning..err # بازه اولویت
journalctl -k -b # (همان dmesg) پیام‌های کرنل این بوت
journalctl _PID=1234 # PID بر اساس
journalctl /usr/sbin/sshd # بر اساس اجرای
journalctl --disk-usage
sudo journalctl --vacuum-size=500M # پاکسازی تا رسیدن به حجم
sudo journalctl --vacuum-time=7d # نگاه داشتن ۷ روز
journalctl -o short-precise|json-pretty|cat
journalctl -u ssh --grep "Failed password" --since "24 hours ago" | tail -20
```

اولویت‌ها (-p): 0 emerg , 1 alert , 2 crit , 3 err , 4 warning , 5 notice , 6 info , 7 debug .

```
# /etc/systemd/journald.conf
[Journal]
Storage=persistent # پیش‌فرض روی دیسک (auto)
SystemMaxUse=1G
SystemKeepFree=500M
MaxRetentionSec=1month
MaxFileSec=1week
ForwardToSyslog=yes
Compress=yes
```

```
sudo mkdir -p /var/log/journal ; sudo systemd-tmpfiles --create --prefix /var/log/journal
sudo systemctl restart systemd-journald
```

۶.۳ **rsyslog** و **/var/log** – لاگ متنی سنتی

فایل	محتوا
(RHEL) /var/log/messages / (Debian) /var/log/syslog	عمومی سیستم
(RHEL) /var/log/secure / (Debian) /var/log/auth.log	احراز هویت، SSH، sudo
/var/log/kern.log	پیام‌های کرنل
/var/log/apt/ , /var/log/dpkg.log	نصب/ارتقای بسته‌ها
error.log , /var/log/nginx/access.log	وب‌سرور
lastlog , wtmp , btmp , /var/log/faillog	ورودها (با last , lastb , lastlog)
/var/log/cron	جاب‌های زمان‌بندی

```
logger -p user.notice "پیام تستی برای rsyslog" # (برای تست)
sudo grep -i "Failed password" /var/log/auth.log | tail
sudo lastb | head ; last | head ; lastlog | head
```

logrotate – جلوگیری از پر شدن دیسک:

```
ls /etc/logrotate.d/ ; cat /etc/logrotate.d/nginx
sudo logrotate -d /etc/logrotate.conf # حالت تست (dry run)
sudo logrotate -f /etc/logrotate.conf # اجرای اجباری
```

```
/var/log/myapp.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 myapp adm
    sharedscripts
    postrotate
        systemctl reload myapp >/dev/null 2>&1 || true
    endscript
}
```

۶.۴ جست‌وجوهای امنیتی روی لاگ (پل به هفته ۳)

```
# های پرتکرار IP و SSH تلاش‌های ناموفق
journalctl -u ssh --since "24 hours ago" --grep "Failed password" | awk '{print $(NF-3)}' | sort |
uniq -c | sort -nr | head
# ورودهای موفق به root
grep "Accepted" /var/log/auth.log | grep "root" | tail
# تغییرات کاربران و sudo
grep -E "useradd|usermod|userdel|sudo:" /var/log/auth.log | tail -20
# تغییرات پکیج
grep -E "install | upgrade " /var/log/dpkg.log | tail -20
```

Self-Test روز ۵ (۱۵ سؤال) (۷)

1. زنجیره‌ی بوت را در ۶ مرحله نام ببر.
2. **UEFI** و **BIOS** در بوت چه تفاوت عملی دارند؟
3. **initramfs** چیست و چه‌وقت باید بازسازی شود؟
4. تفاوت **is-active** و **is-enabled** ؟
5. تفاوت **disable** و **mask** ؟
6. **Type=simple** در برابر **forking** و **oneshot** ؟
7. **After=** و **Requires=** چه تفاوتی دارند؟
8. **WantedBy=multi-user.target** چه معنایی دارد؟
9. **daemon-reload** چه‌وقت لازم است و چرا **restart** جایش را نمی‌گیرد؟
10. در چه شرایطی سرویس **start-limit-hit** می‌دهد؟
11. **GRUB_CMDLINE_LINUX** و **GRUB_CMDLINE_LINUX_DEFAULT** چه تفاوتی دارند؟
12. چرا **update-grub** بعد از ویرایش **/etc/default/grub** لازم است؟ کدام فایل را دستی ویرایش نمی‌کنیم؟
13. اگر **/etc/fstab** خطا داشته باشد، بوت چه می‌شود و چطور نجات می‌دهی؟
14. **journalctl -b -1 -p err** چه چیزی نشان می‌دهد؟
15. **logrotate** چه مشکلی را حل می‌کند و **postrotate** چه می‌کند؟

۸) بانک سؤال مصاحبه‌ی روز ۵ (۱۴ سؤال با پاسخ مدل)

۱) فرایند بوت لینوکس را توضیح بده. فیرمور (UEFI/BIOS) POST و پیدا کردن بوت‌لودر؛ GRUB2 که کرنل و initramfs را از `/boot` بارگذاری می‌کند؛ کرنل با کمک initramfs درایورهای لازم را می‌آورد و ریشه را مانت می‌کند؛ سپس `systemd` به‌عنوان PID 1 `default.target` را با حل وابستگی‌ها می‌سازد و در نهایت `getty` / `display-manager` ورود را ممکن می‌کند. اگر جایی بشکند، بسته به مرحله: panic (کرنل/ریشه)، emergency mode (fstab)، یا فقط سرویس نیامده (systemd).

۲) `systemd` چه مزیتی بر `SysV init` دارد؟ راه‌اندازی موازی، مدیریت وابستگی صریح، فعال‌سازی بر اساس cgroup، socket/DBus و کنترل منابع، لاگ ساختاریافته با `journald`، و ابزارهای تحلیلی (`systemd-analyze`). نقطه‌ضعفش: پیچیدگی دامنه و «همه‌کارگی» آن که در بحث‌های طراحی محل نقد است.

۳) یک `unit` برای سرویس بنویس. (قالب بخش ۱.۳ را با `Type`، `User`، `ExecStart`، `Restart`، `WantedBy`، و سخت‌گیری‌های امنیتی مثل `NoNewPrivileges`، `ProtectSystem` بنویس). توضیح می‌دهم که `ExecReload` برای `HUP`، `EnvironmentFile` برای رمزها، و `MemoryMax` / `CPUQuota` برای کنترل منابع است.

۴) سرور بوت نمی‌شود و `Give root password for maintenance` می‌بیند. تشخیص و رفع؟ معمولاً خطای mount در `fstab` (UUID عوض‌شده، دیسک جداشده، فایل‌سیستم خراب). ورود در emergency، `mount -o remount,rw /`، خواندن `journalctl`، `xb`، اصلاح `fstab` (UUID/ `nofail` درست) یا `fsck` دیسک، تست با `mount -a` و `findmnt --verify`، سپس `reboot`. برای جلوگیری: `nofail` برای رسانه‌های جانبی و تست `fstab` قبل از ری‌بوت.

۵) `systemctl status` نشان می‌دهد سرویس `failed` است. مراحل؟ خواندن خروجی `status` و کد خروج/سیگنال، `journalctl -u` `systemctl cat`، `svcs`، `systemctl cat` برای `ExecStart` و `User`، اجرای دستی با همان کاربر، بررسی مجوز فایل‌ها (`sudo -u user test`، `-r`)، پورت‌های اشغال، متغیرهای محیطی، SELinux/AppArmor. سپس رفع، `daemon-reload`، `reset-failed`، `restart` و افزودن پایش/هشدار برای جلوگیری از تکرار.

۶) چطور سرویس را در بوت غیرفعال کنی ولی در همین نشست اجرا بماند؟ `sudo systemctl disable nginx` (و در صورت نیاز `stop`). برای جلوگیری کامل از هر استارت: `mask`. برای «فقط یک بار اجرا کن»: `systemd-run --unit` یا اسکریپت با `nohup`.

۷) تفاوت `reload` و `restart`؟ `reload` با ارسال سیگنال (معمولاً `HUP`) کانفیگ را بازخوانی می‌کند بدون افت سرویس – نیازمند `ExecReload` و پشتیبانی اپ؛ `restart` فرایند را می‌کشد و بالا می‌آورد (کوتاه قطعی دارد). در تولید برای تغییر کانفیگ‌های مجاز، `reload` ترجیح دارد؛ برای تغییر کد/باینری، `restart`.

۸) «چرا سرویس من در بوت اجرا نمی‌شود ولی دستی اجرا می‌شود؟» معمولاً `enabled` نیست؛ یا در `multi-user.target` قرار نگرفته (`WantedBy` غایب و بدون `enable`)، یا وابستگی زمان‌بندی دارد که در بوت آماده نیست (`After=network-online.target` لازم است)، یا شرط‌ها (`ConditionUser`، `ConditionPathExists`) در محیط بوت برقرار نیستند. با `systemctl list-dependencies` و `journalctl -b` تشخیص داده می‌شود.

۹) `journalctl` را چطور برای بازه‌ی خاص و سرویس خاص فیلتر می‌کنی؟ `journalctl -u nginx --since "2026-09-20 08:00"`، برای بوت قبلی `-b -1`، برای رویداد کرنل `-k`، برای فایل اجرایی مسیر کامل، و برای پیدا کردن الگو `--grep`.

۱۰) لاگ‌ها کجا ذخیره می‌شوند و چطور از پر شدن دیسک جلوگیری می‌کنی؟ `journald` در `/var/log/journal` (persistent) یا `/run/`، `log/journal` (volatile) با سهمیه `SystemMaxUse`؛ و `rsyslog` در `/var/log/*.log`. جلوگیری: کانفیگ سهمیه‌ی `journald`، `logrotate` برای اپ‌ها، پایش فضا/inode، ارسال به سرور لاگ مرکزی (Rsyslog/Syslog-ng/ELK/Graylog) و `error` سطح هشدار برای `/var`.

۱۱) `GRUB` را در `UEFI` چطور بازنصب می‌کنی؟ با دیسک زنده/UEFI shell: مانت `ESP` روی `/boot/efi`، ریشه، `chroot`، سپس `grub-install`، `efibootmgr -v` و ترتیب بوت با `efibootmgr -o`. اگر `NVRAM` پاک شده باشد، ساخت مجدد `entry` ضروری است.

۱۲) تفاوت **systemd** و **init** در کنترل سیگنال سرویس‌ها؟ **systemd** هر سرویس را در **cgroup** خودش قرار می‌دهد و در **stop** با **KillMode** تمام فرایندهای **cgroup** را سیگنال می‌دهد؛ به‌همین دلیل سرویس‌هایی که فرایندهای فرزند رهاشده می‌سازند هم کاملاً پاک می‌شوند. **Timeout** خاتمه (**TimeoutStopSec**) و ارتقاء به **SIGKILL** را هم خودش مدیریت می‌کند.

۱۳) **unit** را چطور بدون ویرایش فایل اصلی تغییر می‌دهی؟ با **drop-in**: **systemctl edit myapp** که **/etc/systemd/system/** **myapp.service.d/override.conf** می‌سازد و بین بروزرسانی‌های بسته امن است؛ سپس **daemon-reload** و **restart**. این بهترین‌شیوه (best practice) است و در مصاحبه امتیاز دارد.

۱۴) اگر بوت کند است، چطور پیدا می‌کنی چه چیزی معطل می‌کند؟ **systemd-analyze** (زمان کل)، **systemd-analyze blame** (کندترین واحدها)، **systemd-analyze critical-chain** (مسیر بحرانی)، **systemd-analyze plot > boot.svg** برای نمودار. رفع رایج: غیرفعال کردن سرویس‌های بی‌استفاده، **network-online.target** های بی‌مورد، **systemd-random-seed / random**، **GRUB_TIMEOUT**، و مشکل واقعی مثل انتظار روی **NFS**/دیسک معیوب.

۹) چک‌لیست تکمیل روز ۵

- [] زنجیره‌ی بوت را از **UEFI** تا **login** بی‌نگاه توضیح می‌دهم
- [] **initramfs** را فهرست کردم و می‌دانم چه‌وقت بازسازی لازم است
- [] سرویس **hello.service** را ساختم، فعال کردم و با **Restart** تست کشتن کردم
- [] با **systemctl edit** یک **drop-in** ساخته و اعمال کرده‌ام
- [] **CPUQuota / MemoryMax** را روی سرویس ست و مشاهده کرده‌ام
- [] تفاوت **enable --now / mask / disable** را تست کرده‌ام
- [] **/etc/default/grub** را تغییر دادم، **update-grub** زدم و منو را دیدم
- [] یک ورودی **40_custom** ساختم
- [] سناریوی پاک کردن رمز **root** را (روی **VM** آزمایشی) اجرا کرده‌ام
- [] سناریوی **fstab** خراب را شبیه‌سازی و نجات داده‌ام
- [] **journalctl** با **-b**، **-u**، **-p**، **--since**، **--vacuum-size** تمرین کرده‌ام
- [] یک **logrotate** سفارشی برای اپ خودم نوشته‌ام
- [] به هر ۱۴ سؤال مصاحبه‌ی این روز بلند جواب داده‌ام

بسته‌ی مطالعه‌ی LPIC-1 – روز ۶

مبانی شبکه در لینوکس

ip/ss • Diagnostics • SSH & SCP • Firewall (ufw/firewalld/nftables) • NAT

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۰۲ • روز ۶ (یکشنبه) – مورنینگ ۹:۰۰–۱۳:۰۰ • ایوینینگ ۱۷:۰۰–۲۱:۰۰

روز ۶ – مبانی شبکه در لینوکس

مسیر مطالعه‌ی این روز (طبق پلن ۳۰ روزه): مورنینگ: IP و Hostname (09:00–10:30) → ابزارهای شبکه: `ss` / `netstat` , `ping` (10:45–12:15) → SSH و SCP (12:30–13:00) ایوینینگ: Lab: تنظیمات شبکه (17:00–18:30) → SSH Key Login (18:45 → 20:15) → مبانی فایروال (20:30–21:00)

هدف روز: آدرس‌دهی و مسیریابی را از خط فرمان لینوکس ببینی/تنظیم کنی (`ip` , `nmcli` , `resolvectl`)، ابزارهای عیب‌یابی را حرفه‌ای به کار ببری، SSH را سخت و کلیدمحور کنی و فایروال میزبان را با `ufw` / `firewalld` / `nftables` ببندی. این روز، محل تلاقی هفته ۱ (شبکه) و هفته ۲ (لینوکس) است.

0) مرور ۳ دقیقه‌ای روزهای ۱ تا ۵ + پل به هفته ۱

موضوع	امروز کجا به کار می‌آید
<code>mount</code> , <code>/etc/fstab</code> (روز ۲)	<code>/etc/network/interfaces</code> , <code>netplan</code> و پایداری کانفیگ شبکه
اسکرپت <code>health.sh</code> (روز ۳)	امروز <code>ss</code> / <code>ip</code> را به آن اضافه می‌کنیم
<code>systemd</code> و <code>systemd</code> ها (روز ۵)	<code>firewalld</code> , <code>systemd-networkd</code> , <code>NetworkManager</code> سرویس‌های <code>systemd</code> هستند
<code>journalctl</code> (روز ۵)	لاگ شبکه: <code>journalctl -u NetworkManager -b</code>
هفته ۱: Subnetting (روز ۳)	آدرس/ماسک IP و محاسبه‌ی شبکه در <code>ip addr</code>
هفته ۱: Routing/Static (روز ۴)	<code>ip route</code> , <code>longest prefix</code> , <code>default gateway</code>
هفته ۱: OSPF (روز ۵)	نصب FRR/quagga روی لینوکس – مسیر دینامیک روی سرور
هفته ۱: DHCP/NAT/ACL (روز ۶)	<code>dnsmasq</code> (DHCP) , <code>nftables</code> (NAT) , <code>ufw</code> / <code>nft</code> (ACL)

جمله‌ی وصل‌کننده (پاسخ طلایی در مصاحبه):

«همان کارهایی که در IOS با `interface` , `ip address` , `ip route` , `access-list` می‌کردم، در لینوکس با `ip addr add` , `ip route add` , `nft add rule` انجام می‌شود؛ مفاهیم یکی است، لایه‌ی پیاده‌سازی متفاوت.»

۱) آدرس‌دهی، هاست‌نیم و (09:00–10:30) DNS

۱.۱ دیدن و تنظیم آدرس با ip

```
ip a # تمامی اینترفیس‌ها و آدرس‌ها
ip -br a # خلاصه (brief): UP/DOWN + آدرس
ip -br link # وضعیت لینک‌ها
ip addr show dev ens33
ip -4 a ; ip -6 a
ip link set ens33 up
ip addr add 10.20.0.10/24 dev ens33
ip addr del 10.20.0.10/24 dev ens33
ip link show ens33 | head -2 # MAC وضعیت
cat /sys/class/net/ens33/address # MAC (منبع دوم)
ethtool ens33 | head -20 # درایور/duplex/سرعت
ethtool -S ens33 | head -20 # آماره‌های درایور (دورریز/CRC خطای)
```

⚠ موقتی در برابر دائمی: `ip addr add` تا ریboot دوام می‌آورد. برای پایداری از `nmcli` , `netplan` , `systemd-networkd` یا `/etc/network/interfaces` استفاده کن (بخش ۵).

۱.۲ مسیریابی با ip route

```
ip route # جدول مسیر IPv4
ip -6 route
ip route get 8.8.8.8 # مسیر واقعی که انتخاب می‌شود + آدرس مبدأ
ip route show default
ip route add 10.99.0.0/24 via 10.20.0.1 dev ens33
ip route del 10.99.0.0/24
sudo ip route add default via 10.20.0.1
ip neigh # همسایه/ARP جدول (ARP cache)
ip neigh flush all # پاک‌کردن کش ARP
arp -a # روش قدیمی (net-tools)
```

نگاشت مستقیم به هفته ۱:

لینوکس	IOS
<code>ip -br a</code>	<code>show ip interface brief</code>
<code>ip route</code>	<code>show ip route</code>
<code>ip route add 10.99.0.0/24 via 10.20.0.1</code>	<code>ip route 10.99.0.0 255.255.255.0 10.20.0.1</code>
<code>ip route add default via <gw></code>	<code>ip route 0.0.0.0 0.0.0.0 <gw></code>
<code>ip neigh</code>	<code>show arp</code>
<code>ip neigh flush all</code>	<code>clear arp-cache</code>

```
hostnamectl # اطلاعات + نام سیستم
sudo hostnamectl set-hostname srv2.lab.local
cat /etc/hostname
sudo tee -a /etc/hosts >/dev/null <<'H'
10.20.0.10 srv2.lab.local srv2
10.20.0.11 srv1.lab.local srv1
H
getent hosts srv1 ; getent passwd root # getent به تمامی منابع نام (NSS)
نگاه می‌کند

resolvectl status # systemd-resolved: سرورهای DNS دامنه‌ها و
resolvectl query example.com
cat /etc/resolv.conf # است symlink یک systemd-resolved در سیستم‌های
ls -l /etc/resolv.conf
nmcli dev show | grep -i dns
```

```
# ابزارهای DNS (بسته‌ی dnstools/bind-utils)
dig example.com A +short
dig @1.1.1.1 example.com MX
dig +trace example.com
dig -x 8.8.8.8 +short # PTR / معکوس
nslookup example.com
host example.com
getent ahostsv4 example.com
```

ترتیب حل نام (NSS): ابتدا `/etc/hosts` (طبق `/etc/nsswitch.conf` : `hosts: files dns`)، سپس DNS. برای همین در تست‌ها می‌توان بدون DNS با `/etc/hosts` کار کرد. سؤالی که در مصاحبه بلافاصله می‌آید: «چرا `ping google.com` کار نمی‌کند ولی `ping 8.8.8.8` کار می‌کند؟» → مشکل DNS است نه شبکه.

۱.۴ پورت‌ها، سوکت و سرویس‌ها

```
cat /etc/services | grep -E "(ssh|http|https|dns|smtp|ntp|ldap|mysql|postgres)\b"
ss -tuln # گوش‌دهنده، بدون حل نام، TCP+UDP
ss -tlnp # (برای همه root نیازمند) نام برنامه/PID با
ss -tan state established
ss -s # خلاصه‌ی سوکت‌ها
ss -tp dport = :443
netstat -tulnp # (در سیستم مدرن نصب نیست، net-tools) روش قدیمی
lsof -i :22 # چه فرایندی پورت ۲۲ را نگه داشته
nc -zv 10.20.0.10 22 # تست باز بودن پورت
```

پورت مهم	سرویس
22	SSH
25/587/465	SMTP/Submission
(udp/tcp) 53	DNS
udp 67/68	DHCP
80/443	HTTP/HTTPS
udp 123	NTP
389/636	LDAP/LDAPS
3306/5432	MySQL/PostgreSQL
udp 161/162	SNMP (پل به هفته ۱ – <code>snmp-server community</code>)

۲) ابزارهای عیب‌یابی شبکه (10:45–12:15)

۲.۱ لایه به لایه – ترتیب استاندارد عیب‌یابی

#	لایه	ابزار	نمونه پرسش
1	لینک	<code>ethtool</code> , <code>ip -br link</code>	کارت شبکه UP است؟ کابل/سرعت؟
2	آدرس	<code>ip -br a</code>	IP و ماسک درست؟
3	همسایه	<code>arping</code> , <code>ip neigh</code>	MAC گیت‌وی resolve می‌شود؟
4	مسیر	<code>traceroute</code> , <code>ip route get</code>	مسیر و گیت‌وی درست؟
5	نام	<code>getent</code> , <code>dig</code>	DNS جواب می‌دهد؟
6	پورت/سرویس	<code>curl</code> , <code>nc</code> , <code>ss</code>	سرویس گوش می‌دهد؟ فایروال می‌بندد؟
7	کاربرد	<code>tcpdump</code> , <code>curl -v</code>	لایه‌ی اپلیکیشن چه می‌گوید؟

```

ping -c4 8.8.8.8
ping -c4 -W2 -I ens33 10.20.0.1 # باید به اینترفیس مبدأ
ping6 -c2 2001:4860:4860::8888
ping -M do -s 1472 10.20.0.1 # تست MTU (1472+28=1500)
traceroute -n 8.8.8.8 # مسیر (پیش‌فرض UDP)
traceroute -T -p 443 8.8.8.8 # TCP/443 (عبور از فایروال‌ها)
mtr -rwzc 10 8.8.8.8 # ping+traceroute (بهترین ابزار)
arping -c3 -I ens33 10.20.0.1 # تست ARP
curl -sS -o /dev/null -w "%{http_code} %{time_total}s %{remote_ip}\n" https://example.com
curl -v --resolve srv.lab.local:443:10.20.0.10 https://srv.lab.local/ # تست بدون DNS
wget -S --spider http://10.20.0.10/ # فقط هدرها
nc -zv 10.20.0.11 1-1024 2>&1 | grep -i succeeded # آزمون پورت (آزمایشگاه خودمان!)

```

```
sudo tcpdump -i any -nn -c 20 # ۲۰ بسته روی همه ایترنرفیسها
sudo tcpdump -i ens33 -nn port 53 # DNS
sudo tcpdump -i any -nn host 10.20.0.11
sudo tcpdump -i any -nn 'tcp and port 22 and not src host 10.20.0.1'
sudo tcpdump -i any -nn -A 'port 80' # نمایش ASCII (هدرهای HTTP)
sudo tcpdump -i any -nn -w /tmp/cap.pcap port 443 # ذخیره برای Wireshark
sudo tcpdump -r /tmp/cap.pcap -nn | head -20 # خواندن فایل # 20
sudo tcpdump -i any -nn 'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0' # فقط SYNها
```

سناریوی واقعی: «وبسرور کند است» → tcpdump روی سرور نشان می‌دهد retransmissionهای زیاد (RST/duplicate ACK) هست یا MTU/فایروال بسته می‌گیرد. (پل به هفته ۳: همان مهارت در تحلیل ترافیک حمله استفاده می‌شود).

۲.۳ فشار و کیفیت شبکه

```
iftop -i ens33 # ترافیک زنده به تفکیک اتصال
nload ens33 ; bmon # پهنای‌بند
dstat -tnf 2 # دیسک + CPU + شبکه
speedtest-cli # تست سرعت اینترنت
iperf3 -s (سرور) | iperf3 -c 10.20.0.10 -t 10 -P 4 (کلاینت) # اندازه‌گیری واقعی پهنای‌بند
nmap -sn 10.20.0.0/24 # کشف میزبان‌ها (هفته ۳ عمیق‌تر)
ss -s ; cat /proc/net/dev # آمار ایترنرفیس
```

۳) SSH و SCP (12:30–13:00) و ادامه‌ی عصر

۳.۱ اتصال و کلیدها – جریان کار حرفه‌ای

```
# روی کلاینت
ssh-keygen -t ed25519 -a 100 -C "ali@srv1-2026" -f ~/.ssh/id_ed25519_srv1
ssh-copy-id -i ~/.ssh/id_ed25519_srv1.pub ali@10.20.0.11
ssh -i ~/.ssh/id_ed25519_srv1 ali@10.20.0.11
# میان‌بر و تنظیمات پیش‌فرض -> ~/.ssh/config فایل
```

```
# ~/.ssh/config
Host srv1
  HostName 10.20.0.11
  User ali
  Port 2222
  IdentityFile ~/.ssh/id_ed25519_srv1
  ServerAliveInterval 30
  ForwardAgent no
```

```
ssh srv1 # فقط این config با
ssh -J bastion srv1 # عبور از جامپهاست (ProxyJump)
ssh -L 8080:127.0.0.1:80 srv1 # تونل محلی: پورت ۸۰ سرور روی ۸۰۸۰ من
ssh -R 9000:127.0.0.1:3000 srv1 # تونل معکوس (دسترسی سرور به سرویس محلی من)
ssh -D 1080 srv1 # پروکسی SOCKS5
ssh -N -f -L 5432:db.internal:5432 srv1 # (پسزمینه -f با) تونل دائمی بدون شل
```

۳.۲ کپی و انتقال فایل

```
scp file.txt ali@10.20.0.11:/tmp/
scp -r ./site ali@10.20.0.11:/var/www/
scp -P 2222 ali@10.20.0.11:/var/log/syslog ./
rsync -avz --progress ./site/ ali@10.20.0.11:/var/www/site/
rsync -avz --delete --exclude='*.log' ./data/ ali@10.20.0.11:/data/
rsync -e "ssh -p 2222" -avz ./f/ ali@10.20.0.11:/f/ # با پورت غیراستاندارد
sftp ali@10.20.0.11 # انتقال تعاملی
```

scp یا **rsync**؟ **rsync** فقط تفاوت‌ها را منتقل می‌کند، از **--delete**، **--exclude**، **--link-dest** و ادامه‌ی انتقال نیمه‌کاره پشتیبانی می‌کند و برای بکاپ/همگام‌سازی بهتر است؛ **scp** برای یک فایل کوچک ساده‌تر.

۳.۳ سخت‌سازی SSH – چک‌لیست امنیتی (بسیار مهم مصاحبه)

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%F)
sudo sshd -t # تست کانفیگ قبل از ریاستارت (بدون این کار قفل میشوی!)
sudo systemctl reload sshd
```

```
# /etc/ssh/sshd_config – تنظیمات کلیدی
Port 2222
AddressFamily inet
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers ali deploy@10.20.0.0/24
DenyUsers test
X11Forwarding no
AllowTcpForwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
Protocol 2
```

```
# brute force ابزارهای مقابله با
sudo apt install fail2ban # یا dnf install fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo grep -i "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head
```

پاسخ استاندارد مصاحبه «SSH را چطور امن می‌کنی؟»: کلیدمحرور و غیرفعال کردن رمز، غیرفعال کردن ورود root، محدود کردن کاربران/آدرس‌ها (AllowUsers, Match)، تغییر پورت (کاهش نویز نه امنیت واقعی)، fail2ban، به‌روزرسانی مستمر، لاگ‌گیری و پایش، جداسازی دسترسی مدیریتی در bastion و استفاده از ProxyJump، محدود کردن forwarding، و در صورت نیاز 2FA (TOTP).

۴) آزمایشگاه ۱ – کانفیگ شبکه، sshd و فایروال (17:00–18:30)

۴.۱ توپولوژی آزمایشگاه

دو VM لینوکس در همان شبکه‌ی host-only/NAT:

میزبان	آدرس	نقش
srv1	10.20.0.10/24	سرور (وب/SSH)، فایروال فعال
srv2	10.20.0.11/24	کلاینت/تست
گیت‌وی NAT	10.20.0.1	خروج به اینترنت (VirtualBox NAT/Host-only adapter)

۴.۲ گام ۱ – آدرس‌دهی و پایداری (نمونه‌ها برای سه روش)

```
# الف) Debian/Ubuntu با netplan (فایل: /etc/netplan/01-lab.yaml)
sudo tee /etc/netplan/01-lab.yaml >/dev/null <<'YAML'
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [10.20.0.10/24]
      routes:
        - to: default
          via: 10.20.0.1
      nameservers:
        addresses: [1.1.1.1, 9.9.9.9]
        search: [lab.local]
YAML
sudo netplan try # با rollback (عجب ابزاری!)
sudo netplan apply ; ip -br a ; ip route

# ب) Rocky/RHEL با nmcli
sudo nmcli con mod "System ens33" ipv4.method manual ipv4.addresses 10.20.0.10/24 \
  ipv4.gateway 10.20.0.1 ipv4.dns "1.1.1.1 9.9.9.9" connection.autoconnect yes
sudo nmcli con up "System ens33" ; nmcli dev show ens33 | head -20

# ج) Debian کلاسیک: /etc/network/interfaces
# auto ens33
# iface ens33 inet static
# address 10.20.0.10/24
# gateway 10.20.0.1
# dns-nameservers 1.1.1.1
sudo systemctl restart networking # با ifup ens33
```

```
# روی srv2
ping -c2 10.20.0.10 ; ip neigh | grep 10.20.0.10
ss -tn state established
nc -zv 10.20.0.10 22
sudo tcpdump -i any -nn host 10.20.0.10 -c 10 # ping روی هم‌زمان با
tracpath 10.20.0.10 ; mtr -rwzc3 10.20.0.10
dig @10.20.0.10 lab.local
```

۴.۴ گام ۳ – سخت‌سازی SSH و ورود کلیدمحور

```
# روی srv2 (کلاینت)
ssh-keygen -t ed25519 -C "srv2-to-srv1" -f ~/.ssh/id_srv1
ssh-copy-id -i ~/.ssh/id_srv1.pub ali@10.20.0.10

# روی srv1 (سرور)
sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sshd -t # reload حتماً قبل از
sudo systemctl reload ssh # یا sshd در RHEL
# تست: از یک نشست دیگر (نه همان نشست!) وارد شو
ssh -i ~/.ssh/id_srv1 ali@10.20.0.10 'hostname; id'
```

قاعده‌ی طلایی ادمین‌ها: همیشه یک نشست SSH دیگر باز نگه دار تا وقتی مطمئن شوی کانفیگ جدید درست است. `systemctl reload` یک کانفیگ خراب، می‌تواند تو را از سرور بیرون بیندازد.

۴.۵ گام ۴ – فایروال میزبان (سه پیاده‌سازی، یک مفهوم)

الف) `ufw` (Debian/Ubuntu – ساده و خوانا):

```
sudo ufw default deny incoming ; sudo ufw default allow outgoing
sudo ufw allow from 10.20.0.0/24 to any port 22 proto tcp
sudo ufw allow 80/tcp ; sudo ufw allow 443/tcp
sudo ufw limit 22/tcp # محدودسازی نرخ (ضد brute force)
sudo ufw enable ; sudo ufw status verbose ; sudo ufw status numbered
sudo ufw delete 3 # حذف قاعده با شماره 3
```

ب) `firewalld` (RHEL/Rocky – zone-based):

```
sudo systemctl enable --now firewalld
sudo firewall-cmd --get-active-zones ; sudo firewall-cmd --list-all
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="10.20.0.0/24"
service name="ssh" accept'
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload ; sudo firewall-cmd --list-all
```

ج) `nftables` (مدرن، پشت `iptables-nft` هم هست):

```

sudo nft list ruleset
sudo nft add table inet filter
sudo nft add chain inet filter input '{ type filter hook input priority 0; policy drop; }'
sudo nft add rule inet filter input ct state established,related accept
sudo nft add rule inet filter input iif lo accept
sudo nft add rule inet filter input ip saddr 10.20.0.0/24 tcp dport 22 accept
sudo nft add rule inet filter input tcp dport { 80, 443 } accept
sudo nft add rule inet filter input icmp type echo-request accept
sudo nft add rule inet filter input counter log prefix "DROP-IN: " drop
sudo nft list ruleset | head -30

```

نگاشت هفته ۱ به فایروال میزبان (برای مصاحبه، این جدول طلاست):

nftables	IOS
<code>ip saddr 10.20.0.0/24 accept</code>	<code>access-list 10 permit 10.20.0.0 0.0.0.255</code>
<code>tcp dport 443 accept</code>	<code>access-list 101 permit tcp any host X eq 443</code>
<code>hook input priority 0</code>	<code>ip access-group 101 in</code>
<code>policy drop</code>	<code>implicit deny</code>
<code>table ip nat { chain postrouting { oifname "eth0" masquerade } }</code>	<code>ip nat inside source ... overload</code>
<code>nft list ruleset</code>	<code>show access-lists</code>

NAT/Masquerade روی لینوکس (پل مستقیم به روز ۶ هفته ۱):

```

sudo sysctl -w net.ipv4.ip_forward=1
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-ipforward.conf
sudo nft add table ip nat
sudo nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; }'
sudo nft add rule ip nat postrouting oifname "ens33" masquerade

```

این دقیقاً همان `ip nat inside source list 1 interface g0/0 overload` است، ولی روی لینوکس.

۴.۶ گام ۵ – DHCP روی لینوکس (تمرین اختیاری ولی ارزشمند)

```

sudo apt install dnsmasq
sudo tee /etc/dnsmasq.d/lab.conf >/dev/null <<'CONF'
interface=ens34
dhcp-range=10.30.0.100,10.30.0.200,12h
dhcp-option=3,10.30.0.1
dhcp-option=6,1.1.1.1
dhcp-authoritative
CONF
sudo systemctl restart dnsmasq
sudo journalctl -u dnsmasq -n 20
# روی کلاینت دوم: sudo dhclient -v ens33 یا sudo nmcli con up ...
cat /var/lib/misc/dnsmasq.leases

```

۴.۷ تمرین‌های خراب‌کن و درست‌کن

#	خرابی	علامت	تشخیص/رفع
1	IP اشتباه subnet	ping: connect: Network is unreachable	ip -br a , اصلاح آدرس/ماسک
2	گیت‌وی غایب	به LAN می‌رسد، به اینترنت نه	default → ip route get 8.8.8.8 via
3	DNS اشتباه	ping 8.8.8.8 اوکی، ping google.com نه	/etc/ , resolvectl query resolv.conf
4	فایروال پورت ۸۰ را بسته	hang curl , timeout nc -zv	ufw status , nft list ruleset افزودن قاعده
5	sshd_config خراب + reload	ورود ناممکن شد	از نشست باز دیگر sshd -t و اصلاح؛ در نهایت کنسول
6	PermitRootLogin no + کاربر بدون sudo	ناتوانی در مدیریت	ترمیم از کنسول/بازیابی
7	تغییر پورت SSH بدون باز کردن فایروال	قطع دسترسی بعد از reload	کنسول: افزودن قاعده‌ی پورت جدید
8	MTU نادرست در تونل	اتصال برقرار ولی انتقال بزرگ hang	ip link set , ping -M do -s 1472 mtu 1400

۵) جمع‌بندی محتوای فنی روز ۶ (18:45–20:15 و 20:30–21:00)

۵.۱ مسیرهای کانفیگ پایدار شبکه در توزیع‌های مختلف

توزیع	ابزار/مسیر
+Debian 12+/Ubuntu 18	netplan (/etc/netplan/*.yaml) → networkd/NetworkManager
Debian قدیمی	ifupdown + /etc/network/interfaces
RHEL/Rocky/Fedora	NetworkManager + nmcli / nmtui , فایل در /etc/NetworkManager/system-connections/
systemd-networkd	/etc/systemd/network/*.network
DNS مدرن	systemd-resolved + resolvectl یا dnsmasq / unbound محلی

```
# افزودنی به اسکریپت روز ۳
echo "--- شبکه: اینترفیسها و آدرسها ---" ; ip -br a
echo "--- مسیر پیشفرض ---" ; ip route show default
echo "--- DNS ---" ; resolvectl status | grep -A2 "DNS Servers" | head -6
echo "--- پورت‌های گوش‌دهنده (غیرلوکال) ---"
ss -tulnp | awk 'NR==1 || $5 !~ /^(127\.|\.|::1)/'
echo "--- خطاهای NetworkManager ---"
journalctl -u NetworkManager -b -p warning --no-pager | tail -5
```

۵.۳ sysctl – تنظیمات کرنل مرتبط با شبکه (سطح ادمین)

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl net.ipv4.tcp_syncookies=1 # دفاع SYN flood
sudo sysctl net.ipv4.conf.all.rp_filter=1 # anti-spoofing
sudo sysctl -w net.ipv4.tcp_max_syn_backlog=4096
sysctl -a | grep -E "ipv4.tcp_(fin_timeout|keepalive_time)"
sudo tee /etc/sysctl.d/99-network-hardening.conf >/dev/null <<'SYS'
net.ipv4.ip_forward = 1
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv6.conf.all.accept_ra = 0
SYS
sudo sysctl --system
```

تنظیم	معنا/کاربرد
ip_forward	مسیریابی بین اینترفیس‌ها (برای NAT/روتر لینوکسی)
tcp_syncookies	مقاوم‌سازی در برابر SYN flood
rp_filter	مسیر معکوس – کاهش جعل آدرس
accept_redirects=0	نادیده‌گرفتن ICMP redirect (جلوگیری از دستکاری مسیر)
accept_ra=0	خاموش‌کردن RA برای IPv6 در سرور استاتیک

Self-Test روز ۶ (۱۵ سؤال)

1. تفاوت موقتی/دائمی بودن `ip addr add` و روش دائمی در Ubuntu و RHEL؟
2. پنج فرمان پایه‌ی `ip` را با کاربریشان نام ببر.
3. معادل IOS برای `ip route add 10.99.0.0/24 via 10.20.0.1` چیست؟
4. `ping 8.8.8.8` کار می‌کند ولی `ping google.com` نه – چرا و چطور عیب‌یابی می‌کنی؟
5. `ss -tlnp` و `netstat -tulnp` چه تفاوتی در توزیع‌های مدرن دارند؟
6. `tcpdump` را برای دیدن فقط DNS بنویس.
7. تفاوت `scp` و `rsync` و کدام برای بکاپ؟

8. چهار اقدام اساسی سخت‌سازی SSH را نام ببر.
9. اگر بعد از تغییر `sshd_config` و `reload` نتوانی وارد شوی، چه می‌کنی؟
10. تفاوت `ufw`، `firewalld`، `nftables` و اینکه کدام لایه‌ی زیرین مشترک است؟
11. معادل IOS برای `ip access-group 101 in` در `nftables` چیست؟
12. دستور (PAT) Masquerade در `nftables` و تنظیم لازم کرنل؟
13. `ProxyJump` چه مشکلی را حل می‌کند؟
14. `sysctl` را برای فعال‌کردن مسیریابی و مقاوم‌سازی SYN flood بنویس.
15. `mtr` چه چیزی بیشتر از `traceroute` به تو می‌دهد؟

۷) بانک سؤال مصاحبه‌ی روز ۶ (۱۴ سؤال با پاسخ مدل)

- ۱) تنظیمات شبکه را در لینوکس چطور پیکربندی می‌کنی و چطور پایدارش می‌کنی؟ با `ip` (موقت/دیب‌اگ) و برای پایداری بر اساس توزیع: `netplan` در Ubuntu، `NetworkManager` / `nmcli` در RHEL، `systemd-networkd` در سیستم‌های سیستمی. همیشه تغییر را روی سرویس اعمال و با `systemctl status NetworkManager/networkd` و `ip -br a` تأیید می‌کنم، و برای تغییرات پرخطر `netplan` یا `try` یا کنسول پشتیبان دارم.
- ۲) تفاوت `ip` و `ifconfig` و چرا `ip`؟ `ifconfig` از بسته‌ی قدیمی `net-tools` است و بخشی از عملکرد (نام‌گذاری مدرن اینترفیس‌ها، مسیرهای پیشرفته، rule، VRF) را پوشش نمی‌دهد؛ `iproute2` (`ip`، `ss`، `bridge`، `tc`) استاندارد فعلی کرنل است، سریع‌تر و کامل‌تر، و احتمال حذف‌شدنش کم است. در مصاحبه نشان بده که هر دو را می‌شناسی ولی `ip` را به‌کار می‌بری.
- ۳) یک سرور «شبکه‌اش قطع است». مسیر عیب‌یابی؟ از پایین به بالا: `ip -br link` (UP؟)، `ethtool` (کابل/سرعت/CRC)، `ip -br a` (آدرس)، `ip neigh` (ARP گیت‌وی)، `ip route get 8.8.8.8` (مسیر/گیت‌وی)، `ping` گیت‌وی → اینترنت → DNS (`dig`)، `ss -tlnp` و `nc -zv` برای سرویس، و در لایه‌ی آخر `tcpdump` برای دیدن اینکه بسته‌ها بیرون می‌روند یا فایروال می‌بندد. هر مرحله یک لایه از OSI را رد یا تأیید می‌کند.
- ۴) `ss` بهتر است یا `netstat`؟ چرا؟ `ss` مستقیم از اطلاعات کرنل (`netlink`) می‌خواند و روی سیستم‌های پر بار بسیار سریع‌تر است؛ فیلترهای غنی دارد (`src`، `state`، `dport`). `netstat` از `/proc` می‌خواند، کندتر است و در بسیاری از توزیع‌ها به‌صورت پیش‌فرض نصب نیست (`net-tools deprecated`).
- ۵) تفاوت `-p` در `ss` برای `root` و کاربر عادی چیست؟ نمایش PID/نام فرایند مالک سوکت نیازمند دسترسی به اطلاعات فرایند است؛ کاربر عادی تنها سوکت‌های خودش را می‌بیند و در بقیه موارد PID نشان داده نمی‌شود. برای دیدن کامل باید `sudo ss -tlnp` بزنی. همین منطق در `lsof -i` هم هست.
- ۶) SYN flood چیست و چطور روی سرور لینوکس با آن مقابله می‌کنی؟ حمله‌ای که با SYN‌های نیمه‌باز صف backlog را پر می‌کند. دفاع: `tcp_syncookies=1`، افزایش `tcp_max_syn_backlog` و `somaxconn`، کاهش `tcp_synack_retries`، فعال‌کردن `SYNPROXY` در `nftables`، محدودسازی نرخ در فایروال/`fail2ban`، WAF، و در لایه‌ی شبکه هم ACL/rate-limit روی روتر و دستگاه‌های میانی (پل به هفته ۱).
- ۷) `iptables` و `nftables` چه تفاوتی دارند؟ جایگزین مدرن با یک موتور واحد برای IPv4/IPv6/ARP/bridge، نحو یکنواخت، مجموعه‌ها/maps و کارایی بهتر در قواعد زیاد است؛ `iptables` روی لینوکس‌های جدید معمولاً پشت `iptables-nft` شبیه‌سازی می‌شود. در نصب تازه `nft` را انتخاب می‌کنم؛ در سیستم‌های قدیمی باید هر دو را بلد باشم چون ابزارهایی مثل `docker` قواعد خود را از طریق `iptables` می‌سازند.
- ۸) چطور ثابت می‌کنی فایروال یا شبکه مانع است، نه اپلیکیشن؟ از دو سو: روی سرور `ss -tlnp` (سرویس گوش می‌دهد؟)، از کلاینت `nc` `curl -v` و `-zv host port` (کدام مرحله شکست می‌خورد؟)، هم‌زمان `tcpdump` روی سرور (بسته می‌رسد؟ پاسخ می‌رود؟). اگر SYN رسید و SYN/ACK نرفت، فایروال/سیاست؛ اگر SYN نرسید، شبکه/مسیر؛ اگر TCP کامل شد و پاسخ نبود، اپلیکیشن.

۹) کلید SSH را چطور مدیریت می‌کنی؟ ساخت با `ed25519` و `passphrase`, استفاده از `ssh-agent` (`ssh-add -t 8h`)، فایل `~/.ssh/config` برای میزبان‌ها، توزیع با `ssh-copy-id`، چرخش دوره‌ای کلیدها و حذف کلیدهای قدیمی از `authorized_keys`، استفاده از `Match` و `ForceCommand` برای محدودسازی، و در مقیاس سازمانی: `SSH CA` یا ابزارهای مدیریت کلید (Vault, Teleport). روی سرور: `chmod 700 ~/.ssh`، `600 authorized_keys`، مالکیت درست، و `PasswordAuthentication no`.

۱۰) NAT/masquerade روی لینوکس چطور انجام می‌شود و چه چیزی لازم است؟ `net.ipv4.ip_forward=1` (پایدار در `/etc/sysctl.d/` جدول `nat` و زنجیره‌ی `postrouting` با `masquerade` روی اینترفیس خروجی (`oifname`). در `iptables` معادل: `-t nat -A POSTROUTING -o eth0 -j MASQUERADE` باید مطمئن شوی `FORWARD` هم اجازه می‌دهد و برای حالت `routing-only` بدون NAT، مسیر بازگشت وجود دارد. همین مفهوم در `IOS ... overload ip nat inside source` است.

۱۱) `tcpdump` یک نمونه‌ی کاربردی از خروجی‌اش را توضیح بده. مثلاً، `IP 10.20.0.11.51234 > 10.20.0.10.443: Flags [S]`، یعنی بسته‌ی SYN از کلاینت به پورت ۴۴۳ با شماره‌ی توالی؛ اگر پاسخی با `[S.]` ندیدیم، مقصد بسته را ندیده یا فایروال `drop` کرده است؛ دیدن `[R]` نشان می‌دهد بسته رسیده ولی پورته‌ی گوش نمی‌دهد. همین خواندن ساده، ۸۰٪ ترابل‌شوت شبکه را حل می‌کند.

۱۲) `MTU` مشکل‌ساز می‌شود – چطور تشخیص و رفع می‌کنی؟ علامت: اتصال برقرار می‌شود ولی انتقال حجم بزرگ `hang` می‌کند (مثلاً TLS handshake یا دانلود). تست: `ping -M do -s 1472 host` (۱۴۷۲ = ۲۸ + ۱۵۰۰) و کاهش تدریجی تا موفقیت. رفع: اصلاح `MTU` اینترفیس/تونل (`ip link set dev tun0 mtu 1400`)، `TCP MSS clamping` در فایروال (`nft ... tcp flags syn tcp option`)، و بررسی مسیر شامل `VPN/GRE/PPPoE`، `(maxseg size set rt mtu)`.

۱۳) `ProxyJump` و `bastion` را توضیح بده. در معماری امن، سرورها در شبکه‌ی خصوصی‌اند و فقط یک میزبان عمومی (`bastion`) دسترسی SSH دارد. با `ssh -J user@bastion target` یا `ProxyJump` در `~/.ssh/config`، اتصال از طریق `bastion` تونل می‌شود، کلیدها فقط روی مسیر امن منتقل می‌شوند و لاگ دسترسی متمرکز می‌شود؛ مزیت: تنها یک نقطه‌ی ورود برای سخت‌سازی و پایش. برای انتقال فایل هم `scp -o ProxyJump=...` کار می‌کند.

۱۴) می‌خواهی بفهمی کدام پروسه با یک IP خارجی ارتباط دارد، چه می‌کنی؟

```
ss -tnp | head -20 # اتصال‌های TCP
ss -tnp dst 203.0.113.5
lsof -i @203.0.113.5
تأیید ترافیک # sudo tcpdump -nn host 203.0.113.5 -c 20
sudo netstat -plan | grep ESTABLISHED | head
```

در سناریوی امنیتی (هفته ۳)، همین کار گام اول تشخیص بیکن C2 است – و پس از آن بررسی `/proc/<pid>/exe`، `lsof -p`، و ریشه‌ی فرایند برای پاسخ‌دهی به رخداد.

۸) چک‌لیست تکمیل روز ۶ و جمع‌بندی هفته ۲

- آدرس‌دهی را با `ip` موقت و با `nmcli / netplan` پایدار انجام داده‌ام
- `ip route get` و مسیر پیش‌فرض را می‌خوانم و دستی مسیر اضافه/حذف می‌کنم
- `getent`، `dig`، `resolvectl` و `/etc/hosts` را برای DNS تمرین کرده‌ام
- `ss -tlnp` را برای پیدا کردن سرویس روی پورت اجرا کرده‌ام
- `tcpdump` روی DNS/HTTP زده‌ام و جریان را خوانده‌ام
- `mtr` و `-T traceroute` را برای مسیر استفاده کرده‌ام
- ورود کلیدمحور را راه انداختم و رمز SSH را غیرفعال کردم
- `~/.ssh/config` با یک `Host` و `ProxyJump` ساختم
- فایروال را با هر سه روش (`nftables`، `firewalld`، `ufw`) بستم و قواعد را دیدم
- NAT/masquerade را روی لینوکس با `ip_forward` راه انداختم

- [] `dnsmasq` را برای DHCP تست کردم (اختیاری ولی توصیه شده)
- [] لاگ شبکه را با `journalctl -u NetworkManager -b` بررسی کرده‌ام
- [] به هر ۱۴ سؤال مصاحبه‌ی این روز بلند جواب داده‌ام

هفته ۲ تمام شد. دوره‌ی بعدی (هفته ۳ – CEH) روی همین پایه می‌نشیند: پویش با `nmap` روی شبکه‌ای که در هفته ۱ ساختی، شنود با Wireshark/ `tcpdump` که امروز یاد گرفتی، و تحلیل سرور لینوکسی که این هفته مدیریتش را تمرین کردی.

بسته‌ی مطالعه‌ی LPIC-1 – جمع‌بندی هفته ۲

مرور، آزمون نهایی و بانک جامع مصاحبه

Week 2 Review • Final Mock Exam • 50-Question Interview Bank

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۲ • جمع‌بندی – جدول‌های طلایی + آزمون ۳۰ کوتاه، ۵۰ چندگزینه‌ای، ۱۰ تشریحی، ۱۰ سناریو، ۵ تسک عملی و ۵۰ سؤال مصاحبه (با پاسخنامه)

جمع‌بندی هفته ۲ – مرور روزهای ۱ تا ۶، آزمون‌های پایان هفته و بانک جامع مصاحبه

LPIC-1 (Linux) – از «مبانی» تا «شبکه‌ی سرور»

این فایل سه کار می‌کند: ۱) همه‌ی شش روز را به هم می‌بندد و «جدول‌های طلایی» را یک‌جا می‌آورد (مرور سریع پیش از مصاحبه). ۲) آزمون‌های پایان هفته را می‌گیرد: ۳۰ سؤال کوتاه + ۵۰ سؤال چندگزینه‌ای + ۱۰ کوتاه‌پاسخ + ۱۰ سناریوی عملی + ۵ تسک ترمینالی. ۳) بانک جامع مصاحبه‌ی هفته ۲ (۵۰ سؤال) را ارائه می‌دهد. همه‌ی پاسخ‌ها در انتهای همین فایل (فصل «پاسخنامه‌ی کامل») آمده‌اند؛ اول بدون دیدن آن‌ها جواب بده.

ه) زنجیره‌ی هفته ۲ – شش روز، یک منظومه

روز	موضوع	چیزی که به تو داد	اتصال به روز بعد
۱	مبانی و معماری	درخت FHS، کرنل/ماژول، ناوبری، pipe، لینک، عملیات فایل	«فایل‌ها را پیدا و جابه‌جا می‌کنم» → اما محتوا و مالکیتشان؟
۲	فرمان‌ها، مجوزها، فایل‌سیستم	پردازش متن، inode، rwx/umask/SUID/ACL، پارتیشن و LVM	«چه کسی مجاز است و داده کجا می‌نشیند» → اما این کارهای تکراری؟
۳	شل و اسکریپت‌نویسی	متغیر، شرط، حلقه، trap، دیباگ، بکاپ چرخشی	«کار تکراری را خودکار می‌کنم» → اما اجرای موازی و مراقبت از فرایندها؟
۴	فرایند و job control	ps/top ، سیگنال‌ها، cron/at ، nice/ionice ، timer	«چه چیزی در حال اجراست و چه وقت اجرا شود» → اما اینها چطور بالا می‌آیند؟
۵	systemd و بوت	unit نویسی، سرویس، GRUB، journald/logrotate	«سرویس‌ها و راه‌اندازی سیستم» → اما اتصال به بیرون؟
۶	شبکه‌ی لینوکس	ip ، ss ، tcpdump ، SSH، فایروال، NAT	«میزان، پایش و بستن درها» → و هفته ۳: حمله و دفاع روی همین سطح

جمله‌ی جمع‌بندی (پاسخ طلایی «هفته‌ی لینوکس چه یاد گرفتی؟»):

«یک سرور لینوکسی را از صفر می‌سازم، می‌فهمم چه چیزی در آن می‌گذرد، سرویس‌ها را با systemd مدیریت می‌کنم، کارها را خودکار می‌کنم، و در شبکه آدرس‌دهی/عیب‌یابی/فایروال/SSH را در دست دارم.»

۱) جدول‌های طلایی – مرور پیش از آزمون

۱.۱ درخت فایل‌سیستم (روز ۱)

مسیر	محتوا	نکته
/etc	کانفیگ‌ها	متنی، نسخه‌پشتیبان‌پذیر
/var	لاگ، صف، کش	اولین جایی که پر می‌شود
/usr/local / /usr	برنامه‌های توزیع / نصب دستی	مدیر بسته به /usr/local دست نمی‌زند
/boot	.vmlinuz، initramfs ، GRUB	حیاتی برای بوت
/sys ، /proc	مجازی: فرایند / دستگاه	در RAM
/dev ، /run ، /tmp ، /opt ، /home ، /root ، /srv ، /media ، /mnt	—	700 = /root ، 1777 = /tmp

۱.۲ مجوزها (روز ۲)

عدد	معنا	نمونه
644	--rW-r--r	فایل معمولی
600	-----rW	کلید خصوصی
755	rwXr-Xr-X	باینری/پوشه
700	-----rWX	خانه‌ی خصوصی
640	-----rW-r	کانفیگ حساس
1777	rwXrwxrwt	(sticky) /tmp
2775	rwXrwsr-X	پوشه‌ی تیمی (SGID)
4755	rwsr-Xr-X	باینری SUID

umask → فایل 666-umask ، پوشه 777-umask . حذف فایل = مجوز پوشه‌ی والد.

۱.۳ پردازش متن (روز ۲)

کار	فرمان
جست‌وجوی خط	grep -RIn "x" /etc
ویرایش خط	sed -i.bak -E 's/a/b/g' f
ستون و محاسبه	awk -F: '\$3>=1000{print \$1}' /etc/passwd
پرتکرارترین	sort uniq -c sort -nr head
بریدن ستون	cut -d: -f1,3

۱.۴ اسکریپت‌نویسی (روز ۳)

عناصر	قاعده
سرآغاز	set -euo pipefail + #!/usr/bin/env bash
نقل قول	"\$var" همیشه
پارامترها	\${1:-default} , @\$, \$# , \$1
خروج	0 موفق، 2 استفاده‌ی بد، 127 فرمان نیست، 130 Ctrl+C ، 137 SIGKILL/OOM
امنیت	eval روی ورودی ، flock ، trap cleanup EXIT ، mktemp
تست	PS4='+\$LINENO: ' , bash -x , shellcheck , bash -n

۱.۵ فرایند و سیگنال (روز ۴)

سیگنال	کاربرد
TERM 15	خاتمه‌ی مؤدبانه (پیش‌فرض)
KILL 9	خاتمه‌ی قهری (آخرین راه)
HUP 1	reload کانفیگ
INT / 20 TSTP 2	Ctrl+Z / Ctrl+C
CONT 18	ادامه‌دادن
0	فقط بررسی وجود (بدون ارسال)

nice → 20- تا 19 (CPU) | ionice → I/O | load < تعداد هسته = صف.

۱.۶ cron و timer (روز ۴ و ۵)

دقیقه ساعت روز ماه روز هفته
* * * * *

* * * * * 10/ * هر ۱۰ دقیقه | * * * * 0 2 روزی ۰۲:۰۰ | @reboot در بوت | Persistent=true در timer = جبران اجرای ازقلم‌افتاده.

۱.۷ زنجیره‌ی بوت (روز ۵)

#	مرحله	قطعه‌ی حیاتی
1	Firmware	UEFI+ESP یا BIOS+MBR
2	Bootloader	/boot/grub/grub.cfg → GRUB2 (تولیدشده!)
3	Kernel	initramfs + /boot/vmlinuz-*
4	ریشه و PID 1	LVM/LUKS → systemd , /etc/fstab
5	target	/ multi-user.target graphical.target
6	ورود	getty / display-manager / sshd
		سه دستور نجات: systemctl status , journalctl -xb , systemd-analyze blame

لینوکس	IOS
<code>ip -br a</code>	<code>show ip interface brief</code>
<code>ip route</code>	<code>show ip route</code>
<code>ip route add default via GW</code>	<code>ip route 0.0.0.0 0.0.0.0 GW</code>
<code>ip neigh</code>	<code>show ip arp</code>
<code>nft ... tcp dport 443 accept</code>	<code>access-list 101 permit tcp ... eq 443</code>
<code>chain input { hook input ... }</code>	<code>ip access-group 101 in</code>
<code>ip_forward=1 + nft ... postrouting masquerade</code>	<code>ip nat inside source ... overload</code>
<code>ss -tnp</code>	<code>show tcp brief</code>
<code>tcpdump -nn -i any</code>	<code>debug ip packet</code>
<code>sshd_config + ssh user@server</code>	<code>ssh admin@router</code>
<code>/etc</code> , فایل unit و کانفیگ در <code>systemctl enable</code>	<code>copy run start</code>

(۲) آزمون کوتاه – ۳۰ سؤال (۵ سؤال از هر روز)

الف) روز ۱ – مبانی و معماری

۱. تفاوت کرنل و توزیع در یک جمله؟
۲. `/proc` و `/sys` چه تفاوتی دارند؟
۳. تفاوت `hard link` و `symlink`؟
۴. `cmd 2>&1 > out.txt` چه اشکالی دارد؟
۵. تفاوت `ctime` , `mtime` , `atime` و دستور نمایش؟

ب) روز ۲ – فرمان‌ها، مجوزها، فایل سیستم

۱. خروجی `umask 027` برای فایل و پوشه‌ی جدید؟
۲. SUID چه خطری از دید امنیتی دارد؟
۳. `setfacl -m d:u:mary:rwX /srv/shared` چه می‌کند؟
۴. `df -h` فضا نشان می‌دهد ولی نمی‌توان فایل ساخت – سه علت؟
۵. سه لایه‌ی LVM و دلیل استفاده؟

ج) روز ۳ – شل و اسکریپت

۱. `set -euo pipefail` هر بخش چه می‌کند؟
۲. تفاوت `$@` و `$*` و کدام در حلقه؟
۳. `trap 'rm -f "$tmp"' EXIT` چرا لازم است؟
۴. `flock` چه مشکلی را در جاب‌های کرون حل می‌کند؟
۵. چگونه یک اسکریپت را بدون اجرا از نظر نحوی بررسی می‌کنی؟

د) روز ۴ – فرایند

1. زامبی چیست و چرا نمی‌توان آن را `kill -9` کرد؟
2. در `top` ستون `wa` بالا یعنی چه و با چه ابزاری عمیق‌تر می‌روی؟
3. تفاوت `nice` و `ionice`؟
4. `nohup` و `tmux` هر کدام چه می‌کنند؟
5. سه علت «کرون اجرا نشد»؟

ه) روز ۵ – systemd و بوت

1. تفاوت `is-active` و `is-enabled`؟
2. `disable` در برابر `mask`؟
3. `Type=oneshot` برای چه سرویس‌هایی؟
4. `daemon-reload` چه وقت لازم است؟
5. اگر `/etc/fstab` خطا داشته باشد بوت چه می‌شود و راه‌حل؟

و) روز ۶ – شبکه

1. `ping 8.8.8.8` کار می‌کند، `ping google.com` نه – علت و عیب‌یابی؟
2. معادل لینوکسی `ip route add 10.99.0.0/24 via 10.20.0.1` در IOS؟
3. `ss -tlnp` چه چیزی را و چرا بهتر از `netstat`؟
4. چهار اقدام سخت‌سازی SSH؟
5. `ip_forward=1` به همراه چه قاعده‌ای در `nftables` = NAT/PAT؟

۳) آزمون چندگزینه‌ای – ۵۰ سؤال

- ۱) کدام مسیر محل فایل‌های پیکربندی سیستم است؟ الف) `/var` ب) `/etc` ج) `/usr/lib` د) `/run`
- ۲) `/var/log` چرا مهم است؟ الف) فقط لاگ کرنل است ب) داده‌ی متغیر سیستم و اولین جای پر شدن است ج) فقط خواندنی و روی RAM است د) جای فایل‌های موقت است
- ۳) کدام فرمان مسیر واقعی یک symlink را نشان می‌دهد؟ الف) `ls -li` ب) `realpath` یا `readlink -f` ج) `find -l` د) `which -a`
- ۴) با `umask 022` مجوز پوشه‌ی جدید چیست؟ الف) 644 ب) 755 ج) 700 د) 750
- ۵) کدام بیت باعث می‌شود فایل‌های جدید در پوشه گروه پوشه را به ارث ببرند؟ الف) SUID ب) SGID ج) sticky د) umask
- ۶) `find / -perm -4000 -type f 2>/dev/null` چه می‌یابد؟ الف) فایل‌های ۴۰۰۰ بیتی ب) فایل‌های SUID ج) فایل‌های world-writable د) پوشه‌های private
- ۷) حذف یک فایل داخل پوشه‌ای به کدام مجوز بستگی دارد؟ الف) مجوز خود فایل ب) مجوز `w` روی پوشه‌ی والد ج) مالکیت فایل د) umask
- ۸) تفاوت `uniq` و `sort -u` در چیست؟ الف) هیچ تفاوتی نیست ب) `uniq` فقط تکراری‌های مجاور را حذف می‌کند ج) `sort -u` فقط عددی است د) `uniq` فایل را تغییر می‌دهد
- ۹) کدام فرمان ستون سوم `/etc/passwd` (UID) را برای کاربران $1000 \leq$ چاپ می‌کند؟ الف) `cut -d: -f3 /etc/passwd` ب) `sed -n '1,1000p' /etc/passwd` ج) `grep 1000 /etc/passwd` د) `awk -F: '$3>=1000{print $3}' /etc/passwd`

۱۰) برای بازوی خطوط ۱۰ تا ۲۰ در `sed` کدام درست است؟ الف) `sed -n '10,20p'` ب) `sed '10-20p'` ج) `sed -r 10:20` د) `sed -e 10,20`

۱۱) `df -i` چه می‌گوید؟ الف) مسیر دیسک‌ها ب) مصرف inode ج) I/O دیسک د) پارتیشن‌های غیرفعال

۱۲) کدام گزینه درباره‌ی XFS نادرست است؟ الف) عملکرد خوب روی فایل‌های بزرگ ب) در RHEL پیش‌فرض است ج) به راحتی قابل کوچک‌سازی است د) ژورنال‌دار است

۱۳) کدام گزینه در `/etc/fstab` باعث می‌شود نبود دستگاه بوت را نشکند؟ الف) `nofail` ب) `defaults` ج) `ro` د) `auto`

۱۴) ترتیب بزرگ‌کردن LV در ext4 چیست؟ الف) `resize2fs` بعد `lvextend` ب) `lvextend` بعد `resize2fs` ج) فقط `lvextend` کافی است د) اول `mkfs` سپس `lvextend`

۱۵) شبنگ صحیح برای اسکریپت `bash` چیست؟ الف) `#!/bin/bash` ب) `#!/usr/bin/env bash` ج) هر دو درست است د) `#!/bin/ bash`

۱۶) `set -u` چه می‌کند؟ الف) اجرا را متوقف می‌کند ب) استفاده از متغیر تعریف نشده را خطا می‌کند ج) خروجی را مرتب می‌کند د) خطاها را نادیده می‌گیرد

۱۷) خروجی `$?` بعد از فرمانی که فرمان پیدا نشده چیست؟ الف) 0 ب) 1 ج) 126 د) 127

۱۸) صفت خروجی 137 در یک سرویس چه معنایی دارد؟ الف) خروج عادی ب) `SIGTERM` ج) `SIGKILL` (معمولاً OOM) د) خطای مجوز

۱۹) کدام گزینه در `[[ ]]` کار می‌کند ولی در `[ ]` نه؟ الف) `-f` ب) `==` (regex) ج) `-z` د) `-d`

۲۰) `while read -r line; do ...; done < file` چه مزیتی بر `cat file | while read ...` دارد؟ الف) سریع‌تر است چون word splitting ندارد و در پوسته‌ی جاری اجرا می‌شود ب) رنگ می‌دهد ج) فایل را قفل می‌کند د) تفاوتی ندارد

۲۱) `mktemp` چه می‌کند؟ الف) فایل موقت با نام امن و یکتا می‌سازد ب) فایل را حذف می‌کند ج) پوشه‌ی `/tmp` را پاک می‌کند د) متغیر محیطی می‌سازد

۲۲) `shellcheck` چه کار می‌کند؟ الف) اسکریپت را اجرا و زمان‌سنجی می‌کند ب) تحلیل ایستا برای باگ‌ها و اشکالات رایج اسکریپت ج) مجوزها را چک می‌کند د) اسکریپت را کامپایل می‌کند

۲۳) کدام گزینه در اسکریپت خطرناک است؟ الف) `rm -rf "$dir"` ب) `rm -rf $dir` با `set -u` و متغیر خالی ج) `rm -i file` د) `find -delete` با تست قبلی

۲۴) تفاوت `( cd /tmp; ls )` و `{ cd /tmp; ls; }`؟ الف) یکی در زیرپوشته و دیگری در پوسته‌ی جاری اجرا می‌شود ب) هیچ تفاوتی ندارند ج) دومی خطا می‌دهد د) اولی سریع‌تر است

۲۵) `fork` و `exec` بهترتیب چه می‌کنند؟ الف) اجرا و پایان ب) ساخت کپی از فرایند و جایگزینی حافظه با برنامه‌ی جدید ج) توقف و ادامه د) خواندن و نوشتن

۲۶) زامبی چه وضعیتی در `ps` دارد؟ الف) `R` ب) `D` ج) `T` د) `Z`

۲۷) `kill -HUP <pid>` معمولاً چه می‌کند؟ الف) فرایند را می‌کشد ب) درخواست reload کانفیگ ج) متوقف می‌کند د) اولویت را تغییر می‌دهد

۲۸) `load average: 12` روی سیستمی با ۴ هسته یعنی؟ الف) ۱۲ کاربر ب) ۳ برابری نسبت به ظرفیت ج) ۱۲ درصد CPU د) ۱۲ فرایند زامبی

۲۹) در `top` ستون `wa` بالا نشانه‌ی چیست؟ الف) مصرف CPU کاربر ب) انتظار I/O ج) زمان دزدیده شده د) مصرف swap

۳۰) کدام اولویت `nice` بالاترین است؟ الف) -20 ب) 0 ج) 19 د) 10

۳۱) چرا برای بکاپ سنگین `ionice` لازم است؟ الف) چون `nice` فقط CPU را کنترل می‌کند و نه I/O را ب) چون `ionice` سرعت دیسک را زیاد می‌کند ج) چون `nice` منسوخ است د) چون `ionice` رمزنگاری می‌کند

۳۲) `Ctrl+Z` چه سیگنالی می‌فرستد؟ الف) `SIGINT` ب) `SIGTSTP` ج) `SIGKILL` د) `SIGCONT`

- ۳۳) در cron، 1-5 * * 15 3 چه زمانی اجرا می‌شود؟ (الف) هر روز ۱۵:۰۳ (ب) ساعت ۰۳:۱۵ دوشنبه تا جمعه (ج) ۱۵ روز قبل از هر ماه (د) پنج‌شنبه‌ها ۰۳:۱۵
- ۳۴) کدام گزینه «اجرای جبرانی بعد از خاموش بودن سرور» را در timer فراهم می‌کند؟ (الف) OnCalendar (ب) Persistent=true (ج) RandomizedDelaySec (د) AccuracySec
- ۳۵) در /etc/crontab ستون ششم (اضافه بر crontab کاربر) چیست؟ (الف) مسیر لاگ (ب) نام کاربری (ج) اولویت (د) زمان‌بندی
- ۳۶) systemctl enable --now svc چه می‌کند؟ (الف) فقط برای بوت فعال می‌کند (ب) فعال می‌کند و همین حالا استارت می‌زند (ج) فقط استارت می‌کند (د) ماسک می‌کند
- ۳۷) mask با disable چه تفاوتی دارد؟ (الف) هیچ (ب) mask جلوگیری کامل از هر راه استارت است (ج) mask وضعیت پویا را پاک می‌کند (د) disable برای timerهاست
- ۳۸) کدام Type برای یک اسکریپت پیکربندی که تمام می‌شود و باید فعال بماند؟ (الف) simple (ب) forking (ج) oneshot (د) RemainAfterExit=yes
- ۳۹) بعد از تغییر یک فایل service، چه می‌زنی؟ (الف) فقط restart (ب) systemctl daemon-reload و سپس restart (ج) systemctl reexec (د) systemd-analyze verify
- ۴۰) journalctl -b -1 -p err چه می‌دهد؟ (الف) فقط خطاهای این بوت (ب) خطاهای بوت قبلی (ج) همه‌ی لاگ‌ها (د) لاگ کرنل
- ۴۱) در GRUB، کدام فایل را دستی ویرایش نمی‌کنیم؟ (الف) /etc/default/grub (ب) /etc/grub.d/40_custom (ج) /boot/grub/ (د) grub.cfg
- ۴۲) update-initramfs -u چه وقت لازم است؟ (الف) هر بازه‌ی یک‌ماهه (ب) بعد از تغییر کانفیگی که بر بوت اولیه اثر دارد (مثل /etc/crypttab/fstab) (ج) فقط در نصب اولیه (د) بعد از نصب هر بسته
- ۴۳) علت اصلی Give root password for maintenance چیست؟ (الف) کرنل پنیک (ب) خطای mount در fstab یا مشکل فایل‌سیستم (ج) خرابی GRUB (د) نبود رمز root
- ۴۴) ss -tlnp چه سوکت‌هایی را نشان می‌دهد؟ (الف) TCP گوش‌دهنده (ب) UDP فعال (ج) همه‌ی سوکت‌ها (د) اتصال‌های بسته
- ۴۵) کدام فرمان برای مشاهده‌ی DNS در systemd-resolved است؟ (الف) resolvectl status (ب) systemctl dns (ج) nmcli (د) dig -s
- ۴۶) tcpdump -nn port 53 چه چیزی می‌گیرد؟ (الف) HTTP (ب) DNS (ج) SSH (د) ICMP
- ۴۷) کدام گزینه در sshd_config ورود با رمز را می‌بندد؟ (الف) PermitRootLogin no (ب) PasswordAuthentication no (ج) PubkeyAuthentication yes (د) UsePAM yes
- ۴۸) قبل از reload کردن sshd چه می‌زنی؟ (الف) sshd -t (ب) ssh -T (ج) systemctl daemon-reload (د) ssh-keygen -t
- ۴۹) در nftables، معادل policy drop روی زنجیره‌ی input چیست؟ (الف) permit ip any any (ب) deny (ج) همه چیز مگر قواعد (accept) (د) log همه (د) masquerade
- ۵۰) برای NAT روی لینوکس علاوه بر قاعده‌ی masquerade چه چیزی لازم است؟ (الف) net.ipv4.ip_forward=1 (ب) net.ipv6.conf.all.forwarding=1 (ج) sysctl vm.swappiness=10 (د) dnsmasq

۴) آزمون کوتاه‌پاسخ – ۱۰ سؤال تشریحی

۱. پنجره‌ی اول عیب‌یابی «سرور کند شده است» را در ۶ گام بنویس (فرمان + چه چیزی را می‌گوید).
۲. یک اسکریپت بکاپ بنویس که فقط با آرگومان‌ها کار کند، خطاها را به stderr بفرستد، فایل موقت امن بسازد و در خروج پاک‌سازی کند (اسکلت + نکات کلیدی).
۳. تفاوت‌های عملی SUID، SGID و sticky را با یک مثال واقعی برای هر کدام توضیح بده و بگو کدام‌شان ریسک امنیتی جدی دارد و چرا.

4. مسیر کامل بوت را از UEFI تا login بنویس و بگو در هر مرحله یک خرابی نمونه چه خطایی می‌سازد.
5. تفاوت `cron`، `at` و `systemd timer` را در یک جدول سهردیفه بنویس و بگو هر کدام را کجا انتخاب می‌کنی.
6. یک سرور «نمی‌تواند به اینترنت وصل شود» – به ترتیب کدام لایه‌ها را با چه فرمان‌هایی رد یا تأیید می‌کنی؟
7. چرا `kill -9` راه دوم است و نه اول؟ در چه حالتی مجبور می‌شوی از آن استفاده کنی و بعد از آن چه بررسی‌ای می‌کنی؟
8. مجوزها، SELinux/AppArmor، ACL و فایروال را از نظر «لایه‌ی کنترل دسترسی» مرتب کن و بگو در مواجهه با `Permission denied` به چه ترتیبی سراغ هرکدام می‌روی.
9. چطور یک سرویس را از «دستی اجرا می‌شود» به «سرویس systemd امن و مقاوم» تبدیل می‌کنی؟ حداقل ۸ کلیدواژه‌ی unit را نام ببر و نقش هرکدام را بگو.
10. یک سناریوی واقعی بنویس که در آن «SSH را سخت کردم و خودم بیرون افتادم» – چه اشتباهی رخ داده و رویه‌ی پیشگیری چیست؟

۵) آزمون سناریومحور – ۱۰ سناریو

- س۱ – دیسک پر شده است. سرور وب با «No space left on device» خطا می‌دهد. گزارش تشخیص و رفع در ۸ گام (فرمان‌ها + تصمیم‌ها + پیشگیری).
- س۲ – سرور کند شده است. `uptime` عدد ۱۸ روی ۴ هسته نشان می‌دهد، CPU نمودار ۳۰٪، `wa` = ۶۵٪. تشخیص، ابزارها، علت محتمل و اقدام.
- س۳ – کاربر جدید و دسترسی تیمی. تیم جدید `analytics` باید روی `/srv/data` بخواند/بنویسد، یک کاربر فقط‌خواندن، و فایل‌های جدید گروه `analytics` بگیرند. مجوزها و ACL‌های دقیق را بنویس.
- س۴ – سرویس بعد از ری‌بوت بالا نمی‌آید. سرویس `hello.service` دستی استارت می‌شود ولی در بوت نه. ۵ علت ممکن و روش تشخیص هرکدام.
- س۵ – رمز `root` را گم کرده‌ایم. مراحل بازیابی روی سروری که فیزیکی/VM است و به کنسول دسترسی داریم (سه روش).
- س۶ – بکاپ لازم است. بکاپ روزانه‌ی `/etc` و `/var/www` با نگهداری ۱۴ نسخه، لاگ‌دار، زمان‌بندی‌شده، بدون قطع سرویس. طراحی کامل (اسکرپت + زمان‌بندی + تست بازیابی).
- س۷ – ورود SSH کند شده است. ورود ۲۰ SSH ثانیه طول می‌کشد. چه چیزی باعث می‌شود و چطور با `sshd -d` و لاگ عیب‌یابی می‌کنی؟
- س۸ – حمله‌ی `brute force` روی SSH. ۵۰۰۰ تلاش ناموفق در یک ساعت. تشخیص با لاگ، اقدام فوری، اقدام پیشگیرانه و گزارش به مدیریت.
- س۹ – می‌خواهی سرور لینوکسی را به‌عنوان روتر/NAT بین دو شبکه راه بیندازی. کانفیگ کامل (اینترفیس‌ها، `forward`، `masquerade`، DHCP، DNS و اتصال به مفاهیم هفته ۱).
- س۱۰ – زامبی‌ها زیاد شده‌اند. ۲۰۰۰ فرایند در حالت `Z` روی سرور تولید. علت‌یابی، پیدا کردن والد، رفع موقت و رفع ریشه‌ای.

۶) آزمون عملی روی ترمینال – ۵ تسک با معیار قبولی

#	تسک	معیار قبولی
۱	ساخت ساختار <code>/srv/app/{conf,logs,data}</code> با گروه <code>app</code> ، کاربر سرویس <code>appsvc</code> ، SGID و ACL خواندن برای <code>audit</code>	<code>find /srv/app -printf '%M %u %g %p\n'</code> خروجی مورد انتظار بدهد؛ کاربر نامجاز نتواند بنویسد
۲	اسکرپت <code>backup.sh</code> با آرگومان <code>set -euo pipefail</code> ، <code>trap</code> ، لاگ، چرخش ۷ نسخه و کد خروج معنادار	<code>shellcheck</code> بدون خطای surface؛ اجرای خطادار کد ۱ بدهد؛ ۱۰ اجرا → ۷ نسخه بماند
۳	سرویس <code>hello.service</code> با کاربر غیرمجازمحرور، <code>Restart</code> ، <code>MemoryMax</code> ، <code>ProtectSystem</code>	<code>systemctl is-active/is-enabled</code> فعال؛ <code>kill</code> → خودکار بالا بیاید؛ <code>systemctl show</code> سقف حافظه را نشان دهد
۴	timer روزانه ۰۲:۳۰ با <code>Persistent=true</code> برای بکاپ	<code>systemctl list-timers</code> ورودی را نشان دهد؛ <code>journalctl -u backup.service</code> لاگ موفق بدهد
۵	سخت‌سازی SSH + فایروال <code>nftables</code> با قاعده‌های 22/80/443 ، drop ، ICMP، پیش‌فرض و NAT برای شبکه‌ی داخلی	ورود کلیدمحرور موفق؛ ورود رمزی رد شود؛ <code>nft list ruleset</code> قواعد را نشان دهد؛ کلاینت داخلی اینترنت داشته باشد

۷) بانک جامع مصاحبه‌ی هفته ۲ – ۵۰ سؤال

الف) معماری و فایل سیستم

۱. لینوکس چیست و از چه اجزایی ساخته شده است؟ ۲. FHS چیست و چرا اهمیت دارد؟ ۳. تفاوت `/usr/local` و `/opt` ؟ ۴. `proc` و `/sys` چه تفاوتی دارند؟ ۵. سه زمان فایل (`ctime` ، `mtime` ، `atime`) و فرمان نمایش همه؟ ۶. `inode` چیست و چه وقت تمام می‌شود؟ ۷. تفاوت `find` و `locate` و انتخاب درست؟ ۸. `xargs` چرا لازم است؟

ب) مجوزها و امنیت فایل

۹. مجوز 640 برای چه فایل‌ی مناسب است؟ ۱۰. تفاوت SUID و SGID با مثال؟ ۱۱. sticky bit چه حل می‌کند؟ ۱۲. `umask` چگونه محاسبه می‌شود؟ ۱۳. ACL را توضیح بده و چه وقت لازم است؟ ۱۴. `sudo` در برابر `su` – کدام و چرا؟ ۱۵. لیست SUID را چگونه مدیریت و چطور پایش می‌کنی؟

ج) پردازش متن و ابزارها

۱۶. تفاوت `grep` ، `sed` ، `awk` ؟ ۱۷. چطور ۱۰ IP پرمصرف را از لاگ وب در می‌آوری؟ ۱۸. تفاوت `>` و `>>` و خطر رایج؟ ۱۹. `tee` چه کاربردی دارد؟ ۲۰. تفاوت `sort` و `sort -nr | uniq -c` در یک خط تحلیل؟

د) اسکرپت‌نویسی

۲۱. `set -euo pipefail` را توضیح بده. ۲۲. `trap` و کاربردهای واقعی؟ ۲۳. تفاوت `'...'` ، `"..."` و بدون نقل‌قول؟ ۲۴. تفاوت `source script.sh` ، `./script.sh` ، `bash script.sh` ؟ ۲۵. اسکرپت «idempotent» یعنی چه؟ ۲۶. چطور یک اسکرپت را امن و قابل اتکا تحویل می‌دهی (تست‌ها + مستندسازی)؟ ۲۷. کد خروج‌ها چه معنایی دارند و چرا در خودکارسازی مهم‌اند؟ ۲۸. چطور ورودی کاربر را اعتبارسنجی می‌کنی؟

ق۲۹. فرایند چطور ساخته می‌شود (`fork/exec/wait`)؟ ق۳۰. زامبی، یتیم و راه‌حل هرکدام؟ ق۳۱. `load average` چطور تفسیر می‌شود؟ ق۳۲. `top` را بخوان: `us` , `sy` , `wa` , `st` , `si/so` چه می‌گویند؟ ق۳۳. تفاوت `SIGTERM` و `SIGKILL` و ترتیب درست؟ ق۳۴. `nice` و `ionice` چه تفاوتی دارند؟ ق۳۵. `nohup` , `disown` , `setuid` , `tmux` و انتخاب درست؟ ق۳۶. `cron` چرا گاهی اجرا نمی‌شود؟ (حداقل ۴ علت) ق۳۷. `systemd timer` در برابر `cron` ؟

و) systemd و بوت

ق۳۸. `systemd` چه مزیتی بر `SysV init` دارد؟ ق۳۹. سه سطح وضعیت سرویس و فرمان‌های هرکدام؟ ق۴۰. یک `unit` سرویس با کلمات کلیدی حیاتی بنویس. ق۴۱. تفاوت `simple` , `forking` , `oneshot` ؟ ق۴۲. `drop-in` و `systemctl edit` چرا بهترین شیوه است؟ ق۴۳. زنجیره‌ی بوت را توضیح بده. ق۴۴. `fstab` خراب و بازیابی؟ ق۴۵. بازیابی رمز `root` در سه روش؟ ق۴۶. `journalctl` و `logrotate` چطور دیسک را کنترل می‌کنند؟

ز) شبکه‌ی لینوکس

ق۴۷. تنظیمات شبکه را پایدار چطور اعمال می‌کنی (سه توزیع)؟ ق۴۸. «سرور به بیرون وصل نمی‌شود» را چطور عیب‌یابی می‌کنی (لایه‌به‌لایه)؟ ق۴۹. `sshd` را چطور سخت می‌کنی؟ (فهرست کامل + `fail2ban` + `bastion`) ق۵۰. فایروال میزبان را با `ufw` / `firewalld` / `nftables` مقایسه کن و NAT لینوکسی را بنویس.

۸) چک‌لیست پایانی هفته ۲ و نقشه‌ی هفته ۳

۸.۱ چک‌لیست پایانی

- [] هر شش روز را با جدول‌های طلایی در کمتر از ۳۰ دقیقه مرور می‌کنم
- [] آزمون ۳۰ سؤالی و ۵۰ چندگزینه‌ای را بدون پاسخنامه زده‌ام
- [] نمره‌ی MCQ من ۴۰+ از ۵۰ است (زیر ۳۵ = مرور هدفمند روزهای ضعیف)
- [] هر ۱۰ سناریو را روی کاغذ/ترمینال طراحی کرده‌ام
- [] هر ۵ تسک عملی را در VM اجرا و معیار قبولی را راستی‌آزمایی کرده‌ام
- [] به ۵۰ سؤال بانک مصاحبه بلند جواب داده‌ام (نه در ذهن)
- [] لابراتوار VM من شامل: دو VM لینوکس + کلید SSH + فایروال فعال + بکاپ زمان‌بندی شده است
- [] همه‌ی اسکرپت‌ها و فایل‌های `unit` من در یک مخزن `Git` هستند (پل به هفته ۳ و رزومه)

روز	موضوع	پایه‌ی هفته ۲ که مستقیماً استفاده می‌شود
۱	امنیت اطلاعات، CIA، قوانین	مدیریت سرویس/لاگ و مالکیت فایل
۲	Reconnaissance و Footprinting	nc, whois, dig, اسکرپت‌نویسی
۳	Enumeration و Scanning	ss, nmap, سرویس‌های systemd, netstat
۴	Privilege Escalation و System Hacking	SUID, /etc/passwd, sudo -l, فرایند و کرنل
۵	Threats و Malware	/proc, lsof, کشف فرایند مشکوک، پایداری با cron/systemd
۶	Security و Web Attacks	nginx/apache, لاگ‌ها، فایروال، TLS و SSH

۹) پاسخنامه‌ی کامل (بخش نهایی)

۹.۱ پاسخ آزمون کوتاه (۳۰ سؤال)

۱. کرنل = هسته‌ی سیستم‌عامل (حافظه، CPU، فایل‌سیستم، شبکه، درایور)؛ توزیع = کرنل + ابزارهای GNU + مدیر بسته + مخازن + سرویس‌ها + نصب‌کننده.
۲. /proc پنجره‌ای به فرایندها و وضعیت کرنل است (/proc/<pid>), /sys مدل دستگاه/درایور را نشان می‌دهد (sysfs). هر دو مجازی و در RAM.
۳. Hard link به inode اشاره می‌کند (عبور از فایل‌سیستم ممکن نیست، حجم داده می‌گیرد)؛ symlink یک مسیر است (قابل شکستن، قابل ساخت بین فایل‌سیستم‌ها) – ln در برابر ln -s.
۴. در cmd 2>&1 > out.txt ابتدا stderr به ترمینال فعلی وصل و بعد stdout به فایل منتقل می‌شود؛ درست: cmd > out.txt 2>&1. ریدایرکت‌ها چپ‌به‌راست ارزیابی می‌شوند.
۵. atime آخرین دسترسی، mtime آخرین تغییر محتوا، ctime آخرین تغییر متادیتا – با stat.
۶. فایل 640 (زیر umask 027) و پوشه 750.
۷. باینری SUID با دسترسی مالک اجرا می‌شود؛ اگر باینری آسیب‌پذیر یا مخرب باشد (یا SUID روی ابزاری مثل vim / find) مستقیم به ارتقای دسترسی تا root منتهی می‌شود.
۸. ACL پیش‌فرض روی پوشه می‌سازد: فایل‌های جدید در /srv/shared برای کاربر mary مجوز rwx ارث می‌برند.
۹. (الف) inode تمام شده (df -i)؛ (ب) فایل حذف‌شده‌ای که فرایند باز نگه داشته (lsof +L1)؛ (ج) سهمیه‌ی کاربر/گروه یا فایل‌سیستم فقط‌خواندنی.
۱۰. PV → VG → LV؛ مزایا: توسعه/کوچک‌سازی آنلاین، اسنپ‌شات، pool مشترک چند دیسک، نام‌گذاری معنایی.
۱۱. -e توقف روی خطای فرمان، -u خطای متغیر تعریف‌نشده، -o pipefail شکست کل خط لوله در صورت خطای هر عضو.
۱۲. @\$ هر آرگومان را مستقل نگه می‌دارد (در حلقه با "\$@" استفاده کن)؛ \$* همه را یک رشته می‌کند.
۱۳. تضمین پاک‌سازی فایل موقت در خروج (حتی روی خطا/سیگنال) → جلوگیری از پر شدن /tmp و نشت داده.
۱۴. از اجرای هم‌زمان دو نسخه‌ی اسکرپت (مثلاً کرون با اجرای طولانی) جلوگیری می‌کند؛ flock -n 9 || exit.
۱۵. bash -n script.sh (نحو) و برای تحلیل عمیق‌تر shellcheck.
۱۶. فرایند تمام‌شده‌ای که والدش wait نگرفته؛ چون مرده است سیگنال نمی‌گیرد – فقط ورودی جدول فرایند را اشغال می‌کند؛ راه‌حل اصلاح والد یا ری‌استارت والد (init آن‌های یتیم را جمع می‌کند).
۱۷. wa = انتظار I/O؛ نشانه‌ی دیسک کند/بکاپ سنگین/NFS؛ عمیق‌تر با iostat -xz, iotop, vmstat.

18. `nice` اولویت CPU، `ionice` اولویت/کلاس I/O (کلاس `idle` برای بکاپ).
19. `nohup` فرایند را در برابر `SIGHUP` قطع SSH مقاوم و ورودی/خروجی را جدا می‌کند؛ `tmux` نشست تعاملی پایدار با امکان اتصال دوباره می‌دهد.
20. `PATH` و محیط محدود کرون، نبود `+x`/شَبَنگ اشتباه، نبود مسیر مطلق/ریدایرکت خروجی، کاراکتر `%`، خاموش بودن سرویس `cron`، متغیرهای تعریف نشده.
21. `is-active` وضعیت در حافظه، `is-enabled` فعال بودن در بوت (پیوندهای `systemd`).
22. `disable` پیوند بوت را حذف می‌کند (استارت دستی ممکن است)؛ `mask` به `/dev/null` پیوند می‌زند و هر راه استارت را می‌بندد.
23. کارهای یک‌باره/اسکرپیتی مثل مونت، پیکربندی، اتمام بکاپ؛ با `RemainAfterExit=yes` «فعال» باقی می‌ماند.
24. هر زمان فایل `unit` یا `drop-in` تغییر کند؛ بدون آن `systemd` نسخه‌ی قدیمی را در حافظه دارد.
25. سیستم در `emergency mode` می‌ایستد؛ ورود با رمز `root`، `mount -o remount,rw /`، `journalctl -xb`، اصلاح `fstab` (یا `nofail`)، `mount -a`، `findmnt --verify`، `reboot`.
26. DNS؛ مسیر: `resolvectl query`، `cat /etc/resolv.conf`، `dig @1.1.1.1`، `getent hosts`، بررسی `/etc/` `nsswitch.conf` و `/etc/hosts` و فایروال `UDP/53`.
27. در `IOS`: `ip route 10.99.0.0 255.255.255.0 10.20.0.1` (یا `ip route add ...`).
28. سوکت‌های TCP گوش‌دهنده با `PID`/برنامه؛ `ss` از `netlink` می‌خواند و بسیار سریع‌تر از `netstat` (که از `/proc` و منسوخ است).
29. `PasswordAuthentication no` + کلیدمحور، `PermitRootLogin no`، `Match / AllowUsers` محدود، کاهش `LoginGraceTime / MaxAuthTries`، تغییر پورت + `fail2ban`، غیرفعال کردن `forwarding`، پایش لاگ.
30. `ip_forward=1` + قاعده‌ی `postrouting ... masquerade` روی اینترفیس خروجی (به‌همراه سیاست `FORWARD` مناسب).

۹.۲ پاسخ‌نامه‌ی ۵۰ سؤال چندگزینه‌ای

س	پاسخ	س	پاسخ	س	پاسخ	س	پاسخ	س	پاسخ
۱	ب	۱۱	ب	۲۱	الف	۳۱	الف	۴۱	ج
۲	ب	۱۲	ج	۲۲	ب	۳۲	ب	۴۲	ب
۳	ب	۱۳	الف	۲۳	ب	۳۳	ب	۴۳	ب
۴	ب	۱۴	ب	۲۴	الف	۳۴	ب	۴۴	الف
۵	ب	۱۵	ج	۲۵	ب	۳۵	ب	۴۵	الف
۶	ب	۱۶	ب	۲۶	د	۳۶	ب	۴۶	ب
۷	ب	۱۷	د	۲۷	ب	۳۷	ب	۴۷	ب
۸	ب	۱۸	ج	۲۸	ب	۳۸	ج	۴۸	الف
۹	ب	۱۹	ب	۲۹	ب	۳۹	ب	۴۹	ب
۱۰	الف	۲۰	الف	۳۰	الف	۴۰	ب	۵۰	الف

نمره‌گذاری MCQ: +۴۵ = آماده‌ی مصاحبه | ۳۵-۴۴ = مرور هدفمند روزهای ضعیف | زیر ۳۵ = مرور کامل با جدول‌های طلایی.

توضیح نکات تله‌دار: سؤال ۱۵ (هر دو شَبَنگ درست است، ولی `env` قابل انتقال‌تر است) | سؤال ۱۸ ($9+128 = 137$) `SIGKILL` → سؤال ۲۰ (نسخه‌ی `file` < در پوسته‌ی جاری اجرا می‌شود و متغیرها بیرون می‌مانند) | سؤال ۳۰ (`-20` بالاترین اولویت) | سؤال ۴۴ (فقط گوش‌دهنده‌های TCP).

(۳۱) سرور کند شده – شش گام:

```
۱) uptime -> load average نسبت به nproc
۲) top -b -n1 -> وضعیت کاربر، us/sy/wa/st، swap
۳) vmstat 2 5 -> r/b، si/so، فشار حافظه
۴) iostat -xz 2 5 -> %util و await دیسک‌ها
۵) ps --sort=-%cpu|-%mem | head -> PID واقعی مقصر
۶) ss -s ; lsof -i -> فشار شبکه/اتصالات
```

سپس اقدام: مهار (limit/renice/kill) مؤدبانه، رفع علت، ثبت زمان خط (timeline)، پیشگیری (پایش، محدودیت منابع unit، هشدار).

(۳۲) اسکلت اسکریپت بکاپ (نکات کلیدی):

```
#!/usr/bin/env bash
set -euo pipefail
tmp="$(mktemp)"; trap 'rm -f "$tmp"' EXIT
log(){ printf '%s %s\n' "$(date +%F %T)" "$*"; }
err(){ log "ERROR: $" >&2; }
[[ $# -eq 2 ]] || { err "usage: $0 <src> <dst>"; exit 2; }
src="$1"; dst="$2"; [[ -d "$src" ]] || { err "no such dir: $src"; exit 2; }
mkdir -p "$dst"
arc="$dst/${basename "$src"}-$(date +%Y%m%d-%H%M%S).tar.gz"
tar -czf "$arc" -C "$(dirname "$src")" "$(basename "$src")" || { err "tar failed"; exit 1; }
[[ -s "$arc" ]] || { err "empty archive"; exit 1; }
log "OK $arc $(du -h "$arc" | cut -f1)"
```

نکات: اعتبارسنجی ورودی، `mkdir -p` (idempotent)، نقل قول کامل، کد خروج معنادار، لاگ با مهر زمان، تست صحت آرشیو، قفل با `flock` در اجرای کرون، مستندسازی در سرآغاز فایل.

(۳۳) SUID / SGID / Sticky

- SUID (4755): اجرای باینری با UID مالک – مثال `/usr/bin/passwd` (نوشتن در `/etc/shadow`). **ریسک امنیتی جدی:** هر SUID آسیب‌پذیر یا اضافی = مسیر ارتقای دسترسی؛ باید فهرست پایه داشته باشی و تغییرات را پایش کنی.
- SGID **روی فایل** (2755): اجرا با GID گروه مالک. **روی پوشه** (2775): فایل‌های جدید گروه پوشه را ارث می‌برند – مثال: پوشه‌ی تیمی مشترک؛ ریسک کم، خطای رایج در فراموش کردن آن.
- Sticky (1777): در پوشه فقط مالک/root می‌تواند فایل دیگران را حذف کند – مثال `/tmp`؛ بدون آن هر کاربری فایل هر کاربری را حذف می‌کرد.

(۳۴) مسیر بوت و خرابی نمونه: UEFI/BIOS (خرابی: ترتیب بوت/NVRAM → پیام «no bootable device» → GRUB (خرابی: نبود `grub.cfg` → منوی `grub>` → `update-grub / grub-install` → `grub+initramfs` کرنل (خرابی: `VFS: Unable to mount root` → `fs` → بازسازی `initramfs`/اصلاح پارامتر `root=` → `mount` ریشه `/fstab` (خرابی: `emergency mode` → اصلاح `fstab` → `systemd/` → `target` (خرابی: سرویس `failed` → `journalctl -b`، `systemctl --failed` → `login` (خرابی: `sshd` نیامده → کنسول و `systemctl status ssh`).

(۳۵) `cron / at / systemd timer`

معیار	cron	at	systemd timer
نوع	تکراری	یک باره	تکراری
دقت	۱ دقیقه	ثانیه‌ای (تقریبی)	ثانیه‌ای + AccuracySec
لاگ	syslog/cron log	لاگ atd	journal یکپارچه
جبران ازقلم افتاده	ندارد (anacron لازم)	-	Persistent=true
کنترل منابع	ندارد	ندارد	Nice/IOWeight/MemoryMax
انتخاب من	سازگاری، سیستم قدیمی	کار یک باره‌ی زمان دار	پیش فرض مدرن

۳۶ «به اینترنت وصل نمی‌شوم» – ترتیب لایه‌ها: **ip -br link** (؟UP) → **ethtool** (کابل/سرعت/CRC) → **ip -br a** (آدرس/ماسک) → **ip neigh** (ARP گیت‌وی) → **ip route get 8.8.8.8** (مسیر و مبدأ) → **ping GW** سپس **ping 8.8.8.8** → **dig** / **getent** (DNS) → **ss -tlnp** و **curl -v / nc -zv** برای سرویس → هم‌زمان **tcpdump -nn -i any** روی سرور و کلاینت برای دیدن اینکه بسته می‌رسد/می‌رود → بررسی **ufw status / nft list ruleset** و **sysctl ip_forward** در سناریوی روتر.

۳۷ چرا **kill -9** راه دوم است: چون SIGKILL قابل گرفتن نیست؛ فرصت بستن فایل، آزادسازی lock، پاک‌سازی سوکت و flush داده را می‌گیرد (ریسک خرابی دیتا و قفل‌های باقی‌مانده؛ در کانتینر cgroup مشکل‌ساز). اجبار در فرایندهای در وضعیت **D** یا بی‌پاسخ به SIGTERM. بعد از آن: بررسی **journalctl / dmesg**، پاک‌کردن lock و فایل‌های موقت (در صورت نیاز)، بررسی سلامت داده، یافتن علت گیرکردن (نمودار **strace / stack**) و اقدام پیشگیرانه (timeout در **TimeoutStopSec** و پایش).

۳۸ لایه‌های کنترل دسترسی و ترتیب بررسی **Permission denied**:

۱) مجوزهای POSIX (مالک/گروه/سایر روی همان فایل و پوشه‌های والد – **namei -l** → ۲) **ACL** (**getfacl** → ۳) ویژگی‌های فایل (**lsattr : ۴** → **immutable/append-only**) **SELinux/AppArmor** (**ausearch -m avc -ts recent** , **getenforce** , **journalctl -k | grep denied** → ۵) **فایروال** (اگر خطای شبکه‌ای است نه فایلی) → ۶) **پرو فایل سرویس** (محدودیت‌های systemd: **ReadWritePaths** , **PrivateTmp** , **ProtectSystem**).

۳۹ تبدیل یک برنامه به سرویس systemd امن:

```
[Unit] Description, After=Wants=network-online.target, Requires=<db>
[Service] Type=simple, User/Group, WorkingDirectory, EnvironmentFile,
ExecStartPre, ExecStart, ExecReload(/bin/kill -HUP $MAINPID),
Restart=on-failure, RestartSec, TimeoutStopSec,
NoNewPrivileges=yes, PrivateTmp=yes, ProtectSystem=strict,
ProtectHome=yes, ReadWritePaths=..., LimitNOFILE, MemoryMax, CPUQuota
[Install] WantedBy=multi-user.target
```

رویه: کاربر سیستمی اختصاصی، مجوز دقیق روی پوشه‌های لاگ/داده، تست دستی با همان کاربر، **enable** , **systemd-analyze verify** , **Restart** تست **--now** با کشتن فرایند، تست بوت با ریboot، و مستندسازی.

۴۰ «SSH را سخت کردم و بیرون افتادم»: اشتباه: **PasswordAuthentication no** بدون اینکه کلید کار کند؛ یا محدودکردن **AllowUsers** , تغییر پورت، خرابی سینتکس و reload بدون تست، و بستن پورت در فایروال هم‌زمان. تشخیص: **journalctl -u ssh** , **sshd -t** , پیام‌های دقیق (**Permission denied (publickey)** , **no matching key exchange**). رویه‌ی پیشگیری: تست با **ssh -b** , تست در پورت دوم با یک نشست موازی (**sshd -T -f /tmp/test_config + Port 2222**), باز نگه‌داشتن یک SSH فعال تا تأیید کامل، دسترسی کنسول/VM آماده، تغییرات در پنجره‌ی نگهداری و مستندسازی، و اعتبارسنجی کلید قبل از غیرفعال‌کردن رمز (**ssh -o**). **(PreferredAuthentications=publickey)**.

۹.۴ پاسخ سناریوها (کلید راهنما)

س ۱ – دیسک پر:

```
df -hT ; df -i
du -xh --max-depth=1 / | sort -h | tail -15
du -xh --max-depth=1 /var | sort -h | tail -10
find / -xdev -type f -size +500M -exec ls -lh {} \; 2>/dev/null | sort -k5 -h | tail
ls -l +L1 # فایل حذف‌شده بار -> ری‌استارت سرویس دارند
journalctl --disk-usage ; journalctl --vacuum-size=300M
find /var/log -name "*.gz" -mtime +30 -delete
grep -E "(/var/)" /etc/logrotate.d/* -l ; logrotate -d /etc/logrotate.conf
ls -la /var/cache/apt/archives ; apt clean
```

پیشگیری: پایش فضا و inode با هشدار ۸۰٪، چرخش لاگ، سهمیه‌ی ژورنال، جداسازی پارتیشن‌های `/var` و `/var/log` و `LVM` برای توسعه‌ی سریع.

س۲ – سرور کند با `wa` بالا: علت محتمل فشار I/O؛ `iostat -xz 2 5` برای `%util` / `await` دیسک، `iotop -oPa` برای فرایند مقصر (بکاپ/دیتابیس/دیسک خراب)، `dmesg | grep -i -E "I/O error|ata"` برای خطای سخت‌افزار. اقدام: مهار (`ionice -c3` , `nice` , توقف بکاپ)، رفع (توزیع I/O، انتقال داده، تعویض دیسک) و پیشگیری (پنجره‌ی بکاپ، پایش `SMART` با `smartctl -a`).

س۳ – دسترسی تیمی:

```
sudo groupadd analytics
sudo usermod -aG analytics alice ; sudo usermod -aG analytics bob
sudo mkdir -p /srv/data ; sudo chown root:analytics /srv/data
sudo chmod 2775 /srv/data # SGID برای ارث‌بری گروه
setfacl -m g:analytics:rwX /srv/data
setfacl -m d:g:analytics:rwX /srv/data # ACL برای فایل‌های جدید
setfacl -m u:report:r-x /srv/data # کاربر فقط خواندنی
setfacl -m d:u:report:r-x /srv/data
getfacl /srv/data ; sudo -u report touch /srv/data/x # باید رد شود
```

س۴ – سرویس در بوت بالا نمی‌آید: (۱) `is-enabled` (؟disable) (۲) نبود `WantedBy` و اجرا نکردن `enable` (۳) وابستگی آماده‌نبودن `After=network-online.target` (غایب) (۴) شرطها (`ConditionPathExists/User`) (۵) خطای محیطی که در بوت رخ می‌دهد (مسیر/PATH). تشخیص: `systemctl is-enabled` , `systemctl cat` , `systemctl show -p WantedBy -p After` , `systemd-analyze verify` , `journalctl -b -u svc`.

س۵ – بازیابی رمز `root`: (الف) `GRUB` → `e` → `rw init=/bin/bash` → `passwd root` → `exec /sbin/reboot -f` ؛ (ب) `systemd.unit=rescue.target` یا `single` ؛ (ج) دیسک زنده + `chroot` و `passwd`. پیشگیری: رمز مدیریتی در گاو صندوق/سرور رمز سازمانی، ثبت تغییرات، محدودکردن دسترسی فیزیکی/کنسول.

س۶ – بکاپ طراحی‌شده:

```
# /etc/systemd/system/lpic-backup.service : Type=oneshot, User=root,
# ExecStart=/usr/local/bin/backup.sh /etc /var/backups && .../backup.sh /var/www /var/backups
# /etc/systemd/system/lpic-backup.timer : OnCalendar=*-*- 02:30, Persistent=true,
RandomizedDelaySec=600
```

داخل اسکریپت: قفل `flock`، لاگ در `/var/log/backup.log`، چرخش با `KEEP=14`، `ionice -c3 nice -n19` برای جلوگیری از فشار I/O، تست بازیابی هفتگی (`rsync --dry-run` , `tar -tzf`)، استخراج نمونه و مقایسه‌ی هش)، گزارش موفقیت/شکست به پایش.

س۷ – SSH کند: معمولاً `UseDNS yes` با DNS کند، تلاش `GSSAPIAuthentication`، مشکل `reverse lookup`، کندی `PAM` / `systemd-logind` یا `LDAP`. عیب‌یابی: `ssh -vvv` , `sshd -d -p 2223` روی پورت آزمایشی. `journalctl -u ssh -f` هم‌زمان با اتصال، تست `getent hosts <client-ip>`. رفع: `UseDNS no` , `GSSAPIAuthentication no`، اصلاح `DNS/PTR`، به‌روزرسانی سرویس‌های احراز هویت.

س۸ – `SSH` `force` `brute` تشخیص با `journalctl -u ssh | grep "Failed password" | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head`

اقدام فوری: مسدودسازی IP در فایروال (`fail2ban-client set` یا `ufw deny from X , nft add rule ... ip saddr X drop`)
`sshd banip X`، بررسی ورودهای موفق مشکوک (`grep Accepted`)، تغییر رمزها/کلیدها در صورت نفوذ. پیشگیرانه: غیرفعال کردن رمز،
 کلید + 2FA، `rate-limit` روی فایروال، `bastion` + `AllowUsers` محدود، هشدار SIEM روی نرخ شکست، به روزرسانی و پچ.

س۹ – روتر/NAT لینوکسی:

```
# اینترفیسها: lan=ens34 10.30.0.1/24 , wan=ens33 (DHCP از ISP)
sudo sysctl -w net.ipv4.ip_forward=1
sudo nft add table ip nat
sudo nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; }'
sudo nft add rule ip nat postrouting oifname "ens33" masquerade
# FORWARD: اجازه ی خروجی از lan بسته ، ورودی مرتبط ، drop بقیه
sudo nft add table inet router
sudo nft add chain inet router forward '{ type filter hook forward priority 0; policy drop; }'
sudo nft add rule inet router forward iifname "ens34" oifname "ens33" accept
sudo nft add rule inet router forward ct state established,related accept
# برای عیب یابی ICMP/ping و (بخش ۴.۶ روز ۶) ens34 روی dnsmasq با DHCP+DNS
```

نگاشت هفته ۱: `ip nat inside source ... overload = masquerade`؛ `ip nat inside source ... overload = forward` زنجیره ی `forward` =
 ACL بین اینترفیس ها؛ و اگر مسیر دینامیک لازم بود، FRR/OSPF را روی لینوکس اجرا می کنی.

س۱۰ – ۲۰۰۰ زامبی:

```
ps -eo stat,pid,ppid,cmd | awk '$1 ~ /^Z/' | head
ps -eo ppid,stat | awk '$2 ~ /^Z/{print $1}' | sort | uniq -c | sort -nr | head # والد های مقصر
```

مقصر یک فرایند والد است که فرزندانش را `wait` نمی کند (باگ آپ یا سرویس قدیمی). رفع موقت: ری استارت والد (زامبی ها به `init` می رسند و جمع می شوند). رفع ریشه ای: ارتقا/پچ آپ، افزودن `waitpid` / `SIGCHLD` handler، یا اجرای آپ زیر سوپروایزر مسئول (`systemd`) با `KillMode` / ری استارت). پایش: هشدار روی تعداد زامبی، و مستندسازی برای تیم توسعه.

۹.۵ کلید پاسخ بانک مصاحبه (۵۰ سؤال)

پاسخ های کامل این ۵۰ سؤال در بانک های روزانه ی هر موضوع آمده است؛ جای دقیق هر کدام:

گروه سؤال	جای پاسخ تشریحی
ق۱-۸ (معماری و فایل سیستم)	روز ۱ – بخش مصاحبه (۱، ۵، ۹) + جدول های طلایی
ق۹-۱۵ (مجوز و امنیت فایل)	روز ۲ – بخش مصاحبه (۱ تا ۷)
ق۱۶-۲۰ (پردازش متن)	روز ۲ – بخش ۱ و مصاحبه ۸ و ۹
ق۲۱-۲۸ (اسکریپت نویسی)	روز ۳ – بخش مصاحبه (۱ تا ۱۴)
ق۲۹-۳۷ (فرایند و زمان بندی)	روز ۴ – بخش مصاحبه (۱ تا ۱۴)
ق۳۸-۴۶ (systemd و بوت)	روز ۵ – بخش مصاحبه (۱ تا ۱۴)
ق۴۷-۵۰ (شبکه)	روز ۶ – بخش مصاحبه (۱ تا ۱۴)

کلید کوتاه (برای مرور سریع): ق۱ کرنل+فضای کاربر | ق۲ استاندارد مسیرها | ق۳ نصب خودکار در برابر دستی/جانبی | ق۴ فرایند در برابر دستگاه | ق۵ `atime/mtime/ctime` با `stat` | ق۶ متادیتای فایل، تمام شدن با فایل های کوچک پرشمار | ق۷ دقیق/لحظه ای در برابر سریع / دیتابیس | ق۸ ساخت آرگومان برای فرمان های بی پذیرش `pipe` | ق۹ کانفیگ حساس مشترک گروهی | ق۱۰ اجرا با هویت مالک در برابر اثر بری گروه | ق۱۱ حذف فقط توسط مالک در پوشه های اشتراکی | ق۱۲ `umask - 666/777` | ق۱۳ مجوز افزوده برای کاربر/گروه مشخص با `setfacl` | ق۱۴ سیاست دقیق و لاگ دار در برابر شل کامل | ق۱۵ فهرست پایه + مقایسه ی دوره ای + `chmod u-s` | ق۱۶ انتخاب خط، ویرایش خط، فیلد و محاسبه | ق۱۷ `head -n 1 | sort -nr | uniq -c | sort | awk '{print $1}'` | ق۱۸ بازنویسی در برابر افزودن؛

خطر `sort f > f` | ۱۹ انشعاب خروجی و ذخیره همزمان | ۲۰ شمارش فراوانی‌ها | ۲۱ توقف روی خطا/متغیر تعریف نشده/خطای pipe | ۲۲ هندلر خروج و خطا برای پاک‌سازی | ۲۳ خام/بسط‌دار/شکست‌واژه و glob | ۲۴ مفسر جدید/شبنگ/پوسته‌ی جاری | ۲۵ اجرای چندباره بدون تغییر وضعیت مضر | ۲۶ `shellcheck` + تست‌های سناریویی + مستند سرآغاز | ۲۷ زبان موفقیت ابزارهای خودکارسازی | ۲۸ لیست سفید + نقل‌قول + بدون `eval` | ۲۹ `fork` → `exec` → `wait` | ۳۰ تمام‌شده‌ی بی‌والد / بی‌والد / اصلاح والد و `init` | ۳۱ نسبت به تعداد هسته‌ها | ۳۲ کاربر/کرnl/انتظار I/O/دزدیده‌شده/swap | ۳۳ مؤدبانه در برابر قهری، ترتیب ۱۵ سپس ۹ | ۳۴ اولویت CPU در برابر اولویت/کلاس I/O | ۳۵ مقاوم‌سازی در برابر HUP و نشست پایدار | ۳۶ PATH/محیط، مجوز و شبنگ، مسیر مطلق و % , سرویس خاموش | ۳۷ کنترل منابع و لاگ و اجرای ازقلم‌افتاده | ۳۸ موازی‌سازی، وابستگی، journal، cgroup | ۳۹ `active/enabled` | ۴۰ `failed` | ۴۰ `Type/User/ExecStart/Restart/WantedBy` و سخت‌گیری‌ها | ۴۱ فرایند اصلی/فورک/یک‌باره | ۴۲ `override` بدون دست‌زدن به unit بسته | ۴۳ `UEFI` → `GRUB` → `kernel` → `initramfs` → ریشه/`login` → `target` → `PID1` | ۴۴ `emergency mode` و اصلاح + `nofail` | ۴۵ `init=/bin/bash` , `rescue`, `chroot` | ۴۶ سهمیه‌ی `journal` و `logrotate` | ۴۷ `systemd` / `nmcli` / `netplan` - `networkd` | ۴۸ لایه‌به‌لایه با `ip/ethtool/ip route/dig/ss/tcpdump` | ۴۹ کلیدمحور + `bastion` + `fail2ban` + `no root` + لاگ | ۵۰ `ufw/firewalld/nft` و `masquerade` با `ip_forward` .

۱۰) پایان هفته ۲

هفته ۲ تمام شد. اگر آزمون‌ها را با نمره‌ی قابل قبول رد کردی، اکنون در سه حوزه حرف داری:

۱. **مدیریت سیستم:** کاربر، مجوز، فایل‌سیستم، سرویس، بوت و لاگ.
 ۲. **خودکارسازی:** اسکریپت قابل اتکا، زمان‌بندی، گزارش و پایش.
 ۳. **شبکه‌ی میزبان:** آدرس‌دهی، عیب‌یابی، SSH امن و فایروال – همان پایه‌ای که در **هفته ۳ (CEH)** به ابزار حمله و دفاع تبدیل می‌شود.
- قدم بعد:** هفته ۳ – روز ۱ (مبانی امنیت اطلاعات: CIA Triad، سیاست‌ها، مدیریت ریسک) که مستقیماً روی مهارت‌های امروز می‌نشیند: هر کنترل امنیتی که در مصاحبه تعریف می‌کنی، باید بتوانی «در لینوکس کجا و با چه فرمانی پیاده می‌شود» را هم نشان بدهی.